

Velran

Rust szintaxisú webalkalmazás-fejlesztő nyelv és szerver, ahol a biztonság az első.

Webalkalmazás-fejlesztői kézikönyv

Zsolt Krüpl

Security-first kiadás – 2026. szeptember

Ellenőrzött fejlesztési mérföldkő

Előszó

Projektazonosság. A hivatalos magyar megfogalmazás: **Velran — Rust szintaxisú webalkalmazás-fejlesztő nyelv és szerver, ahol a biztonság az első.** A „Rust Web” szókapcsolat rövid leírásként használható, de nem külön projektnév.

Ez a könyv azoknak a webfejlesztőknek szól, akik először találkoznak az Velran nyelvvel. Nem a szintaxissal kezd, hanem magával az alkalmazással: üzleti adatokkal, engedélyezett műveletekkel és HTTP-határokkal.

A tanulási sorrend szándékos: előbb mentális modell és toolchain, utána egy kicsi end-to-end alkalmazás, majd a nyelv, adatkezelés, authentication, fájlok, API-k, security és production mélyebb részei. A CLI és a teljes `server.toml` anyag a könyv végén referencia, nem előfeltétel.

Kiadási és verifikációs állapot

Ez a kiadás a jelenlegi Velran implementáció **Ellenőrzött fejlesztési mérföldkövét** rögzíti. A jelenlegi forrásfán a megbízható fejlesztői környezetben sikeresen lefutott a `cargo fmt`, a teljes workspace tesztkészlet, a `./verify.sh`, valamint a helyi tesztkiszolgálás. A könyv nyelvi, package- és security-leírásai ezt az ellenőrzött forrásfát követik.

Ez a mérföldkő tudatosan szűkebb állítás, mint a production release-ready minősítés. A célkörnyezethez kötött deployment-, recovery- és további performance evidence, a secret-provisioning, a TLS/reverse-proxy beállítás és az operátori elfogadás továbbra is a konkrét production környezet feladata. A dokumentáció az ellenőrzött repository-baseline-t rögzíti; nem helyettesíti a production change controlt és a környezetfüggő bizonyítékokat.

Alapfogalmak

A könyv néhány angol eredetű technikai szót következetesen használ. Már az elején érdemes ugyanazt érteni alattuk:

model fordító által ismert, névvel ellátott typed adatalak; nem válik automatikusan adatbázis-táblává vagy sémává;

typed az adat alakja és típusa a fordító számára ismert, nem csak futás közben derül ki;

capability explicit hozzáférési lehetőség egy erőforráshoz vagy érzékeny művelethez, például Db, Transaction vagy AppFs;

policy a külső határon érvényes szabály, például auth, rate limit, CORS vagy trusted proxy;

runtime a lefordított Velran programot végrehajtó szerveroldali környezet, amely a limiteket és capability-határokat is kikényszeríti.

A későbbi fejezetek ezeket pontosítják, de a fenti jelentés első olvasásra elég.

Hogyan olvasd a könyvet?

Első olvasáskor ne próbáld megjegyezni az összes konfigurációs kulcsot vagy production limitet. Az első cél az, hogy három kérdésre mindig tudj válaszolni:

1. *Milyen typed adatot kezel ez a feature?*
2. *Milyen művelet olvassa vagy módosítja ezt az adatot?*
3. *Milyen authority vagy capability engedi ezt a műveletet?*

Ha ez a három kérdés tiszta, a route, a form, az SQL, az auth és a konfiguráció már ugyanannak a modellnek a különböző határai lesznek, nem egymástól független trükkök.

A könyvben végig hasznos szétválasztani, mit bizonyít a Velran, és mit kell továbbra is az alkalmazás tervezőjének eldöntenie:

Velran/platform felelőssége	Alkalmazás/operator felelőssége
Typed boundaryk, escaping, capability check, bounded erőforrások, secure session mechanika	Üzleti szabályok, ki melyik objektumon mit tehet, retention és recovery célok
Fail-closed compiler/runtime invariantok	A helyes access policy és production topológia kiválasztása
Biztonságos SQL-, redirect-, upload-, egress- és crypto-primitívek	Annak eldöntése, mikor megengedett az üzleti művelet és mi történjen domain conflict esetén

A modell security plumbingot vesz le a fejlesztőről; az alkalmazás üzleti authorization szabályait viszont nem találja ki helyette.

Válassz tanulási útvonalat

Nem kell minden fejezetet azonos prioritással olvasnod. Válassz az alábbi két út közül; később bármikor átválthatsz, ha változik a szereped vagy az alkalmazás igénye.

Gyors webfejlesztői út. Ezt válaszd, ha mielőbb hagyományos, autentikált Velran webalkalmazást szeretnél építeni és kiadni. A javasolt sorrend:

Szakasz	Fejezetek és eredmény
Mentális modell és első vertikális szelet	1–4.: értsd meg a határokat, telepítsd/futtasd az eszközöket, építs egy teljes request-to-response utat, és tanuld meg a Rust-szerű nyelvi/HTTP surface-t.
Perzisztens alkalmazás	5–7.: formok/CRUD, adatbázis-invariánsok, autentikáció és objektumszintű authorization.
Válaszd ki a szükséges webes surface-t	8., ha fájl/médiát kezelsz; 9. JSON/API munkához; 10. static asset és böngészős enhancement esetén. Ezek ágak, nem egymás kötelező előfeltételei.
Tedd megbízhatóvá	14–15.: tesztelés/hibadiagnosztika, majd az összefoglaló security-hardening modell.
Add ki és üzemeltesd	18–20.: production deployment, recovery és CLI workflow. Zárásként végezd el a 23. fejezet release-readiness checklistjét.

A 11–13. és 16–17. fejezetet addig elhalaszthatod, amíg ténylegesen nincs szükséged nagyobb modulszerkezetre, külső integrációra, cache/skálázásra vagy mélyebb observabilityre. A 15. fejezetet viszont production kiadás előtt ne hagyd ki.

Teljes Velran út. Olvasd végig sorrendben az 1–23. fejezetet. Ez az út azoknak szól, akik az alkalmazásfejlesztés mellett architektúráért, security review-ért, platformintegrációért, skálázásért, observabilityért vagy production üzemeltetésért is felelősek lesznek. A teljes út az opcionális webes feature-öket és az operátori anyagot is összefüggéseiben tanítja.

Átváltás a két út között. A gyors út nem gyengébb security mód: csak a témakörök szélességéből hagy ki, az invariantokból nem. Ha egy kihagyott capability bekerül az alkalmazásodba, implementálás előtt olvasd el az adott fejezetet, oldd meg a gyakorlatát, és használd a canonical companion példát. Bármikor folytathatod sorrendben a teljes úton anélkül, hogy újra kellene tanulnod az addigi anyagot.

Használd újra a Rust-tudásodat

A Velran szándékosan nem egy második általános célú nyelv, amelyet a Rust mellett újra meg kell tanulnod. A hétköznapi számításoknál tartsd meg a Rust mentális modelljét mindenütt, ahol a webes biztonsági contract nem igényel framework-tulajdonú szerkezetet. A visszatérő **Rust** → **Velran** blokkok három dolgot mutatnak: mi vihető át változatlanul, mit ad hozzá a framework egy határon, és mit nem érdemes külön Velran-fogalomként újratanulni.

Ha már érted a `let`, `let mut`, `struct`, `enum`, `Vec<T>`, `Option<T>`, `Result<T,E>`, `match`, metódushívás, modul és `?` hibaterjesztés alapjait, ezeket használd tovább. Az új tanulási energiát a route-okra, typed request határokra, querykre, capabilitykre, authorizationre és production policyra fordítsd. Az unsafe Rust, kézi FFI, allocator-internals, proc-macro írás és alacsony szintű async-runtime mechanika nem előfeltétele a hétköznapi Velran webfejlesztésnek.

Minimum Rust a Velranhoz

Nem kell széles körű Rust-tudás ahhoz, hogy Velran alkalmazást kezdj építeni. A gyors webfejlesztői úthoz néhány hétköznapi Rust-fogalom biztos ismerete elég. Ennek a rövid ismétlőnek nem az a célja, hogy a Rustot a Velran mellett külön megtanítsa, hanem hogy megmutassa, mikor elegendő már a meglévő Rust-intuíció.

1. Bindingok és hétköznapi értékek

Tudd olvasni az immutable bindingokat és az alkalmanként szükséges mutable lokálist. Alap esetben maradj immutable adatfolyamnál, és csak akkor használj módosítható lokálist, ha a helyi számítás ténylegesen állapotot változtat.

```
let title = "Release notes";
let mut retries = 0;
retries = retries + 1;
```

Velran webfejlesztéshez ez elég a legtöbb lokális számítás megértéséhez. Az első page vagy action megírása előtt nem kell az ownership ritka peremeseteit tanulmányoznod.

2. A négy alap adattípus: String, Vec, Option, Result

A hétköznapi alkalmazáskód nagy részét négy típus viszi:

Rust-fogalom	Mit jelent Velran alkalmazáskódban?
<code>String</code>	saját szöveges adat; HTTP-határon viszont a framework typed request típusait használd.
<code>Vec<T></code>	nulla vagy több typed érték, például query eredmény vagy renderelendő elemek.
<code>Option<T></code>	az érték jelen lehet vagy hiányozhat; a hiány explicit marad, nem magic sentinel.
<code>Result<T,E></code>	a művelet sikerülhet vagy hibázhat; a hibát tudatosan továbbadjuk vagy kezeljük.

A `?` operátort olvasd így: „ha ez hiba, add vissza most; különben folytasd a sikeres értékkel.”

```
let note = queries::loadNote(db, id)?;
let owner = note.ownerUsername;
```

Ehhez a mintához kezdő Velran kódban még nincs szükség összetett error-type tervezésre.

3. A struct és enum a domain szókincse

Structot használj, amikor több névvel ellátott mező tartozik össze. Enumot akkor, amikor a domain egy ismert, zárt állapothalmaz egyik elemét engedi.

```
struct NoteSummary {
    id: i64,
    title: String,
}

enum PublicationState {
    Draft,
    Published,
}
```

A legtöbb webes domain-adat modellezéséhez ez a Rust-tudásszint elég. A Velran lényegi kiegészítése az, ami akkor történik, amikor ezek a típusok request-, query-, authorization-, JSON- vagy persistence-határt lépnek át.

4. Match, metódushívás, modulok és ?

Tudj elolvasni egy rövid `match`-et, használd a `value.method()` metódushívást, értsd a kvalifikált modulneveket (például `queries::loadNote`), és tudd követni a `?` hibaterjesztést. Ez a négy készség lefedi a framework-owned határok közötti számítás nagy részét.

```
let label = match state {
    PublicationState::Draft => "draft",
    PublicationState::Published => "published",
};

let count = title.chars().count();
let note = queries::loadNote(db, id)?;
```

5. Borrowing: csak azt tanuld meg, amivel ténylegesen találkozol

A könyv első részéhez elég az a gyakorlati különbség, hogy egy owned érték, például a `String`, birtokolja az adatát, míg egy borrowed referencia, például a `&str`, ideiglenesen máshol birtokolt adatot néz. Ha a compiler hétköznapi compute kódban egy borrow pontosítását kéri, javítsd ezt a lokális kapcsolatot; emiatt ne szakítsd meg a webes tanulási utat teljes lifetime-elmélettel.

Framework-owned web surface-en a dokumentált Velran típusokat és signature-öket használd. Ne tervezz referencia-nehéz handler API-t csak azért, mert a Rust ezt megengedi.

6. Mit halaszthatsz későbbre?

Produktív Velran fejlesztő lehetsz az unsafe Rust, kézi lifetime-tervezés, custom allocatorok, pinning, advanced trait machinery, procedural macro-k, FFI-internals és async-runtime implementációs részletek ismerete nélkül. Ezek fontosak lehetnek, ha magát a Velran compilert vagy runtime-ot fejleszted, de alkalmazásfejlesztéshez nem előfeltételek.

Indulási önellenőrzés. Elég Rustot tudsz a gyors út elkezdéséhez, ha a fenti négy listinget el tudod olvasni, el tudod mondani az `Option` és `Result` különbségét, felismered a structot és enumot, követni tudsz egy rövid `match`-et, és érted, hogy a `?` hibát terjeszt tovább. Minden mást akkor tanulhatsz meg, amikor a könyvnek ténylegesen szüksége lesz rá.

Velran 60 perc alatt

Ezt első alkalmas vertikális szeletként használd, ne a könyv többi részének helyettesítésére. A cél, hogy egyszer összekapcsold a részeket, mielőtt külön-külön mélyebben tanulnád őket. Dolgozz az ellenőrzött Secure Notes baseline-nal: `examples/secure-notes/checkpoints/01-baseline/main.vrn`. Ha egy lépésben említett kód nem a `main.vrn` fájlban van, nézd meg a mellette lévő modulokat is.

Menet	Fókusz	Mit figyelj meg?
1	Page és route	Keresd meg a page handlert és az azt kivezető route-ot. Ellenőrizd, hogy a hozzáférés explicit, nem implicit.
2	Typed input és form	Kövess végig egy formértéket a request boundarytól a typed alkalmazáskódig. Azonosítsd a hossz-/erőforrás-korlátját.
3	Adatbázis-olvasás és -írás	Kövess végig egy queryt és egy mutationt. Figyeld meg a typed bind értékeket, a transaction scope-ot, valamint az SQL-szerkezet és a felhasználói adat szétválasztását.
4	Authentication és authorization	Keresd meg a trusted principalt, majd a külön ownership/authorization ellenőrzést a protected read vagy mutation előtt.
5	Response	Kövess vissza a sikeres utat HTML-ig vagy typed redirectig. Kapcsold a request lifecycle-hoz: parse, validate, authenticate, authorize, effect, response.
6	Ellenőrzés és magyarázat	Futtasd a <code>velran-cli check</code> parancsot a <code>examples/secure-notes/checkpoints/01-baseline/main.vrn</code> fájlban. Ezután mondd meg, melyik boundary utasítana el egy unsafe változatot, mielőtt új feature-t adnál hozzá.

A 60 perces sikerfeltétel. Ne azt mérd, mennyi szintaxist jegyeztél meg. Akkor lépj tovább, ha egy requestet végig tudsz követni, felismered, hol változik a trust, meg tudsz tenni egy kicsi biztonságos módosítást, és a compiler/verifier eredménye alapján el tudod dönteni, mit kell javítani. Ha valamelyik lépés még idegen, az 1–7. fejezettel folytasd, ne advanced Rust kitérővel.

Tíz megjegyzendő biztonsági invariáns

A könyv az alábbi szabályokat szándékosan ismétli. Külső határnál kezeld őket rövid mentális checklistként:

1. minden route explicit access policyt kap;
2. a külső inputot protected effect előtt parse-oljuk, normalizáljuk, validáljuk és korlátozzuk;
3. az SQL szerkezete compiler-owned, felhasználói adat csak typed bind paraméterként kerülhet bele;
4. az authentication azt bizonyítja, ki a hívó; az authorization azt, hogy mit tehet, és ennek a protected effect előtt kell megtörténnie;
5. a kliens fájlneve és MIME-deklarációja metaadat, nem bizalmi bizonyíték;
6. a CORS böngészős origin-policy, nem authentication vagy authorization;
7. kifelé hálózati hozzáférés csak explicit, névvel ellátott és konfigurált capabilityn keresztül létezik;
8. security szabályhoz negatív bizonyíték is kell: a tiltott változatnak fail-closed módon el kell buknia;
9. a log és trace nem válhat secretok vagy request bodyk második példányává;
10. production candidate csak fordítás és validálás után aktiválható; hiba esetén a known-good generáció marad aktív.

Amikor egy szabály újra megjelenik valamelyik fejezetben, **Biztonsági invariáns** blokk jelöli. Az ismétlés szándékos: a cél az, hogy implementálás közben automatikusan felidézhető legyen.

A könyv konvenciói

- A `main.vrn` az alkalmazás belépési pontja.
- A production konfiguráció elsődleges felülete a trusted TOML fájl.
- Secret érték nem kerül parancssorba vagy plaintext TOML mezőbe; a konfiguráció `*_file` hivatkozásokat használ.
- SQL szöveg compiler-owned, felhasználói érték csak typed bind paraméterként kerülhet lekérdezésbe.
- HTML-ben a `{ ... }` interpoláció escape-elt; raw HTML escape hatch nincs.
- Üzleti módosítás transactionben történik.

A könyv a V1 surface-t célozza. Ha egy capability nincs benne ebben a surface-ben, azt nem kerülőúttal kell megvalósítani, hanem külön nyelvi/runtime feature-ként.

A kiadás hatóköre és a source of truth

Ez a kiadás a jelenlegi Rust-first, native-only Velran surface-t írja le. A normál compute szintaxis tudatosan Rust-szerű mindenütt, ahol a webes security modell nem igényel framework-owned konstrukciót: az attribútumos callable-ok `#[page] fn`, `#[action] fn` és `#[query] fn` alakúak; a kollekciók `Vec<T>` és `Option<T>` típusokat használnak; a módosítható compute lokális `let mut` deklarációt és közvetlen assignmentet kap; a lebegőpontos kód `f32`-t és például `x.sin()` metódusszintaxist használ. Ha egy konstrukció nem lowerelhető biztonságosan natív kódra, a fordítás fail-closed hibával leáll; alkalmazáskódhoz nincs bytecode/VM végrehajtási fallback.

A `examples/` alatt található futtatható példák és a `docs/` feature-szintű leírásai jelentik a copy/paste és részletes technikai source of truth-ot. A könyv listingjei tanítási célúak, ezért rövidíthetnek környező setupot, de ugyanazt az elfogadott nyelvi surface-t kell használniuk. A repository tudatosan külön kezeli az olvasásra és adaptálásra szánt programokat a csak verifikációs anyagoktól: az `examples/` tanulható alkalmazásokat tartalmaz, a `tests/fixtures/` fail-closed rejection/security eseteket, a `tests/manifests/` pedig verification metaadatokat. Szintaxis- vagy security-invariáns változásakor a canonical példákat, szükség esetén a fixture-öket és ezt az angol/magyar könyvpárt együtt kell frissíteni.

Fokozatos gyakorlási görbe

A gyakorlatok négy lépcsőben veszik el a segítséget. A későbbi *Gyakorlási mód* címkék definíciója ez:

Vezetett, 1–4. fejezet. Azonosítsd a határt vagy invariantot, végezd el a legkisebb módosítást, majd hasonlítsd össze az ellenőrzött companion példával. Akkor kész, ha el tudod magyarázni a változást, a védő határt és egy elutasított hibát.

Támogatott, 5–10. fejezet. Előbb próbáld meg a companion nélkül; ha elakadsz, az implementáció előtt csak az interfészt, policyt vagy adatformát nézd meg. Akkor kész, ha a funkció működik, a security invariant explicit, és egy negatív változat elbukik.

Önálló, 11–17. fejezet. Követelményként kezeld a feladatot, tervezd és implementáld önállóan, majd vedd össze a canonical példával. Akkor kész, ha a tervet és a sikeres/hibás útvonalakat meg tudod védeni.

Üzemeltetési szcenárió, 18–23. fejezet. Nevezd meg a veszélyeztetett invariantot, gyűjts bizonyítékot, válaszd a legkisebb biztonságos beavatkozást, és előre határozd meg a rollbacket vagy recoveryt. Akkor kész, ha egy másik operátor követni tudná a bizonyítékot és az utólagos feljegyzést.

Ne a jelenlegi feladat leggyorsabb befejezésére, hanem arra optimalizálj, hogy a következőhöz kevesebb segítség kelljen.

Tartalomjegyzék

Előszó	ii
I. Első lépések: gondolkodásmód és működő alkalmazás	1
1. Hogyan gondolkodjunk egy Velran alkalmazásról?	2
1.1. Először a fogalmak, utána az oldalak	2
1.2. Mit jelent a model?	3
1.3. A model a rétegek közti szerződés	3
1.4. Az üzleti fogalomhoz műveletek tartoznak	4
1.5. Capability: a hozzáférés is része a modellnek	4
1.6. A route nem az üzleti modell	4
1.7. Egy egyszerű tervezési módszer	5
1.8. Mit vigyél tovább a következő fejezetekbe?	5
1.9. Gyakorlat: rajzold fel a határtérképet	5
1.10. Tipikus hiba: a route-okból indulunk ki a modell helyett	5
1.11. Folyamatos mintaprojekt: Secure Notes	6
2. Telepítés, binárisok és futtatási topológiák	7
2.1. Build a forrásból	7
2.2. A binárisok telepítési helye	8
2.3. Első alkalmazás	9
2.4. A konfigurációból elsőre csak a minimum kell	10
2.5. Production process és reverse proxy	10
2.6. Futtatási topológiák részletesen	11
2.7. Közvetlen, egydomaines HTTPS	11
2.8. Apache reverse proxy mögött	12
2.9. Nginx reverse proxy mögött	13
2.10. Ajánlott production könyvtárstruktúra	13
2.11. Első ellenőrzés	14
2.12. Gyakorlat: ellenőrizd a telepítési határokat	14
2.13. Tipikus hiba: a konfigurációt alkalmazásadatként kezeljük	14
2.14. Folyamatos mintaprojekt: Secure Notes	14
3. A-tól Z-ig: egy minimális webalkalmazás	15
3.1. A request életciklusa: egyetlen mentális modell a legtöbb webes feladathoz	16
3.2. A könyvtár létrehozása	16
3.3. Az adatbázis-séma	16
3.4. A model és a lekérdezések	17
3.5. A GET oldal	17

3.6.	A POST action	18
3.7.	A route-ok és a validáció	19
3.8.	A teljes alkalmazás egy helyen	19
3.9.	SQLite kapcsolat developmenthez	19
3.10.	Forrásellenőrzés és migration	19
3.11.	Minimális development konfiguráció	20
3.12.	Az első HTTP kérés	20
3.13.	Mit történik hibás inputnál?	21
3.14.	Developmentből production felé	21
3.15.	Mit tanultunk ebből az egy példából?	21
3.16.	Ellenőrzött companion példa	22
3.17.	Gyakorlat: kövess végig egy kérést	22
3.18.	Tipikus hiba: összekeverjük az olvasási és módosítási folyamatot	22
3.19.	Folyamatos mintaprojekt: Secure Notes	22
 II. Alap webalkalmazás-fejlesztés		23
 4. Nyelvi alapok és HTTP-határok		24
4.1.	Forrásstruktúra és statement-határok	24
4.2.	Typed számítás webalkalmazás-kódban	27
4.3.	Domainértékek és handlerkód	30
4.4.	HTTP- és prezentációs határ	32
4.5.	Gyakorlat: dönts el, kié a szintaxis	33
4.6.	Tipikus hiba: kényelmi szintaxist találunk ki	33
4.7.	Folyamatos mintaprojekt: Secure Notes	34
 5. Adatbevitel: typed SQL, CRUD, formok és PRG		35
5.1.	A request-életciklus alkalmazása CRUD módosításra	35
5.2.	Typed SQL és CRUD	36
5.3.	Formok, validáció és PRG	37
5.4.	Teljes szerkesztési ciklus: validáció, authorization, konkurencia	39
5.5.	Adatbázis-invariantok és felhasználóbarát validáció	40
5.6.	Mutációs review-checklist	40
5.7.	Ellenőrzött companion példák	40
5.8.	Gyakorlat: tervezz szerkesztési ütközést	40
5.9.	Tipikus hiba: csak a tranzakció előtt validálunk	41
5.10.	Folyamatos mintaprojekt: Secure Notes	41
 6. Adatbázis mélyebben: tranzakció, migráció és konkurencia		42
6.1.	Három külön felelősség	42
6.2.	A query visszatérési típusa szerződés	43
6.3.	Bindolj, ne SQL-t építs	44
6.4.	Olvasás és mutáció authority-határa	45
6.5.	Optimistic locking: ne írd felül más munkáját	46
6.6.	Indexet a hozzáférési mintához tervezz	47
6.7.	Migration: a séma a release része	47
6.8.	Expand/contract: tartsd meg a rollback lehetőségét	48
6.9.	Adatbázis-hiba kezelése	49
6.10.	SQLite, PostgreSQL vagy MariaDB?	49
6.11.	Backup szempont adatbázis-fejlesztőként	49

6.12.	Egy CMS adatbázis-rétegének mintája	50
6.13.	Fejezet végi ellenőrzőlista	50
6.14.	Gyakorlat: kezeld a query-t interfészként	50
6.15.	Tipikus hiba: az adatbázist csak tárolónak tekintjük	51
6.16.	Folyamatos mintaprojekt: Secure Notes	51
7.	Autentikáció, session és jogosultságkezelés	52
7.1.	Hol helyezkedik el az authentication és authorization az életciklusban?	52
7.2.	A teljes login útvonal	53
7.3.	Local auth: külön identity adatbázis	53
7.4.	Account lifecycle operátori CLI-val	54
7.5.	Session invalidation: auth_generation	54
7.6.	Session store: memory vagy Redis	55
7.7.	Cookie policy	55
7.8.	Logout	55
7.9.	LDAP / LDAPS vállalati identityhez	56
7.10.	TOTP és recovery code	56
7.11.	Route authorization	56
7.12.	A principal használata az oldalon	57
7.13.	Route role nem mindig elég	57
7.14.	Login rate limiting	58
7.15.	CSRF, Origin és Fetch Metadata	58
7.16.	Auth audit	58
7.17.	Local auth vagy LDAP?	59
7.18.	Rich text és autentikált CMS	59
7.19.	Production ellenőrzőlista	59
7.20.	Ellenőrzött companion példák	60
7.21.	Gyakorlat: készíts autorizációs mátrixot	60
7.22.	Tipikus hiba: az authenticationt authorizationnek tekintjük	60
7.23.	Folyamatos mintaprojekt: Secure Notes	60
8.	Fájlok, feltöltés és médiatár	61
8.1.	A mentális modell: három külön dolog	61
8.2.	Általános fájlfeltöltés: Upload	61
8.3.	AppFs: az alkalmazás írható fájltere	63
8.4.	Mi történik egy feltöltés során?	63
8.5.	Képfeltöltés: preferáld az Image típust	64
8.6.	Byte-limit és pixel-limit nem ugyanaz	64
8.7.	Kép referencia az adatbázisban	64
8.8.	Biztonságos megjelenítés HTML-ben	66
8.9.	A beépített media endpoint	66
8.10.	CMS-példa: hero kép feltöltése	66
8.11.	Adatbázis és AppFs együtt alkothat üzleti állapotot	67
8.12.	Statikus asset és feltöltött fájl nem ugyanaz	67
8.13.	Mit ne csinálj?	67
8.14.	A jelenlegi media surface határai	67
8.15.	Fejlesztői ellenőrzőlista feltöltéshez	67
8.16.	Ellenőrzött companion példák	67
8.17.	Gyakorlat: threat-modellezz egy feltöltést	68
8.18.	Tipikus hiba: uploadot teszünk a static root alá	68
8.19.	Folyamatos mintaprojekt: Secure Notes	68

9.	JSON API, CORS és böngészős kliensintegráció	69
9.1.	Ugyanez az életciklus érvényes az API-kra is	70
9.2.	Az első typed JSON endpoint	70
9.3.	Adatbázisból JSON lista	71
9.4.	Typed JSON POST body	72
9.5.	Content negotiation és a 406 válasz	72
9.6.	API hibák	73
9.7.	Same-origin: az alapértelmezett és legegyszerűbb modell	73
9.8.	CORS: exact origin allowlist	74
9.9.	Preflight	74
9.10.	Credentialed CORS session-cookie-val	74
9.11.	CSRF JSON API-n	75
9.12.	Teljes böngészős POST folyamat	75
9.13.	Reverse proxy és origin	76
9.14.	Mit ne tegyél?	76
9.15.	Melyik architektúrát válaszd?	76
9.16.	Kipróbálható példa	76
9.17.	Ellenőrzött companion példák	76
9.18.	Gyakorlat: válaszd külön az API- és a CORS-tervet	77
9.19.	Tipikus hiba: alapból permissive CORS-t kapcsolunk be	77
9.20.	Folyamatos mintaprojekt: Secure Notes	77
10.	Statikus assetek és frontend-integráció	78
10.1.	Három külön fájléletciklus	78
10.2.	A static root konfigurálása	79
10.3.	Fingerprint: a hosszú cache kulcsa	79
10.4.	Egy minimális asset build pipeline	80
10.5.	ETag és conditional GET	80
10.6.	Precompressed Brotli és gzip	80
10.7.	MIME, nosniff és path confinement	81
10.8.	CSS, kép és font a Velran HTML-ben	81
10.9.	Fontos V1-határ: JavaScript kiszolgálható, de Velran HTML-ben nem fut	81
10.10.	Velran szolgálja ki vagy Apache/Nginx?	82
10.11.	Nginx minta közvetlen static kiszolgálásra	82
10.12.	Apache minta közvetlen static kiszolgálásra	83
10.13.	Deploy: asset és alkalmazáskód ugyanahhoz a release-hez tartozzon	83
10.14.	Ellenőrzés curl-lel	84
10.15.	Gyakorlati döntési szabály	84
10.16.	Ellenőrzött companion példa	84
10.17.	Gyakorlat: osztályozd a frontend minden fájlját	84
10.18.	Tipikus hiba: a frontend-kényelem eltünteti a trust boundaryt	85
10.19.	Folyamatos mintaprojekt: Secure Notes	85
11.	Nagyobb alkalmazások szervezése	86
11.1.	Három külön kérdés	86
11.2.	A mod nem szöveges include	86
11.3.	Mikor válts feature-alapú szerkezetre?	87
11.4.	Domain object: üzleti névtér, nem OOP-osztály	88
11.5.	Package-határok újrahasznosítható alkalmazáskódhoz	89
11.6.	A dependency továbbra is explicit	89
11.7.	Miért maradjon a route top-level?	90

11.8.	Ajánlott névkonvenció	90
11.9.	Egy CMS lehetséges szerkezete	90
11.10.	Mit ne csinálj?	91
11.11.	Refaktorálási út: egy fájlból nagyobb alkalmazás	91
11.12.	Ellenőrzött companion példák	92
11.13.	Gyakorlat: refaktorálj változási lokalitás alapján	92
11.14.	Tipikus hiba: csak a fájlszám miatt modularizálunk	92
11.15.	Folyamatos mintaprojekt: Secure Notes	92
III. Alkalmazott fejlesztés, integráció és assurance		93
12.	Gyakorlati alkalmazások: wiki és CMS	94
12.1.	Egyszerű wiki	94
12.2.	CMS és híroldal	95
12.3.	Szerveroldali Markdown renderelés és cache	97
12.4.	Demo CMS-től a szerkesztőségi workflow-ig	98
12.5.	Gyakorlati CMS-rétegmodell	99
12.6.	CMS review-kérdések	99
12.7.	Ellenőrzött companion példák	99
12.8.	Gyakorlat: definiálj editorial state machine-t	99
12.9.	Tipikus hiba: a workflow-t szabad szöveggént tároljuk	100
12.10.	Folyamatos mintaprojekt: Secure Notes	100
13.	Külső szolgáltatások és egress policy	101
13.1.	A jelenlegi Velran-határ	101
13.2.	Miért nem reprezentálható arbitrary URL?	102
13.3.	Named target: a hálózati authority konfigurálása	102
13.4.	A policy több réteget ellenőriz	103
13.5.	IPv4 és IPv6 ugyanazt az authority modellt követi	103
13.6.	Typed outbound műveletek	103
13.7.	Méret-, protokoll- és időkorlátok	104
13.8.	Secreték külső szolgáltatáshoz	104
13.9.	Retry, idempotencia és exceptional outcome	104
13.10.	A webhook az ellenkező irányú boundary	104
13.11.	Szinkron request vagy háttérmunka?	105
13.12.	Minimum tesztelés	105
13.13.	A fejezet lényege	105
13.14.	Ellenőrzött companion példa	105
13.15.	Gyakorlat: írd egress contractot	105
13.16.	Tipikus hiba: tetszőleges kimenő destinationt engedünk	105
13.17.	Folyamatos mintaprojekt: Secure Notes	106
14.	Tesztelés, hibamodellek és fejlesztői diagnosztika	107
14.1.	Security diagnostic olvasása négy menetben	107
14.2.	Az első kapu: <code>velran-cli check</code>	108
14.3.	A negatív teszt ugyanolyan értékes, mint a pozitív	108
14.4.	A hibák négy külön világa	109
14.5.	Konfigurációt külön tesztelj	109
14.6.	Migráció: status, verify, apply, verify	109
14.7.	Üzleti teszt: állapotot bizonyíts, ne implementációt	110

14.8.	Adatbázisvizsgálat: SQLite nem bizonyít mindent	110
14.9.	Auth és authorization tesztmátrix	110
14.10.	Fájl, média, API és cache tesztek	111
14.11.	HTTP hibakód nem egyenlő programhibával	111
14.12.	Exceptional state-ek exhaustív tesztelése	112
14.13.	Request ID-val hibakeresni	113
14.14.	Deployment smoke teszt	113
14.15.	Mit jelent a repository <code>verify.sh</code> ?	113
14.16.	Ajánlott fejlesztői ciklus	114
14.17.	Hibakeresési döntési sorrend	114
14.18.	Minimum release checklist alkalmazáshoz	114
14.19.	Gyakorlat: vezess le negatív teszteket a határokból	115
14.20.	Tipikus hiba: csak sikeres handlereket tesztelünk	115
14.21.	Folyamatos mintaprojekt: Secure Notes	115
15.	Security hardening: a jelenlegi secure-by-construction modell	116
15.1.	Mit dönt el a secure modell – és mit nem?	116
15.2.	Authority és access control	116
15.3.	Critical operation, idempotency és transaction outcome	117
15.4.	Input, injection és browser boundaryk	117
15.5.	Trust, domain típusok és disclosure	117
15.6.	Authentication és session hardening	118
15.7.	Credential, token és kriptográfia	118
15.8.	Verified webhook és safe upload	118
15.9.	Resource safety mint security invariant	118
15.10.	A production konfiguráció security contract	119
15.11.	Supply-chain state a release része	119
15.12.	Security event, redaction és alert	119
15.13.	Production security gate	120
15.14.	Öt review-kérdés minden új feature-höz	120
15.15.	Ellenőrzött companion példák	120
15.16.	Gyakorlat: rendelj kontrollokat threat pathokhoz	120
15.17.	Tipikus hiba: a hardeninget végső deployment fázisnak tekintjük	120
15.18.	Folyamatos mintaprojekt: Secure Notes	121
IV.	Production és üzemeltetés	122
16.	Cache, Redis, rate limiting és többpéldányos működés	123
16.1.	Miért nem általános Redis kliens?	123
16.2.	Production Redis konfiguráció	123
16.3.	Shared session több szerverpéldánnyal	124
16.4.	Public page cache	124
16.5.	Mit szabad public cache-be tenni?	125
16.6.	Operator által szabott cache-határok	125
16.7.	Cache invalidation: generation, nem kulcskeresés	125
16.8.	Stampede védelem és annak határa	126
16.9.	Redis hiba cache-elt request közben	126
16.10.	Route rate limiting	126
16.11.	Rate-limit scope-ok	127
16.12.	Fixed-window modell több instance-nál	127

16.13.	Memory backend csak fejlesztési kivétel	128
16.14.	Egy tipikus többpéldányos topológia	128
16.15.	Rolling restart és cache generation	128
16.16.	Mit monitorozz?	129
16.17.	Redis recovery szerepe	129
16.18.	Gyakorlati példa: cache-elt híroldal publikálással	129
16.19.	Ellenőrzőlista	130
16.20.	Gyakorlat: osztályozd a cache-elhető adatot	130
16.21.	Tipikus hiba: consistency előtt cache-elünk	130
16.22.	Folyamatos mintaprojekt: Secure Notes	130
17.	Naplózás, audit, metrics és hibakeresés	131
17.1.	A három naplóosztály szerepe	131
17.2.	Request ID: egy kérés végigkövetése	132
17.3.	Mit szabad és mit tilos logolni?	132
17.4.	Security audit és HTTP elutasítások	133
17.5.	Business audit: tartós üzleti változástörténet	133
17.6.	Platform security event, redaction és burst alert	134
17.7.	Prometheus metrics külön management socketen	134
17.8.	Apache vagy Nginx mellett ki scrape-elje a metricset?	135
17.9.	Health: liveness és readiness nem ugyanaz	136
17.10.	Logrotate és SIGHUP	136
17.11.	Mi történik, ha a logfájl nem írható?	137
17.12.	Gyakorlati production hibakeresési folyamat	137
17.13.	Minimum monitoring baseline	137
17.14.	Retention, SIEM és auditvédelem	138
17.15.	Ellenőrzőlista	138
17.16.	Gyakorlat: tervezz evidence trailt	138
17.17.	Tipikus hiba: események helyett adatot logolunk	138
17.18.	Folyamatos mintaprojekt: Secure Notes	139
18.	Production deployment és service lifecycle	140
18.1.	Az alkalmazás production contractja	140
18.2.	Release útvonal	141
18.3.	systemd service	141
18.4.	Controlled restart és readiness	141
18.5.	Migration és rollback nem ugyanaz	142
18.6.	Recovery gate a deployment előtt	142
18.7.	Deploy utáni minimum ellenőrzés	142
18.8.	Release-invariantok	142
18.9.	Mi reloadolható és mi igényel restartot?	143
18.10.	Candidate aktiválás: az operátori szabály	143
18.11.	Pre-deploy evidence	144
18.12.	Deployment anti-patternök	144
18.13.	Gyakorlat: próbálj el egy sikertelen aktiválást	144
18.14.	Tipikus hiba: a build sikerét deployabilitynek tekintjük	144
18.15.	Folyamatos mintaprojekt: Secure Notes	144
19.	Backup, restore és disaster recovery	145
19.1.	Először készíts state inventoryt	145
19.2.	Mi nem üzleti source of truth?	145

19.3.	RPO és RTO: a technikai mentés előtt üzleti döntés	146
19.4.	A V1 biztonságos baseline: kontrollált maintenance backup	146
19.5.	Application DB: a backend saját backup eszközét használd	146
19.6.	A local-auth DB külön security backup	147
19.7.	AppFs: a teljes data root tartós állapot lehet	147
19.8.	Minden recovery ponthoz tartozzon manifest	148
19.9.	Restore drill: a backup bizonyítása	148
19.10.	Upgrade előtt legyen recovery döntés	148
19.11.	Expand/contract tartja nyitva a gyors rollback ablakot	149
19.12.	Rollback és restore két külön művelet	149
19.13.	Tiltott rövidítések	150
19.14.	Production recovery checklist	150
19.15.	Gyakorlat: restore-t bizonyíts, ne backupot	150
19.16.	Tipikus hiba: komponenseket mentünk közös recovery modell nélkül	150
19.17.	Folyamatos mintaprojekt: Secure Notes	151
V. Referencia és operátori gyorssegédlet		152
20. A Velran CLI és a napi fejlesztői/üzemeltetői workflow		153
20.1.	Melyik feladathoz melyik program?	153
20.2.	A CLI nem a szerver alternatívája	154
20.3.	Az első parancs: <code>velran-cli check</code>	154
20.4.	Forrásfából futtatás és telepített CLI	154
20.5.	Migration workflow	155
20.6.	Local-auth adminisztráció	155
20.7.	TOTP adminisztráció különösen érzékeny művelet	156
20.8.	A secret fájl is input contract	157
20.9.	Server preflight: a CLI után a szerver saját ellenőrzése jön	157
20.10.	Napi fejlesztői ciklus	157
20.11.	Application release workflow	158
20.12.	A Velran platform release workflowja más	158
20.13.	Automatizálás: read-only és mutáló lépések legyenek külön	158
20.14.	Gyakori hibák	159
20.15.	Operátori emlékeztető	159
20.16.	Gyakorlat: a biztonságos út legyen a legrövidebb	159
20.17.	Tipikus hiba: megjegyzett parancssorokra támaszkodunk	160
20.18.	Operátori checkpoint: Secure Notes	160
21. A <code>server.toml</code> konfigurációs referencia		161
21.1.	Betöltés, precedence és fail-closed szabályok	161
21.2.	Hogyan olvasd a táblákat?	162
21.3.	[server] és [tls]	162
21.4.	[database], [redis] és [auth]	163
21.5.	[web]	164
21.6.	[storage] és [static_assets]	164
21.7.	[lifecycle], [observability] és [logging]	165
21.8.	[rate_limit] és [cache]	165
21.9.	[native]	166
21.10.	[reload]	166
21.11.	[limits]	167

21.12.	[cgroup]	168
21.13.	Production baseline	168
21.14.	Automatikus source reload	169
21.15.	Változtatási workflow	169
21.16.	Gyakorlat: trust domain szerint review-zd a konfigurációt	169
21.17.	Tipikus hiba: nagy konfigurációt másolunk és csak a nyilvánvaló értékeket cseréljük	170
21.18.	Operátori checkpoint: Secure Notes	170
22.	Fejlesztői referencia és továbblépés	171
22.1.	Canonical dokumentációs térkép	171
22.2.	Napi secure fejlesztési checklist	172
22.3.	Hogyan használd a dokumentációs készletet?	172
22.4.	Feladatorientált olvasási útvonalak	173
22.5.	Futtatható companion példák	173
22.6.	A példák hitelességének megőrzése	174
22.7.	Gyakorlat: válaszd ki a megfelelő authority-t	174
22.8.	Tipikus hiba: dokumentációs konfliktust találgatással oldunk fel	175
22.9.	Olvasói checkpoint: Secure Notes	175
A.	Függelékek és production checklist	176
A.1.	Aktuális verifikációs megjegyzés	176
A.2.	Ajánlott production könyvtárstruktúra	176
A.3.	Apache reverse proxy teljes minimálminta	177
A.4.	Nginx reverse proxy teljes minimálminta	178
A.5.	systemd service minta	178
A.6.	logrotate minta	179
A.7.	Release előtti production gate	180
A.8.	Gyors hibadiagnosztikai térkép	181
A.9.	Mit vigyél magaddal productionbe?	182
A.10.	Velran webfejlesztői kompetencia-checklist	182
A.11.	Gyakorlat: evidence-gyűjtésként futtasd a checklistet	184
A.12.	Tipikus hiba: a checklistet engineering judgment helyettesítőjeként kezeljük	184
A.13.	Végső Secure Notes checkpoint	184

I. rész

Első lépések: gondolkodásmód és működő alkalmazás

1. fejezet

Hogyan gondolkodjunk egy Velran alkalmazásról?

Tanulási út: Gyors út: alap. Építsd fel azt a mentális modellt, amelyre minden későbbi fejezet épül.

Rust → Velran

Hozd Rustból: A structok, enumok, `Option`, `Result` és explicit függvényparaméterek továbbra is ismerős typed építőelemek.

A Velran hozzáadja: A model compiler számára láthatóvá teszi az alkalmazás adatstruktúráját, a capability pedig explicitté teszi a külső erőforrásokhoz tartozó authorityt.

Nem kell újratanulnod: Nem kell új objektummodellt megtanulnod. Maradj a typed értékeknél és explicit bemenetekenél; azt tanul meg, hol tesz köréjük a Velran security-határt.

Route vagy adatbázis előtt ezt kérdezd: *milyen üzleti adat létezik, milyen alakban, milyen műveletek végezhetők rajta, és milyen határok között?* A Velran több biztonsági és helyességi garanciája abból származik, hogy ezek az adatalakok, inputok, query-k és erőforrás-hozzáférések explicit szerződések.

1.1. Először a fogalmak, utána az oldalak

Egy terméklistánál indulj a *Product* fogalomból, annak typed alakjából, az olvasó/módosító műveletekből, a szükséges capability-kből és policy-kből, és csak ezután a HTTP felületből. A gondolkodás iránya nagyjából:

```
üzleti fogalom
  -> typed adatalak
    -> muveletek
      -> capability-k es policy-k
        -> HTTP route
          -> HTML / JSON
```

Ez a sorrend segít, hogy az üzleti szabályok ne ad hoc route-okba, template-ekbe vagy összefűzött SQL-be szóródjanak.

1.2. Mit jelent a model?

A könyvben két közeli fogalommal találkozunk:

- **adatmodell vagy domain modell:** annak emberi leírása, milyen üzleti fogalmak és kapcsolatok léteznek az alkalmazásban;
- **model deklaráció:** a Velran konkrét, fordító által ismert adatalakja.

Például:

```
model Product {
  id: i64
  name: String
  price: i64
}
```

Ez azt mondja a fordítónak, hogy egy `Product` rekordnak pontosan ilyen mezői és típusai vannak. A typed SQL query eredményalakja ehhez a szerződéshez igazodik, és ugyanez az érték később HTML- vagy JSON-kimenetben is használható.

A model nem automatikus adatbázisséma

A fenti deklaráció önmagában nem hoz létre `products` táblát. Az adatbázissémát migration hozza létre, például:

```
CREATE TABLE products (
  id INTEGER PRIMARY KEY,
  name TEXT NOT NULL,
  price INTEGER NOT NULL
);
```

A rétegek kapcsolódnak, de külön felelősségük van: a migration kezeli a tartós sémát, a `model` az alkalmazás typed adatszerződése, a query pedig explicit módon képez sorokat modelltől. A model ezért nem hoz létre automatikusan adatbázis-entitást vagy azonos nevű táblát.

A model nem mutable objektum

A `Product` nem klasszikus objektumorientált osztály. Nincs mögötte rejtett `save()` metódus, implicit adatbázis-kapcsolat vagy mutable objektuméletciklus. Ha adatot akarunk módosítani, explicit műveletet és explicit erőforrást használunk.

Itt még az aláírás a lényeg: a `listProducts` Db olvasási hozzáférést kap, a `createProduct` pedig explicit `Transaction` erőforrást a módosításhoz. A teljes SQL-törzseket a 3. fejezet mutatja meg, amikor már a teljes request útvonalat látjuk.

1.3. A model a rétegek közti szerződés

A `model` a rétegek között mozgó typed szerződés.

```
Database row
| typed query
v
Product model
|-----|
v                v
```

HTML page	JSON API
v	
felhasználó	

Ha egy query `Product`-ot ígér, a visszaadott mezőknek ehhez az alakhoz kell igazodniuk. Ugyanez a typed rekord-szerződés tér vissza később az object authorizationnál és a JSON-kimenetnél.

1.4. Az üzleti fogalomhoz műveletek tartoznak

Az üzleti adathoz műveletek tartoznak. Ilyen a `Product.list` vagy az `Article.publish`. Kis programban ezek lehetnek top-level függvények; nagyobb programban a később bemutatott object namespace csoportosíthatja őket. Fontos különbség:

Elem	Kérdés, amelyre válaszol
<code>model</code>	Milyen alakú adat a <code>Product</code> ?
<code>#[query] fn</code>	Hogyan olvassuk vagy módosítjuk a tartós adatot?
<code>object</code>	Mely üzleti fogalomhoz tartoznak ezek a műveletek?
<code>#[page] fn/#[action] fn</code>	Hogyan kapcsolódik az üzleti művelet a webes handlerhez?
<code>route</code>	Milyen HTTP útvonalon, inputtal és policy-val érhető el?

1.5. Capability: a hozzáférés is része a modellnek

Velranban nem csak az adat típusa explicit, hanem az is, milyen erőforráshoz fér hozzá egy művelet. Egy függvény nem „valahonnan” kap adatbázist, fájlrendszert vagy current usert.

```
#[page]
fn home(ctx: PageContext, db: Db)
  -> Result<Html, PageError> {
  let products = listProducts(db)?;
  return Ok(html {
    <ul>
      @for product in products {
        <li>{{ product.name }}</li>
      }
    </ul>
  });
}
```

A `db: Db` az aláírásban látható. Fájlkezelésnél `AppFs` capability, módosító adatbázis-műveletnél `Transaction`, autentikált műveletnél pedig az `auth` policy és a hozzá tartozó typed kontextus válik relevánssá.

Tervezési szabály: ha egy műveletnek érzékeny erőforrás kell, annak a kódból és a policy-ból látszania kell.

1.6. A route nem az üzleti modell

A route fontos, de a rendszer külső határa. Például:

```
route create POST "/products"
```

```
form name<String> price<i64>
validate name length 1 100 price range 0 100000000
public => create;
```

A route a külső HTTP-határt írja le: metódus, path, typed input, validáció és policy. Maga a `Product` ettől függetlenül HTML-, JSON-, admin- vagy más route mögött is újrahasználható.

Ezért nagyobb alkalmazásban különösen hasznos a következő határ:

```
domain/model      : mi az adat es milyen muvelet ertelmes rajta?
HTTP route        : ki, hogyan es milyen policy-val hivhatja?
runtime config    : milyen infrastruktura-es eroforras-hatarok kozott futhat?
```

1.7. Egy egyszerű tervezési módszer

Amikor új feature-t kezdesz, a következő öt kérdés jó kiindulópont:

1. **Mi az üzleti főnév?** – `Product`, `Article`, `Invoice`, `WikiPage`.
2. **Milyen typed adatra van ténylegesen szükség?** – ne automatikusan a teljes adatbázissort másold át.
3. **Mely műveletek olvasnak, és melyek módosítanak?**
4. **Milyen capability kell hozzájuk?** – `Db`, `Transaction`, `AppFs`, `auth` és `named` hálózati egress integration.
5. **Milyen HTTP/policy felületen tesszük elérhetővé?** – `route`, `input`, `validáció`, `auth`, `cache`, `rate limit`.

CRUD-nál ez pár perc; nagyobb rendszerben megakadályozza, hogy a `route`-ok, `SQL`-ek és jogosultsági szabályok szétcsússzanak.

1.8. Mit vigyél tovább a következő fejezetekbe?

Egy szabályt vigyél tovább: a `typed` adat, `query`, `tranzakció`, `capability` és `HTTP/security` `policy` maradjon explicit, fölösleges objektumhierarchia nélkül. A következő fejezetek telepítik a rendszert, majd ezt a modellt egy teljes `Product` requesten alkalmazzák.

1.9. Gyakorlat: rajzold fel a határtérképet

Gyakorlási mód: vezetett. Használd az előszóban meghatározott vezetett módszert.

Válassz egy egyszerű, jól ismert funkciót, például jegyzetet, feladatot vagy készletelemet. Kódírás előtt sorold fel a megbízható állapotot, a kérésből érkező adatokat, a tartós rekordokat, a módosítási pontokat és a kívülről látható válaszokat. Ezután jelöld meg, mely átmenetekhez kell validáció, autorizáció, tranzakció vagy explicit `capability`.

A cél a rejtett feltételezések láthatóvá tétele még a kód előtt.

1.10. Tipikus hiba: a route-okból indulunk ki a modell helyett

A route-központú tervezés gyakran olyan handlereket eredményez, amelyek újra és újra ugyanazokat az üzleti szabályokat fedezik fel. Ha több `route` ugyanazt az állapotot parsolja, ugyanazt a tulajdonosi ellenőrzést végzi vagy ugyanazokat az invariánsokat építi újra, vidd vissza a gondolkodást a típusos modellekhez, `query`-khez és explicit módosítási határokhoz.

1.11. Folyamatos mintaprojekt: Secure Notes

A compilerrel ellenőrzött `examples/secure-notes/checkpoints/01-baseline/main.vrn` *Secure Notes* legyen a folyamatos mintaprojekt. A baseline authenticated, principal szerint szűrt hozzáférést, bounded inputot, CSRF-védett formokat, ownership ellenőrzést és tranzakciós módosításokat kényszerít ki; a későbbi checkpointok ezeket az invariánsokat nem gyengítik.

2. fejezet

Telepítés, binárisok és futtatási topológiák

Tanulási út: Gyors út: alap. Juss el ismételtető helyi check/build/run workflow-ig; első olvasáskor hagyd későbbre a backend belső részleteit.

2.1. Build a forrásból

A kiinduló release forráscsomag Rust workspace. A két telepíthető program előállításához a szokásos release build:

```
cargo build --locked --release
```

A repository saját teljes önellenőrzése a `./verify.sh`; ezt akkor futtasd, amikor magát a Velran forrásfát validálsz vagy release-eled. Egy alkalmazás használatához nem ez az első lépés.

A `-locked` biztosítja, hogy a build a release-hez tartozó `Cargo.lock` függőséggráfot használja. Ha csak a két futtatható program kell:

```
cargo build --locked --release -p velran-server -p velran-cli
```

A build után a két fontos program:

```
target/release/velran-server  
target/release/velran-cli
```

- **velran-server**: a Velran alkalmazás fordítását és HTTP(S) kiszolgálását végző hosszú életű service process;
- **velran-cli**: operátori/adminisztratív program, például auth- és migrációs műveletekhez.

A két bináris szerepe szándékosan különbözik: a reverse proxy kizárólag a **velran-server** processhez kapcsolódik. A **velran-cli** nem webserver és nem kell publikus vagy proxy mögötti listener.

Futtatási követelmény: a rustc a Velran runtime toolchain része

Minden olyan gépen, amely Velran alkalmazást futtat, rendelkezésre kell állnia egy használható **rustc** fordítónak. A Velrannak nincs interpreter- vagy VM-fallbackje: az alkalmazás backendje az ellenőrzött Velran kódot safe Rust kóddá alakítja, majd a **rustc** állítja elő a szervertől aktivált immutable natív **cdylib**-et. Ezért önmagában a **velran-server** telepítése **rustc** nélkül nem elegendő sem alkalmazásaktiváláshoz, sem source reloadhoz.

Alapértelmezésben a szervert az aktív toolchainből ismeri fel a **rustc**-t. Production telepítésnél a **server.toml [native]** szekciójában explicit, abszolút compiler-útvonal is megadható. A Cargo magának a Velrannak a forrásból történő buildjéhez kell; az alkalmazás-végrehajtási backend közvetlenül a **rustc**-t hívja.

Amit az alkalmazás aktiválásáról kezdetben tudnod kell

Normál webfejlesztéshez először csak ezt a szabályt jegyezd meg: a Velran egy teljes candidate alkalmazást ellenőriz és fordít le, mielőtt az leválthatná az aktuálisan aktív verziót. Ha a candidate hibás, az utolsó ismert jó verzió szolgál tovább. Nincs interpreter- vagy VM-fallback, amely a nem támogatott alkalmazáskódot csendben futtatná.

Az első alkalmazások elkészítéséhez nem kell ismerned a natív compiler pipeline részleteit. Ezek szándékosan később, az „A motorháztető alatt” részben és a production fejezetben jelennek meg.

Fejlesztés közben elegendő lehet:

```
cargo check --workspace
cargo test --workspace
```

2.2. A binárisok telepítési helye

Production gépen ne a Cargo **target/** könyvtárából futtasd tartósan a szolgáltatást. Kézi/forrásos telepítésnél a könyv a **/usr/local/bin** alá teszi a binárisokat, a **/usr/local/etc/velran** alá a géphez tartozó konfigurációt; így ezek elkülönülnek a disztribúció által kezelt fájloktól.

Egy egyszerű, jól auditálható elrendezés:

```
/usr/local/bin/velran-server
/usr/local/bin/velran-cli
/usr/local/etc/velran/server.toml
/srv/velran/current/
/srv/velran/data/
/run/secrets/velran/
/var/log/velran/
```

Telepítés például:

```
sudo install -d -m 0755 /usr/local/bin
sudo install -d -m 0755 /usr/local/etc/velran
sudo install -m 0755 target/release/velran-server /usr/local/bin/velran-server
sudo install -m 0755 target/release/velran-cli /usr/local/bin/velran-cli
sudo install -m 0644 config/server.toml.sample \
  /usr/local/etc/velran/server.toml
```

A `/usr/local/bin` a szokásos rendszer-PATH része, ezért külön symlinkre nincs szükség. A könyv példáiban a valódi binárisneveket, `velran-server` és `velran-cli`, használjuk.

Debian csomag esetén szándékosan más az elrendezés

A fenti elrendezés a kézi/forrásból telepítésre vonatkozik. Ha a Velran Debian csomagból kerül a gépre, a fájlokat a `dpkg` kezeli, ezért a Debian filesystem-konvenció érvényes, nem a lokális adminisztrátori `/usr/local` prefix:

```
/usr/bin/velran-server
/usr/bin/velran-cli
/etc/velran/server.toml
/usr/lib/systemd/system/velran.service
```

A csomag Debian/Ubuntu build gépen így készíthető és telepíthető:

```
make deb
sudo dpkg -i dist/velran_1.0.0-1_amd64.deb
```

A csomag szándékosan nem indítja el automatikusan a szolgáltatást: telepítsd az alkalmazást és a secreteket, állítsd be és validáld az `/etc/velran/server.toml` fájlt, majd engedélyezd az `velran.service` unitot. Csomagolt telepítésnél `/usr` és `/etc`, kézi telepítésnél `/usr/local` a helyes. A teljes folyamat: [docs/hu/36-debian-csomag.md](https://docs.hu/36-debian-csomag.md).

2.3. Első alkalmazás

Ajánlott minimális projekt:

```
myapp/
|-- main.vrn
|-- pages.vrn
`-- public/
```

`main.vrn`:

```
mod pages;
```

`pages.vrn`:

```
#[page]
fn home(ctx: PageContext) -> Result<Html, PageError> {
    return Ok(html {
        <main><h1>Hello Velran</h1></main>
    });
}

route home GET "/" public => home;
```

Fejlesztői indítás közvetlenül a workspace-ből:

```
cargo run -p velran-server -- \\\
  --app /abs/path/myapp/main.vrn \\\
  --listen 127.0.0.1:8080 \\\
```

```
--insecure-dev-cookies
```

Ugyanez a lefordított release binárisal:

```
/usr/local/bin/velran-server \\  
--app /abs/path/myapp/main.vrn \\  
--listen 127.0.0.1:8080 \\  
--insecure-dev-cookies
```

Az `--insecure-dev-cookies` csak lokális developmentre való.

2.4. A konfigurációból elsőre csak a minimum kell

A könyv ezen pontján még nem cél a teljes `server.toml` megtanulása. Kezdőként elég három dolgot szétválasztani:

- az *alkalmazáskód* mondja meg, milyen modellek, queryk, oldalak és route-ok léteznek;
- a *szerverkonfiguráció* mondja meg, hol hallgat a process, milyen külső erőforrásokat érhet el, és milyen operátori limitek érvényesek;
- a *secret fájlok* tartalmazzák azokat az érzékeny értékeket, amelyek nem kerülhetnek sem az alkalmazáskódba, sem plaintext TOML-ba.

Az A–Z fejezet egy minimális development konfigurációt ad. A production-specifikus kulcsokat akkor vezetjük be, amikor már van mit üzemeltetni; a teljes kulcs/default/sample referencia a könyv végén található. Első olvasáskor tehát ne próbáld a teljes konfigurációs felületet memorizálni.

A motorháztető alatt: natív alkalmazásaktiválás

Első olvasáskor ezt a részt nyugodtan átugorhatod. Belsőleg az aktiválási út tudatosan lépcsőzetes, hogy az ellenőrzés azelőtt történjen meg, hogy az új kód éles generációvá válhatna:

```
.vrn forras  
-> parser + security/type verification  
-> verified executable intermediate representation (IR)  
-> general safe Rust  
-> rustc  
-> immutable cdylib generation  
-> atomic activation
```

A generált alkalmazás-Rust tiltja az unsafe kódot. A nem támogatott konstrukció fail-closed hibát ad interpreter/VM kerülőút helyett. Source reload közben a hibás candidate ezért nem váltja le az utolsó ismert jó generációt. A natív compiler cache a változatlan, ellenőrzött source/toolchain/policy identity újrafordítását kerüli el.

2.5. Production process és reverse proxy

Apache vagy Nginx mögött a tipikus adatút:

```
Internet  
|  
| HTTPS :443  
v
```

```
Apache vagy Nginx
|
| HTTP 127.0.0.1:8080
v
/usr/local/bin/velran-server
|
+-- /usr/local/etc/velran/server.toml
+-- /srv/velran/current/main.vrn
+-- DB / Redis / AppFs
```

A proxy nem indítja és nem helyettesíti a Velran processzt. A **velran-server** külön service-ként fusson, például systemd alatt, a proxy pedig a loopback listenerre továbbítja a kéréseket. A **velran-cli** csak operátori parancsok idejére indul el.

Egyszerű systemd ExecStart részlet:

```
ExecStart=/usr/local/bin/velran-server \\  
--config /usr/local/etc/velran/server.toml
```

A process felhasználója ne legyen **root**; csak a szükséges alkalmazás-, secret-, data- és log-fájlokhoz kapjon jogosultságot. Ha a TLS-t Apache vagy Nginx terminálja, a Velran listener maradjon loopbacken, például 127.0.0.1:8080.

2.6. Futtatási topológiák részletesen

A binárisok helye és a processzek szerepe ugyanaz marad, akár a Velran terminálja a TLS-t, akár Apache vagy Nginx áll előtte. A különbség a listener, a TLS-termináció és a trusted proxy határ konfigurációja.

2.7. Közvetlen, egydomaines HTTPS

Ha a Velran szerver maga terminálja a TLS-t, a publikus listener lehet például 443:

```
[server]
app = "/srv/velran/current/main.vrn"
listen = "0.0.0.0:443"
insecure_dev_cookies = false

[tls]
cert_file = "/run/secrets/velran/tls-fullchain.pem"
key_file = "/run/secrets/velran/tls-key.pem"
handshake_timeout_ms = 5000
http_redirect_listen = "0.0.0.0:80"
public_host = "www.example.com"
```

A **public_host** trusted konfigurációból jön. HTTP-ről HTTPS-re csak GET/HEAD kérés redirectelhető.

Mivel a közvetlen minta 80/443 portot használ, unprivileged systemd service esetén a processnek célzott **CAP_NET_BIND_SERVICE** capability kell. Ne futtasd emiatt rootként. Reverse proxy mögött, például 127.0.0.1:8080 listenerrel ez a capability nem szükséges.

2.8. Apache reverse proxy mögött

Ebben a felállásban az Apache kezeli a publikus TLS-t, a Velran pedig csak loopback interfészen hallgat. Példa Velran config:

```
[server]
app = "/srv/velran/current/main.vrn"
listen = "127.0.0.1:8080"
insecure_dev_cookies = false

[tls]
public_host = "www.example.com"

[web]
trusted_proxy_cidrs = ["127.0.0.1/32", "::1/128"]
allow_missing_origin = false
cors_origins = []
cors_allow_credentials = false
```

Reverse proxy módban nincs Velran oldali certificate/key, de a `tls.public_host` továbbra is szükséges: ez a böngésző által látott canonical host trusted forrása, amelyhez a Host és az Origin/Referer ellenőrzés kötődik. A production plain HTTP upstream csak loopback listeneren, explicit trusted proxy listával és ilyen public hosttal engedett. Minden normál proxied kérésnek trusted proxytól származó `X-Forwarded-Proto: https` (vagy ekvivalens szabványos `Forwarded: proto=https`) metadata-t kell hordoznia.

Az Apache VirtualHost minimális mintája:

```
<VirtualHost *:443>
    ServerName www.example.com

    SSLEngine on
    SSLCertificateFile /etc/letsencrypt/live/www.example.com/fullchain.pem
    SSLCertificateKeyFile /etc/letsencrypt/live/www.example.com/privkey.pem

    ProxyPreserveHost On
    ProxyPass / http://127.0.0.1:8080/
    ProxyPassReverse / http://127.0.0.1:8080/

    RequestHeader set X-Forwarded-Proto "https"
</VirtualHost>
```

Ehhez tipikusan a `proxy`, `proxy_http`, `headers`, `ssl` modulok szükségesek. A forwarding headereket a Velran csak a `trusted_proxy_cidrs` tartományból fogadja el; ezért ne tegyél tág, például teljes internetet lefedő trusted proxy CIDR-t a configba.

A port 80-as VirtualHost feladata lehet kizárólag az Apache-s HTTPS redirect:

```
<VirtualHost *:80>
    ServerName www.example.com
    Redirect permanent / https://www.example.com/
</VirtualHost>
```

2.9. Nginx reverse proxy mögött

Az Nginx topológia ugyanazt a trust modellt használja: a publikus TLS-t az Nginx kezeli, a Velran pedig loopbacken hallgat. A Velran oldali `server.toml` megegyezhet az Apache példában látottal.

Minimális Nginx server block:

```
server {
    listen 443 ssl;
    server_name www.example.com;

    ssl_certificate      /etc/letsencrypt/live/www.example.com/fullchain.pem;
    ssl_certificate_key  /etc/letsencrypt/live/www.example.com/privkey.pem;

    location / {
        proxy_pass http://127.0.0.1:8080;
        proxy_http_version 1.1;
        proxy_set_header Host $host;
        proxy_set_header X-Forwarded-Proto https;
        proxy_set_header X-Forwarded-For $remote_addr;
    }
}
```

A példában egyetlen közvetlen reverse proxy van, ezért az `X-Forwarded-For` értéke a proxy által látott klienscím, az `X-Forwarded-Proto` pedig `https`. Ne engeddd, hogy az internet felől érkező tetszőleges forwarding header authority legyen. A Velran csak a pontos `trusted_proxy_cidrs` tartományból érkező forwarding headert fogadja el.

A HTTP listener feladata lehet csak a HTTPS redirect:

```
server {
    listen 80;
    server_name www.example.com;
    return 301 https://$host$request_uri;
}
```

2.10. Ajánlott production könyvtárstruktúra

```
/usr/local/bin/velran-server
/usr/local/bin/velran-cli
/srv/velran/releases/2026-09-04.1/
/srv/velran/current -> /srv/velran/releases/2026-09-04.1
/srv/velran/data/
/usr/local/etc/velran/server.toml
/run/secrets/velran/
/var/log/velran/
```

A bináris, a release és a konfiguráció legyen runtime read-only. Az alkalmazás csak a deklarált data/log helyekre írjon.

2.11. Első ellenőrzés

Mielőtt forgalmat adnál a szerverre:

```
/usr/local/bin/velran-server \\  
  --config /usr/local/etc/velran/server.toml \\  
  --check-config
```

Ez listener nélkül ellenőrzi többek között a konfigurációt, a secret-file referenciákat, az alkalmazás fordíthatóságát és a TLS fájlokat.

2.12. Gyakorlat: ellenőrizd a telepítési határokat

Gyakorlási mód: vezetett. Használd az előszóban meghatározott vezetett módszert.

Egy tesztgépen azonosítsd a binárist, az immutable release-fát, a konfigurációt, a secret fájlokat, az alkalmazásadatokat és a logokat. Ellenőrizd, hogy a service account csak a szándékosan írható adat- és loghelyekre tud írni. Listener nyitása előtt futtasd a `-check-config` ellenőrzést.

2.13. Tipikus hiba: a konfigurációt alkalmazásadatként kezeljük

A konfiguráció és a secret ne sodródjon együtt a módosuló alkalmazásállapottal. A read-only release-fa és konfiguráció jóval egyszerűbbé teszi a rollbacket, a felülvizsgálatot és az incidenskezelést.

2.14. Folyamatos mintaprojekt: Secure Notes

A Secure Notes esetén dönts el, hol él az alkalmazás release artefaktuma, hol van a tartós adatbázis vagy adatkönyvtár, és mely secretek érkeznek fájlból ahelyett, hogy forráskódba kerülnének. Ezeket az útvonalakat még a feature-ök megvalósítása előtt rögzítsd.

3. fejezet

A-tól Z-ig: egy minimális webalkalmazás

Tanulási út: Gyors út: alap. Készítsd el az első teljes, végponttól végpontig tartó vertikális szeletet.

Rust → Velran

Hozd Rustból: A függvények, `Result`, `?` hibaterjesztés, lokális változók és hétköznapi vezérlési szerkezetek ugyanazt a szerepet töltik be, mint Rustban.

A Velran hozzáadja: A callable attribútumok, route-ok, typed request mezők, HTML-építés, context és capability köti ezt a számítást a HTTP-hez.

Nem kell újratanulnod: Ne találj ki külön Velran-specifikus hibaterjesztési vagy lokálisváltozó-stílust; a Rust-szerű mag körüli webes határra koncentrálj.

Biztonsági invariáns: explicit route policy

Route soha nem lesz implicit public. Minden HTTP belépési pont deklarálja a hozzáféréshez szükséges authorityt; a hiányzó policy hiba, nem alapértelmezés.

Compiler diagnostic: SEC-A01-001

Jelentés: Egy route nem kapott explicit hozzáférési policyt. A Velran nem találhatja, hogy a végpont public vagy authenticated legyen.

Javítás: Deklaráld a szándékolt policyt a route-on, például `public` vagy `auth user`; az alkalmazás contractja alapján válassz, ne csak a diagnostic elnémitására.

Mielőtt a nyelvi részleteket külön tanulnánk, építsünk egy teljes, szándékosan kicsi alkalmazást. Csak minimális development konfigurációt használ, és megmutatja az utat a forrástól az adatbázison és HTTP-n át a telepítésig; nem kész webshop.

A példa egy mini terméklista. A kezdőoldal kilistázza az adatbázisban lévő termékeket, és ugyanott egy formmal új terméket is fel lehet venni. Ezzel egyetlen példában megjelenik a model, typed SQL, GET page, POST action, form validation, CSRF, transaction, migration, konfiguráció és a reverse proxy mögötti futtatás.

A repositoryban a kész példa itt található:

```
examples/a-z-products/  
main.vrn
```

```
migrations/0001_create_products.sql
server.toml.example
README.md
```

3.1. A request életciklusa: egyetlen mentális modell a legtöbb webes feladathoz

A könyv alkalmazáskódjának nagy része ugyanazzal a hétlépcsős modellel ellenőrizhető:

Lépés	Kérdés
Request	Melyik route és HTTP metódus fogadta a kérést?
Parse	Mely deklarált query, form, JSON, upload vagy path értékből lesz típusos input?
Validate	Milyen bound, refinement vagy több mezőt érintő szabály kötelező?
Authenticate	Ki a hívó, és szükséges-e session vagy gépi identity?
Authorize	A hívó végrehajthatja-e ezt a műveletet ezen a konkrét objektumon vagy tenanton?
Query/tranzakció	Milyen olvasás vagy állapotmódosítás engedett az előző kapuk után?
Response	Milyen típusos HTML, JSON, redirect, státusz vagy hiba hagyja el az alkalmazást?

A lépések gondolkodási sorrendet jelentenek, nem azt, hogy minden request hét külön runtime függvényt futtat. Egy publikus GET-hez nem feltétlenül kell autentikáció; egy read-only oldal nem feltétlenül nyit tranzakciót. A fegyelem lényege, hogy minden kérdést explicit felteszünk, és csak akkor hagyunk ki egy lépést, ha a contract szerint valóban nincs rá szükség.

Állapotmódosító kérésnél őrizd meg a security sorrendet: protected munka előtt parse és validate, identityfüggő döntés előtt authenticate, protected effect előtt authorize, majd a módosítás a megfelelő tranzakciós vagy critical-operation contract alatt történjen. A response csak ezeken a határokon átjutott adatból épüljön fel.

Hibakereséskor ugyanezen a láncon haladj balról jobbra. Azt keresd, hol tér el először a tényleges viselkedés a deklarált contracttól, ahelyett hogy a „handler” egyetlen átláthatatlan lépésként kezelnéd.

3.2. A könyvtár létrehozása

Fejlesztéshez legyen egy külön alkalmazáskönyvtárunk:

```
mkdir -p my-products/migrations
cd my-products
```

Ebben a példában minden Velran kód egyetlen `main.vrn` fájlban marad. Nagyobb alkalmazásnál később mod fájlokra bontjuk.

3.3. Az adatbázis-séma

A `migrations/0001_create_products.sql` tartalma SQLite esetén:

```
CREATE TABLE products (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
```

```
    name TEXT NOT NULL,  
    price INTEGER NOT NULL CHECK (price >= 0)  
);
```

Ez a migration hozza létre azt a táblát, amelyet az alkalmazás typed SQL lekérdezései használnak. Productionban a migráció külön deploy lépés; a server nem módosítja automatikusan a sémát induláskor.

3.4. A model és a lekérdezések

A `main.vrn` elején a modelleközpontú fejezetben megismert módon először a typed üzleti adatokat írjuk le:

```
model Product {  
    id: i64  
    name: String  
    price: i64  
}
```

A lista lekérdezése:

```
#[query]  
fn listProducts(db: Db) -> Result<Vec<Product>, DbError> sql {  
    SELECT id, name, price  
    FROM products  
    ORDER BY id DESC  
    LIMIT 50  
}
```

A `Vec<Product>` azt mondja ki, hogy a query nulla vagy több `Product` sort adhat vissza. A `SELECT` oszlopai pontosan a model mezőinek nevét és sorrendjét követik.

Az `INSERT` transaction capabilityt kap:

```
#[query]  
fn createProduct(  
    tx: Transaction,  
    name: String,  
    price: i64  
) -> Result<(), DbError> sql {  
    INSERT INTO products(name, price)  
    VALUES (:name, :price)  
}
```

A `:name` és `:price` bind paraméter. Nem szövegkonkatenációval építünk SQL-t felhasználói inputból.

3.5. A GET oldal

A kezdőoldal lekéri a termékeket, megjelenít egy formot és kilistázza az eredményt:

```
#[page]  
fn home(ctx: PageContext, db: Db) -> Result<Html, PageError> {
```

```

let products = listProducts(db)?;

return Ok(html {
  <main>
    <h1>Mini terméklista</h1>

    <form method="post" @action(create)>
      <input type="hidden" name="_csrf" value="{{ csrfToken }}">
      <label>
        Nev
        <input name="name" maxlength="100">
      </label>
      <label>
        Ár
        <input name="price">
      </label>
      <button type="submit">Hozzaadás</button>
    </form>

    <ul>
      @for product in products {
        <li>
          #{{ product.id }} - {{ product.name }} - {{ product.price }}
        </li>
      }
    </ul>
  </main>
});
}

```

A dinamikus szövegek HTML-escape-elve kerülnek a válaszba. A form action nem kézzel írt URL, hanem az `@action(create)` helperből származik. A rejtett `_csrf` mező a POST kérés CSRF-védelméhez kell.

3.6. A POST action

A form sikeres beküldése transactionben ír az adatbázisba, majd redirectel vissza a kezdőoldalra:

```

#[action]
fn create(
  ctx: ActionContext,
  db: Db,
  name: String,
  price: i64
) -> Result<Redirect, PageError> {
  transaction db {
    createProduct(tx, name, price)?;
  }
  return Ok(redirect(home()));
}

```

A redirect célja a typed GET route helper, `home()`, nem string URL. Ez a Post/Redirect/-Get minta: ha a felhasználó frissíti a visszakapott oldalt, a böngésző nem küldi be újra automatikusan az INSERT-et.

3.7. A route-ok és a validáció

A fájl végén két route köti össze a HTTP felületet a handlerekkel:

```
route home GET "/" public => home;

route create POST "/products"
  form name<String> price<i64>
  validate name length 1 100 price range 0 100000000
  public => create;
```

A POST route már a handler előtt typed inputtá alakítja a két mezőt. Üres vagy 100 karakternél hosszabb név, negatív ár, túl nagy ár vagy nem egész szám esetén a kérés nem jut el érvényes üzleti műveletként az INSERT-ig.

A publikus írás ebben a kicsi tanpéldában szándékos egyszerűsítés. Valódi public submission endpointnál explicit dönts el, kell-e authentication, route rate limit, abuse monitoring vagy erősebb business permission. Ne másold át a `public` policyt csak azért, mert a példa ezt használja.

3.8. A teljes alkalmazás egy helyen

Az előző részek együtt alkotják a teljes alkalmazást: `model` + két `#[query] fn` + `#[page] fn` + `#[action] fn` + két route. Nem ismételjük meg még egyszer ugyanazt a hosszú listinget; a repositoryban a pontos, egyfájlos változat közvetlenül megnyitható:

```
examples/a-z-products/main.vrn
```

Ez szándékos könyvszerkesztési döntés is: a következő fejezetekben az egyes nyelvi elemeket már részletesen tárgyaljuk, itt viszont az alkalmazás teljes útvonalán akarunk végigmenni.

3.9. SQLite kapcsolat developmenthez

A database URL-t ne írjuk a forrásba. Készítsünk külön fájlt, abszolút SQLite pathszal:

```
printf '%s\n' 'sqlite:///tmp/velran-a-z-products.db' > dev-db-url
chmod 600 dev-db-url
```

A fájl itt nem valódi titkot tartalmaz, de ugyanazt a `*_file` mintát használjuk, mint productionben a jelszavas database URL esetén.

3.10. Forrásellenőrzés és migration

Ha a release binárisokat már telepítettük például `/usr/local/bin` alá:

```
/usr/local/bin/velran-cli check main.vrn

/usr/local/bin/velran-cli migrate verify \
  --dir migrations \
  --db-url-file "$PWD/dev-db-url"

/usr/local/bin/velran-cli migrate apply \
```

```
--dir migrations \  
--db-url-file "$PWD/dev-db-url"
```

A `check` a nyelvi és statikus szerződést ellenőrzi. A migration CLI külön folyamat; ettől még az alkalmazásszerver nem indult el.

3.11. Minimális development konfiguráció

A `server.toml` legyen például:

```
[server]  
app = "/ABS/PATH/my-products/main.vrn"  
listen = "127.0.0.1:8080"  
insecure_dev_cookies = true  
  
[database]  
url_file = "/ABS/PATH/my-products/dev-db-url"
```

A két `/ABS/PATH` helyére valódi abszolút útvonal kell. Az `insecure_dev_cookies` csak loop-back developmenthez elfogadható; productionben ne kapcsolod be.

Először ellenőrizzük a konfigurációt:

```
/usr/local/bin/velran-server --config "$PWD/server.toml" --check-config
```

Majd indulhat a szerver:

```
/usr/local/bin/velran-server --config "$PWD/server.toml"
```

3.12. Az első HTTP kérés

Másik terminálból:

```
curl -i http://127.0.0.1:8080/
```

Böngészőben nyisd meg ugyanazt az URL-t. A form beküldése után a folyamat:

1. a böngésző POST-ot küld a `/products` route-ra;
2. a runtime ellenőrzi a request schema-t, a typed mezőket, a validációt és a CSRF tokenet;
3. a `create` action transactiont nyit;
4. a `createProduct` bind paraméterekkel INSERT-et futtat;
5. siker esetén commit történik;
6. a handler redirectet ad a `/` URL-re;
7. az új GET már az adatbázisból visszaolvasott sort rendereli.

Ez az egyszerű lánc a későbbi wiki és CMS alkalmazásban is ugyanaz marad; csak több model, query, route, jogosultság és komponens kerül köré.

3.13. Mit történik hibás inputnál?

Próbálg negatív árat vagy nem numerikus értéket megadni. A fontos elv az, hogy a böngésző HTML attribútumai csak felhasználói kényelmet adnak; a valódi szerződést a szerveroldali route schema és validation adja. Az SQL query csak már typed értéket kap.

Érdeemes még a következőket kipróbálni:

- küldj hiányzó **name** mezőt;
- küldj ismeretlen extra form mezőt;
- módosítsd a migrationt alkalmazás után, majd futtasd újra a **migrate verify** parancsot;
- állítsd le az adatbázis elérését, és figyeld meg a readiness illetve a szerverlog viselkedését.

3.14. Developmentből production felé

A példa alkalmazáskódja ugyanaz marad release környezetben is; nem kell újraírni csak azért, mert a szervert már telepített binárisal futtatjuk. A 2. fejezetben bemutatott build és telepítés után a szervert tipikusan így indul:

```
/usr/local/bin/velran-server \\  
--config /usr/local/etc/velran/server.toml
```

Az alkalmazás entripointja lehet például `/srv/velran/current/main.vrn`. Apache vagy Nginx esetén a publikus TLS kapcsolatot a proxy terminálja, a Velran pedig loopbacken hallgat. A teljes Apache/Nginx mintát nem ismételjük meg itt: a telepítési fejezet mutatja be a trust boundaryt, a könyv végi függelék pedig copy/paste-közeli production konfigurációt ad.

3.15. Mit tanultunk ebből az egy példából?

Egy mindössze néhány tucat soros alkalmazásban már láttuk a Velran teljes alap útvonalát:

```
HTTP GET  
-> route  
-> page  
-> typed query  
-> database  
-> escaped HTML  
  
HTTP POST  
-> route schema + validation + CSRF  
-> action  
-> transaction  
-> typed SQL bind  
-> commit  
-> redirect  
-> GET
```

A következő fejezetek ezt a képet bontják szét. Amikor egy-egy nyelvi vagy runtime részlet technikainak tűnik, érdemes visszagondolni erre a két kérésre: *hol lép be az adat a rendszerbe, és milyen typed/capability határokon halad át, mielőtt tartós állapot vagy HTTP válasz lesz belőle?*

3.16. Ellenőrzött companion példa

A fejezet teljes, compilerrel ellenőrzött companion példája az `examples/a-z-products/main.vrn`. A könyv kódblokkjai vezetett részletek; teljes program másolásakor a repository fájlt tekintsd elsődlegesnek, mert a `./verify.sh` pontosan ezt a forrást ellenőrzi.

3.17. Gyakorlat: kövess végig egy kérést

Gyakorlási mód: vezetett. Használd az előszóban meghatározott vezetett módszert.

Válassz egy GET és egy POST kérést a példaalkalmazásból, és mondd végig az összes határt: route-illesztés, típusos input-dekódolás, autorizáció, query vagy tranzakció végrehajtása, válasszépítés és redirect. Ha egy határt nem tudsz megnevezni, térj vissza ahhoz a részhez, amíg az adatfolyam egyértelmű nem lesz.

3.18. Tipikus hiba: összekeverjük az olvasási és módosítási folyamatot

Egy GET page és egy POST action érintheti ugyanazt a domain objektumot, de más biztonsági és életciklus-szemantikájuk van. Az olvasás maradjon mellékhatásmentes; a módosítás menjen validáción, autorizáción, tranzakciós állapotváltozáson és ahol szükséges Post/Redirect/Get (PRG) mintával.

3.19. Folyamatos mintaprojekt: Secure Notes

Valósítsd meg a legkisebb hasznos vertikális szeletet: a jelenlegi felhasználó jegyzeteinek listázását és egy új jegyzet létrehozását. Még ne adj hozzá szerkesztést, megosztást, keresést, feltöltést vagy cache-t. A cél a teljes request–storage–response útvonal bizonyítása a lehető legkevesebb mozgó résszel.

II. rész

Alap webalkalmazás-fejlesztés

4. fejezet

Nyelvi alapok és HTTP-határok

Tanulási út: Gyors út: alap. Tanuld meg a napi munkához szükséges minimális Rust-szerű nyelvi és HTTP surface-t.

Rust → Velran

Hozd Rustból: Használd tovább a Rustból ismert lokális változókat, `Vec<T>`, `Option<T>`, `String`, skalárokat, metódusokat és `match`-et.

A Velran hozzáadja: A Velran tulajdona a verified web surface: callable attribútumok, route/input deklarációk, capabilityk, HTML/JSON határok és a biztonságosan lowerelhető részhalmaz.

Nem kell újratanulnod: A hétköznapi compute-szintaxis nem rivális DSL. Ha egy normál Rust-alakú kifejezés elfogadott, azt használd, ne keress külön Velran-specifikus írásmódot.

Az A–Z példa már megmutatta a fő építőelemeket. Első olvasáskor koncentrálj a modulokra, typed értékekre, handlerekre, route-okra, opcionális/listaértékekre és biztonságos HTML-re; a részletes aritmetikai és szövegkezelési katalógushoz szükség esetén térj vissza. Munkaszabály: *a bemenetet a határon parse-old és normalizáld, bent typed értékekkel dolgozz, explicit query-vel perzisztálj, és minél később renderelj.*

4.1. Forrásstruktúra és statement-határok

Modulok

A `main.vrn` a modulgráf belépési pontja:

```
mod models;
mod queries;
mod components;
mod pages;
mod actions;
```

A `mod application-root` relatív modult tölt be, de a deklarációit nem önti a globális névtérbe. Modulhatáron át az ordinary library-elemek kvalifikált nevet, például `models::Article`-t használnak, és alapból privátak; explicit `pub` tehet publikus API-vá structot, enumot, pure függvényt, inherent metódust vagy struct-mezőt. A webes authority továbbra is route/capability policyből származik, a `pub mod` re-export szemantika pedig külön modul/import kérdés.

Modul-névterek és modulok közötti hívások

A moduldeklaráció egy application-root relatív forrásmodult vesz fel az explicit source graphba; nem másolja annak deklarációit az aktuális scope-ba. A kanonikus leképezés közvetlen:

```
models.vrn          -> models
catalog/queries.vrn -> catalog::queries
catalog/pages.vrn   -> catalog::pages
```

Egy belépési pont például több modult tölthet be:

```
mod models;
mod catalog::queries;
mod catalog::pages;

route catalogIndex GET "/catalog"
  public => catalog::pages::index;
```

Modulhatáron át kvalifikált nevet használj. Például:

```
let recent = catalog::queries::recent(db);
// típus modulhataron át: models::Article
```

A `catalog/queries.vrn` tehát a `catalog::queries` névtérhez tartozik. A deklaráció saját modulján belül a rövid lokális név továbbra is használható. Így a helyi kód rövid marad, a forrásmodulok közötti dependency viszont látható.

A modulbetöltés továbbra is explicit és az application/package rooton belültre zárt: nincs `mod.vrn` fallback, automatikus könyvtár-discovery, `../`, `./` vagy wildcard import. A verified modulgráfon belül támogatottak a Rust-szerű `crate::`, `self::` és `super::` névtérgyökök. A szűk, explicit `use path as alias; import` támogatott. A package-root re-export szándékosan szűkebb a Rustnál: `pub use path as alias;` csak már publikus modul-névteret tehet elérhetővé; wildcard, item-, private-module és private transitive-dependency re-export fail-closed módon tiltott.

A nyelvi műveletek – például `text.trim()`, `x.sin()` és `values.len()` – Rust-szerű metódus-szintaxist használnak, amikor a művelet természetesen egy értékhez tartozik. Framework helper csak ott marad explicit függvény, ahol a Velran webes biztonsági szerződést ad hozzá, például `splitBounded(...)`. Ezek nem alkalmazásmodul-deklarációk, ezért továbbra is kvalifikáció nélkül használhatók.

Rust-szerű structok, inherent metódusok és konstruktorok

Normál pure/domain számításhoz a Rust-alakú objektummodellt használd, ne egy második objektumszintaxist találj ki. A visibility explicit, és alapból private:

```
pub struct Summary {
  pub length: i64,
  hidden: i64,
}

impl Summary {
  pub fn new(length: i64) -> Self {
    return Self { length, hidden: 0 };
  }
}
```

```

    pub fn length_value(&self) -> i64 {
        return self.length;
    }
}

let summary = Summary::new(42);
let n = summary.length_value();

```

Az associated function, például `Type::new(...)`, illetve az inherent `impl`-en belüli `Self` ugyanazon verified pure/native lowering útvonalon halad, mint a normál függvény. A publikus metódus-aláírást a compiler rekurzívan visibility-ellenőrzi, ezért privát struct paraméteren vagy visszatérési típuson keresztül sem szivároghat ki, beleértve a beágyazott `result`/`option` alakokat is.

A jelenlegi biztonságos surface tudatosan szűkebb a teljes Rustnál: az általános mutable/owned receiver (például `&mut self` vagy fogyasztó `self`), a trait `impl` és a generic `impl` továbbra is fail-closed, amíg a compiler az ownership/alias szemantikát a transaction, authorization, tenant isolation és idempotency határok gyengítése nélkül nem tudja bizonyítani.

Lokális package-ek és explicit dependencyk

A workspace lokális, `velran.toml`-lal deklarált package-eket használhat. A dependency path-pinnelt, korlátozott és ciklusellenőrzött DAG-ként oldódik fel; a nyelvi package-rétegben nincs registry, Git dependency, build script vagy ambient hálózati feloldás. A közvetlen dependency saját package-névterén keresztül látható, a tranzitív dependency pedig privát marad a deklaráló package számára, kivéve a kifejezetten engedett publikus modul-névtér re-exportot.

```

[package]
name = "demo_app"
kind = "app"
entry = "main.vrn"

[dependencies]
textkit = { path = "../libs/textkit" }

```

A library visibility és a webes authority két külön fogalom. A `pub` Rust-szerű library API-t tehet elérhetővé, de nem ad route/page/action/query/integration/filesystem/network jogosultságot. A library package-eket továbbra is a compiler-owned capability boundary védi, beleértve a fail-closed library web-authority szabályt.

Rust-szintaxis és framework-owned webes szintaxis

Hasznos szabály: egy Rust-szerű compute nyelvet tanulj meg, és ehhez csak azokat a webspecifikus kiegészítéseket add hozzá, amelyeket a Velrannak kell birtokolnia. A határ szándékosan látható:

Terület	Rust-szerű forma	Velran-owned határ
Függvény és lokális	<code>fn</code> , <code>let</code> , <code>let mut</code> , közvetlen assignment	<code>#[page] fn</code> , <code>#[action] fn</code> , <code>#[query] fn</code>
Érték és kollekció	<code>i64</code> , <code>f32</code> , <code>String</code> , <code>Vec<T></code> , <code>Option<T></code>	domain type, bounded request type és capability type
Control flow	<code>if</code> , <code>while</code> , exhaustív <code>match</code> , ?	instruction/allocation budget és fail-closed verification
Web/data boundary	normál typed kifejezés	<code>route</code> , <code>html</code> , <code>sql</code> , <code>transaction</code> , validation/auth/cache policy

Ez a felosztás nem framework-megkerülés. Rust-szerű szintaxis csak akkor elfogadott, ha a natív lowering közben ugyanaz a verified, bounded webes szemantika megőrizhető.

Statement terminátorok

A Velranban az egyszerű statementeket explicit pontosvessző zárja. A sortörés whitespace, nem statement-határ; nincs automatikus semicolon insertion. Ez determinisztikussá teszi a többsoros kifejezéseket és egyértelműbbé a generált, illetve auditált kódot.

```
let subtotal = price
    * quantity
    + shipping;
let mut retries = 0;
retries = retries + 1;
authorize article owner authorUsername or role Publisher;
return Ok(json(subtotal));
```

A blokkos statementeket és blokkos deklarációkat a kapcsos zárójel zárja, utánuk nincs pontosvessző:

```
let mut state = 0;
if ready {
    state = 1;
}
while state < 3 {
    state = state + 1;
}
```

Ugyanez érvényes tranzakción belül is: az önálló mutating query-hívás és a business-audit statement egyszerű statement, ezért `;` zárja. A top-level blokkos deklarációk – például `model`, `#[page] fn`, `#[action] fn`, `#[query] fn` – és a control-flow blokkok után nincs `;`. A blokk nélküli top-level deklarációk viszont pontosvesszősök: a `mod path;` és a `route ... => handler;` is explicit `;` terminátort kap. Többsoros route esetén is csak ez a záró terminátor fejezi be a deklarációt.

4.2. Typed számítás webalkalmazás-kódban

Kifejezések és aritmetika

A Velran explicit operátorprecedenciát használ. Szorosabbtól lazább felé: unáris `!`; szorzás, osztás és maradékképzés; összeadás és kivonás; shiftek; összehasonlítások; bitenkénti

AND/XOR/OR; logikai AND; logikai OR. Ha javítja az olvashatóságot, használj zárójelet.

```
let total = 12 + 3 * 4;
let remainder = 17 % 5;
let shifted = 3 << 2;
let masked = 12 & 10;
let allowed = ready && authorized;
let fallback = cached || fresh;
let denied = !allowed;
```

Az operátorszerződés szigorúan typed:

Operandusok	Operátorok	Eredmény
i64, azonos típus	+ - * / \%	checked i64.
f32, azonos típus	+ - * / %	f32; véges eredmény szükséges.
Decimal, azonos típus	+ - * / \%	checked Decimal.
i64, azonos típus	< > \% \^{}	shift/bitművelet, i64.
bool, azonos típus	\&\&	bool; short-circuit kiértékelés.
bool	!	logikai negáció.
String, azonos típus	+	budgetelt String-konkatenáció.
Numerikus, azonos típus	< <= > >=	bool.
Támogatott azonos skalártípus	== !=	bool.

Nincs implicit keverés i64, f32 és Decimal között. Lebegőpontos számításhoz az integert explicit value as f32 hívással alakítsd át. Overflow, nullával osztás/maradékképzés, hibás shift vagy nem véges f32 eredmény fail-closed hibát ad.

A numerikus builtinok:

Függvény	Típus	Feladat
x.sin(), x.cos()	f32 -> f32	Trigonometrikus függvények.
x.sqrt()	f32 -> f32	Négyzetgyök; hibás tartomány elutasítva.
x.abs()	i64 -> i64 vagy f32 -> f32	Abszolútérték.
x.ln(), x.log10()	f32 -> f32	Természetes/tízes alapú logaritmus.
x.log(base)	f32, f32 -> f32	Explicit alapú logaritmus.
x.exp()	f32 -> f32	Exponenciális függvény.
x.powf(y)	f32, f32 -> f32	Lebegőpontos hatványozás.
x.round()/x.floor()/x.ceil()	f32 -> f32	Kerekítési műveletek.
x as f32	i64 -> f32	Explicit integer-f32 konverzió.
monotonicNanos()	() -> i64	Process-lokális monoton nanoszekundum-számláló.

Minden f32 eredménynek végesnek kell maradnia. A (-1.0f32).sqrt() és (-1.0f32).ln() jellegű domainhibák NaN/végtelen helyett hibát adnak. A monotonicNanos() eltelt idő mérésére való, nem falióra-időbélyeg; a tőle függő public-cache kimenetet a compiler elutasítja.

Gyakori webes aritmetikai minták

Webalkalmazásban ritkán kell mindenhol haladó matematika, néhány numerikus minta viszont folyamatosan visszatér. Ezeket typed értékekkel és látható kóddal kezeld, ne string-konverziókba vagy SQL-részletekbe rejtisd.

Lapozásnál validált integerekből számolj offsetet:

```
let pageSize = 25;
let offset = (page - 1) * pageSize;
```

Integer flag/mask esetén használhatók a bitműveletek, de üzleti feltételekhez az olvasható boolean logika az alap. A `&&` és `||` short-circuit módon működik:

```
let mayPublish = authenticated
    && ownsArticle
    && !locked;
```

Pénzhez `Decimal`-t használj. `f32`-t akkor válassz, ha maga a feladat lebegőpontos: például geometria, jelfeldolgozás, statisztika vagy explicit matematikai függvény. Pénzösszeget ne alakíts `f32`-re csak azért, hogy `math` builtint lehessen rá alkalmazni.

String- és szövegfüggvények

A string műveletek typed, Unicode-tudatos builtinok a korlátozott runtime-on belül:

```
let cleaned = text.trim();
let left = text.trim_start();
let right = text.trim_end();
let normalized = cleaned.to_lowercase();
let chars = cleaned.chars().count();
let rewritten = normalized.replace(" ", "-");
let part = substring(cleaned, 1, 3);
let first = indexOf(cleaned, "vrn");
let last = lastIndexOf(cleaned, "a");
let ch = charAt("Velran", 1);
let repeated = "vrn".repeat(3);
let parts = splitBounded(rewritten, "-", 4096);
```

Függvény	Szignatúra
<code>text.chars().count()</code>	<code>String -> i64</code>
<code>text.trim()/trim_start()/trim_end()</code>	<code>String -> String</code>
<code>text.to_lowercase()/to_uppercase()</code>	<code>String -> String</code>
<code>text.contains(needle)</code>	<code>String, String -> bool</code>
<code>text.starts_with(part) /</code> <code>text.ends_with(part)</code>	<code>String, String -> bool</code>
<code>text.replace(search, replacement)</code>	<code>String, String, String -> String</code>
<code>splitBounded(text, delimiter,</code> <code>maxItems)</code>	<code>String, String, i64 -> Vec<String></code>
<code>substring(text, start[, length])</code>	<code>String, i64[, i64] -> String</code>
<code>indexOf/lastIndexOf(text, needle)</code>	<code>String, String -> i64</code>
<code>charAt(text, index)</code>	<code>String, i64 -> String</code>
<code>text.repeat(count)</code>	<code>String, i64 -> String</code>

A `text.chars().count()`, `substring`, `indexOf`, `lastIndexOf` és `charAt` Unicode skalárérték-pozíciókat használ, nem UTF-8 byte indexet. Sikertelen keresésnél `-1` az eredmény; negatív vagy érvénytelen index `fail-closed` hibát ad. A `replace` nem enged üres keresőszöveget. A `splitBounded` nem enged üres delimitert, compile-time elemszám-korlátot kér a támogatott tartományban, és túllépéskor hibázik a csendes csonkolás helyett. A `repeat` az eredmény létrehozása előtt bekerül az allocation budgetbe. A `Vec<String>` typed compute collection marad `parts.len()` és ellenőrzött `parts[index]` használattal.

Korlátozott reguláris kifejezéses feldolgozáshoz elérhető a `regexMatch`, `regexReplace` és `regexCaptures`; ezek továbbra is a validációs és resource limitek alatt futnak.

Gyakori szövegkezelési minták

A szövegmanipulációt kezeld boundary-feladatként. Keresési és filter inputot érdemes összehasonlítás előtt normalizálni, de az eredeti értéket tartsd meg, ha a domainben számít a pontos írásmód.

```
let queryText = q.trim();
let normalizedQuery = queryText.to_lowercase();
let hasVelran = normalizedQuery.contains("velran");
```

A `replace` determinisztikus helyettesítésre való, nem általános parser helyett:

```
let displayTag = tag.trim().replace("_", " ");
```

Szeletelésnél és pozíciókeresésnél a Velran Unicode scalar-value pozíciókat használ UTF-8 byte offset helyett. Így a `substring`, `charAt` és indexfüggvények alkalmazásszövegnél nem teszik ki a fejlesztőt encoding-részleteknek. Strukturált azonosítóknál – URL, slug, email, dátum, UUID – viszont a dedikált Velran típusokat és validátorokat használd, ne string függvényekből építsd újra a szabályokat.

Melyik eszközt válaszd alkalmazáskódban?

Webfejlesztési feladat	Preferált Velran megoldás
Keresési/filter szöveg normalizálása	<code>trim</code> , majd <code>lower/upper</code> csak akkor, ha az összehasonlítás case-insensitive.
Ismert token helyettesítése	<code>replace</code> ; regexet csak valódi mintakereséshez használj.
Email, URL, slug, dátum, UUID	Dedikált typed érték és validáció, ne ad-hoc string parsing.
Pénz/összegzés	<code>Decimal</code> .
Lapozás/számlálók	<code>i64 checked</code> aritmetikával.
Üzleti feltételek	<code>bool</code> short-circuit <code>&&</code> , <code> </code> , <code>!</code> operátorokkal.
Tudományos/statisztikai matematika	<code>f32</code> és explicit <code>math</code> builtin; előbb dönts el, hogy a lebegőpont valóban illik-e a domainhez.
Eltelt idő mérése	<code>monotonicNanos()</code> , soha ne falióra-időbélyegként.

4.3. Domainértékek és handlerkód

Modellek és üzleti típusok

```
model Article {
  id: i64
  slug: Slug
  title: String
  body: String
}
```

A gyakori típusok: `String`, `i64`, `bool`, továbbá `Date`, `DateTime`, `Uuid`, `Decimal` és `Slug`. Pénzhez `Decimal`-t használj, ne lebegőpontos számot.

Page és action

A page GET-szerű megjelenítési logika:

```
#[page]
fn show(ctx: PageContext, db: Db, id: i64)
  -> Result<Html, PageError> {
  let article = articleById(db, id)?;
  return Ok(html { <h1>{{ article.title }}</h1> });
}
```

Az action állapotváltoztató kérés kezelője:

```
#[action]
fn create(ctx: ActionContext, db: Db, title: String)
  -> Result<Redirect, PageError> {
  transaction db {
    insertArticle(tx, title)?;
  }
  return Ok(redirect(adminArticles()));
}
```

A példa egy `adminArticles` nevű GET route-ot feltételez; a `redirect` cél route call, nem kézzel másolt URL string.

Változók és hibaterjedés

A `let` immutable typed lokális értéket hoz létre. Ha az értéket módosítani kell, Rust-szerű `let mut` deklarációt használj, majd normál assignmentet, például `count = count + 1` vagy `samples[i] = value`. A statikus típus nem változhat, a bounds check és az erőforrás-elszámolás megmarad, a legacy `set` szintaxis pedig tiltott. A `?` a typed `Result` hibáját továbbítja.

Enum és exhaustive match

Zárt választási halmazhoz ne szabad szöveget használj, hanem enumot. Ez form-, route-, JSON- és domain-contractként is erősebb. Az enum egyben zárt sum type: ha a vezérlés varianttól függ, használj exhaustive `match`-et.

```
enum PublishState {
  Draft
  Review
  Published
}
```

```
match state {
  Draft => { return Ok(json("draft")); }
  Review => { return Ok(json("review")); }
  Published => { return Ok(json("published")); }
}
```

Ennél a security-releváns formánál nincs wildcard fallback. Ha új variant kerül a típusba, a régi match-ek addig nem fordulnak le, amíg az új állapot nincs explicit kezelve. A tranzakciós fejezet ugyanezt használja a `CommitUnknown` esetén.

Object

Összetartozó model/query/page műveletek domain object alatt csoportosíthatók. Ez különösen nagyobb alkalmazásban segít a domain API körülhatárolásában.

4.4. HTTP- és prezentációs határ

Route

```
route article GET "/cikk/:slug<Slug>" public => article;
```

A path paraméter típusa a route-ban deklarált. Hibás input nem jut el a handlerig typed értékként.

HTML escape

```
return Ok(html {
  <h1>{{ article.title }}</h1>
});
```

A `{ ... }` HTML-escaped. Adatbázisból érkező string sem kivétel. Raw HTML API nincs.

Lista és optional érték

A listát iterálni kell, az optional modelt pedig explicit jelenlétvizsgálat után lehet mezőzni. A compiler nem engedi, hogy `Vec<Model>` vagy ellenőrizetlen `Option<Model>` úgy viselkedjen, mintha egyetlen rekord lenne.

```
@for article in articles {
  <li>{{ article.title }}</li>
}

@if article {
  <h1>{{ article.title }}</h1>
}
```

A query-cardinality pontos futásidejű szerződését – például hogy a `Option<Model>` több sor esetén hibát jelent – az adatbázis-fejezet tárgyalja.

Typed URL

```
<a @href(article, article.slug)>Megnyitas</a>
<form method="post" @action(deleteArticle, article.id)>
```

Preferáld a route helpereket

Saját alkalmazás-route-ok URL-jét ne másold kézzel több helyre. HTML-ben POST formhoz az `@action(...)` és linkhez az `@href(...)` a preferált minta, mert a route neve és typed paraméterei maradnak a kapcsolat forrásai. A beépített runtime endpointok – például a `/__velran/auth/logout` – természetesen nem alkalmazás-route helperek.

Ne építs dinamikus `href="{{ value }}"` linket. A route helper ismeri a célt és a paraméter-típust.

Component és layout

```
#[layout]
fn Main(title: String) -> Html {
    html {
        <html>
            <head><title>{{ title }}</title></head>
            <body><main>@content</main></body>
        </html>
    }
}
```

Page-ben:

```
return Ok(html {
    @layout(Main, article.title) {
        <article><h1>{{ article.title }}</h1></article>
    }
});
```

A layout pontosan egy `@content` slotot tartalmazhat; component/layout csak explicit typed paramétereket lát.

4.5. Gyakorlat: dönts el, kié a szintaxis

Gyakorlási mód: vezetett. Használd az előszóban meghatározott vezetett módszert.

Nézz meg egy page-et, actiont, query-t, route-ot, komponenst és egy hétköznapi kifejezést. Mindegyiknél dönts el, hogy normál Rust-szerű szintaxisról vagy Velran framework által birtokolt webes határról van-e szó. Ez segít új szintaxis kitalálása helyett helyesen következtetni: ahol lehet, Rust-szerű alakot várj; külön forma főleg security-, HTTP-, resource- vagy framework-contractnál indokolt.

4.6. Tipikus hiba: kényelmi szintaxist találunk ki

Ne feltételezz dokumentálatlan aliasokat, legacy callable alakokat vagy egyedi collection-típusneveket. A nyelv szándékosan közelít a Rust szintaxisához, és csak azokat a framework konstrukciókat tartja meg, amelyek a webes határokat explicitté és ellenőrizhetővé teszik.

4.7. Folyamatos mintaprojekt: Secure Notes

Definiáld a jegyzetlista page-et és a létrehozó actiont az aktuális callable attribútumokkal, típusos route inputokkal és explicit access policy-val. Az üzleti adat maradjon típusos érték, ne handlererek között továbbadott strukturálatlan string.

5. fejezet

Adatbevétel: typed SQL, CRUD, formok és PRG

Tanulási út: Gyors út: alap. Adj bounded formokat és CRUD-ot az alkalmazáshoz az authorization és transaction határok gyengítése nélkül.

Rust → Velran

Hozd Rustból: A typed értékek, structok, `Option/Result`, összehasonlítások és normál vezérlés maradnak a business rule-ok nyelve.

A Velran hozzáadja: A route form deklaráció, validációs boundok, transaction capability, authorization proof és typed redirect definiálja a védett mutation-határt.

Nem kell újratanulnod: Ne írd át a business validációt framework-mágiává. A domain logika maradjon Rust-szerű, Velran-elemet ott használj, ahol adat trust- vagy effect-határon lép át.

Biztonsági invariáns: input korlátozása effect előtt

A request adatát kezeld nem megbízhatóként mindaddig, amíg a várt típusra nem parse-oltad, szükség szerint nem normalizáltad, és explicit boundokkal nem validáltad. Ez adatbázis-módosítás, fájlírás, redirect-cél vagy más protected effect előtt történjen meg.

Verifier diagnostic: SEC-A01-005

Jelentés: Egy mutation query úgy kapott objektumkulcsot, hogy nincs authorization proof az adott model/key párra.

Javítás: Töltsd be az objektumot, authorize-old, majd az authorize-olt kulcsot (vagy változatlan lokális aliasát) add át a mutationnek.

A CRUD módosítás egy ciklust követ: olvasás, typed form input, validáció, autorizáció, tranzakciós módosítás, majd redirect és újraolvasás. A kulcskülönbség: a *valid input még nem authorized input*.

5.1. A request-életciklus alkalmazása CRUD módosításra

Egy tipikus szerkesztési kéreśnél a hétlépcsős modell konkrétan így olvasható:

```
POST /notes/42
-> parse: Form<NoteEdit>
-> validate: title/body bounds
-> authenticate: user session required
-> authorize: load note 42, prove ownership/permission
-> transaction: optimistic-lock update + audit
-> response: typed redirect to GET
```

Három szabály a legfontosabb. A validáció soha nem bizonyít authorizationt. Az authentication soha nem bizonyít objektumtulajdont. A tranzakció előtt sikeres read soha nem helyettesíti a mutationben végrehajtott authoritative concurrency checket. Ha ezeket külön tartod, sok CRUD sérülékenységgel nem tud egyszerű formhibának álcázódni.

5.2. Typed SQL és CRUD

Lekérdezés

```
#[query]
fn articleBySlug(db: Db, slug: Slug)
  -> Result<Article, DbError> sql {
    SELECT id, slug, title, body
    FROM articles
    WHERE slug = :slug
  }
```

Az SQL szöveg statikus és compiler-owned. Felhasználói érték csak typed bindként, például `:slug`, kerülhet bele.

Mutáció transactionben

```
#[query]
fn articleById(db: Db, id: i64)
  -> Result<Article, DbError> sql {
    SELECT id, slug, title, body
    FROM articles
    WHERE id = :id
  }

#[query]
fn updateArticle(
  tx: Transaction,
  id: i64,
  title: String,
  body: String
) -> Result<(), DbError> mutates Article by id sql {
  UPDATE articles
  SET title = :title, body = :body
  WHERE id = :id
}
```

Az action betölti az objectet, létrehozza az authorization proofot, és csak ezután használja az autorizált object kulcsát a mutációhoz:

```
#[action]
fn update(
  ctx: ActionContext,
  db: Db,
  id: i64,
  title: String,
  body: String
) -> Result<Redirect, PageError> {
  let article = articleById(db, id)?;
  authorize article authenticated;
  transaction db {
    updateArticle(tx, article.id, title, body)?;
  }
  return Ok(redirect(adminArticles()));
}
```

Hétköznapi lokális tranzakciónál siker esetén commit, ismert adatbázishibánál rollback történik. A `Transaction` capability nem escape-elhet a blokkból. Retry-sensitive critical műveletnél viszont explicit transaction outcome-ot kell kezelni ahelyett, hogy minden hibáról rollbacket feltételeznénk; a `CommitUnknown` szemantikát a 6. fejezet részletezi.

Portable adatbázis-séma

A V1 SQLite, PostgreSQL és MariaDB irányban is hordozható mintát céloz. `Date`, `DateTime`, `Uuid` és `Decimal` portable reprezentációja canonical text lehet.

Migration

A szerver nem migrál automatikusan startupkor. Ajánlott folyamat:

```
migrate verify
migrate apply
migrate verify
```

A migration credential legyen elkülönítve a normál application runtime credentialtól.

5.3. Formok, validáció és PRG

Named form schema

```
form ArticleForm {
  title<String>
  body<String>
  published<bool>
  validate title length 2 200 body length 1 200000
}
```

Route:

```
route articleCreate POST "/admin/articles"
  form ArticleForm
  auth role Editor
  => create;
```

A handler paramétersorrendje és típusa kövesse a form mezőit.

Hibás beküldés

Named formnál a hiányzó kötelező mező, a mezőszintű typed decode hiba vagy a deklarált validation hibája 422 *Unprocessable Content*. A runtime a mezőértékeket escape-elve tölti vissza, mezőhibát ad és CSRF tokenet illeszt a formba. Ismeretlen vagy duplikált mező viszont 400, mert az request-schema megsértése.

Ez szándékosan más, mint az inline form- vagy JSON-schema: ott a typed decode/validation hibája request-contract sértésként 400. Named formot akkor használj, amikor a hibát ugyanazon felhasználói űrlapon, mezőszintű visszajelzéssel akarod kezelni.

Sikeres POST és PRG

Sikeres módosítás után redirectet adj vissza; így a böngésző GET-tel tölti be a céloldalt. Ez a Post/Redirect/Get minta megszünteti a véletlen újraküldés tipikus problémáját.

Egyszerű teljes form-action minta

```
#[action]
fn create(
  ctx: ActionContext,
  db: Db,
  title: String,
  body: String,
  published: bool
) -> Result<Redirect, PageError> {
  let articleSlug = slug(title);
  transaction db {
    insertArticle(tx, articleSlug, title, body)?;
  }
  return Ok(redirect(adminArticles()));
}
```

A checkbox-szerű hiányzó bool mező named form esetén `false`.

Gyakorlati döntési sorrend

Állapotváltoztató form tervezésénél és review-jánál ezt a sorrendet használd:

1. decode és validation a route/form boundaryn;
2. meglévő rekord módosításánál töltsd be a cél objectet;
3. hozd létre az object/permission/tenant authorization proofot;
4. a megfelelő transaction vagy critical-operation contract alatt módosíts;
5. typed GET route helperre redirectelj;
6. csak azt a cache-t invalidáld, amelyet a módosítás ténylegesen elavulttá tett.

Ez a sorrend kizár egy gyakori hibát: a jól formázott identifier vagy form önmagában nem bizonyítja, hogy a hívó módosíthatja a hivatkozott objectet.

5.4. Teljes szerkesztési ciklus: validáció, authorization, konkurencia

Létrehozó formnál gyakran elég a validáció és a tranzakció. Meglévő üzleti object szerkesztésénél azonban megjelenik egy harmadik kérdés: a felhasználónak megjelenített példány a POST beérkezésére már elavulhat. A Velran ezeket a kérdéseket külön kezeli; egy sikeres validáció nem válik automatikusan minden további lépés bizonyítékává.

Gyakorlati edit-flow:

1. a request dekódolása a deklarált form-típusokra;
2. mező- és cross-field validáció;
3. az aktuális object betöltése és authorization proof létrehozása;
4. a beküldött verzió ellenőrzése optimistic-locking mutációval;
5. a módosítás és szükség esetén a business audit commitja ugyanabban a tranzakcióban;
6. siker után redirect egy GET route-ra.

A verzióellenőrzés magába a mutációba tartozik. Egy külön „létezik-e még ez a verzió?” SELECT újra race-t hozna létre az ellenőrzés és az írás között.

```
model Article {
  id: i64
  title: String
  version: i64
}

#[query]
fn updateArticle(
  tx: Transaction,
  id: i64,
  title: String,
  version: i64
) -> Result<Changed, DbError> mutates Article by id sql {
  UPDATE articles
  SET title = :title, version = version + 1
  WHERE id = :id AND version = :version
}
```

A **Changed** azt jelenti, hogy pontosan egy sornak kell módosulnia. Nulla érintett sor stale szerkesztést jelent és conflict lesz belőle, nem pedig valaki újabb módosításának csendes felülírása. Egynél több sor a mutáció saját invariantjának megsértése, ezért adatbázishiba.

Négy boundary, amit nem szabad összemosni

Boundary	Kérdés	Tipikus eredmény
Request schema	Szerkezetileg megengedett-e a request?	ismeretlen/duplikált mező: 400
Named-form validation	A felhasználó javíthatja-e ezt az értéket?	field/type/deklarált validation hiba: 422
Authorization	Cselekedhet-e ez a principal ezen az objecten?	fail closed a mutáció előtt
Concurrency invariant	Még aktuális-e a beküldött állapot?	stale optimistic-lock verzió: 409

Review közben ez a szétválasztás különösen hasznos: a 422 nem authorization döntés, és a sikeres authorization sem bizonyítja, hogy egy régi böngészőfűl még a legfrissebb adatbázis-állapotot tartalmazza.

5.5. Adatbázis-invariantok és felhasználóbarát validáció

Az application validation javíthatja a visszajelzést, de az authoritative uniqueness helye az adatbázis. Ha slugnak, emailnek, külső azonosítónak vagy hasonló értéknek egyedinek kell lennie, ezt adatbázis UNIQUE constraint kényszerítse ki. Preflight SELECT használható barátságosabb hibaüzenethez, de nem helyettesítheti a constraintet, mert két párhuzamos request egyszerre is átmehet a preflight ellenőrzésen.

Általánosabban is ez a minta: presentation-orientált ellenőrzés a request boundaryn, domain refinement typed/domain értékekben, authorization a védett effect előtt, race-sensitive integritás pedig az adatbázisban vagy más authoritative platform-primitívben.

5.6. Mutációs review-checklist

CRUD mutáció elfogadása előtt ellenőrizd:

- a route explicit access policyval rendelkezik;
- a request mezők typed és bounded értékek;
- object-level authorization előtt betöltöd a célobjektet;
- SQL-be érték csak typed bindon keresztül kerül;
- az állapotmódosítás a megfelelő transaction/critical-operation contract alatt fut;
- ahol lost update számít, a konkurens szerkesztés optimistic lockingot használ;
- a uniqueness és más race-sensitive invariant authoritative módon van kikényszerítve;
- rekonstruálható üzleti módosításnál a business audit tranzakció;
- siker után PRG történik, nem mutable POST response renderelése;
- a cache invalidáció szűk, és csak a ténylegesen módosított adathoz kötődik.

5.7. Ellenőrzött companion példák

A fejezet copy/paste szempontból elsődleges, compilerrel ellenőrzött repository-programjai:

- `examples/crud/app.vrn` – CRUD és objektumszintű autorizáció;
- `examples/forms/app.vrn` – típusos formok és validáció;
- `examples/optimistic-locking/main.vrn` – elavult szerkesztés elleni védelem;
- `examples/prg-flash/app.vrn` – PRG és flash állapot.

5.8. Gyakorlat: tervezz szerkesztési ütközést

Gyakorlási mód: támogatott. Használd az előszóban meghatározott támogatott módszert.

Szimulálj két böngésző-sessiont, amelyek ugyanazt a rekordot nyitják meg. Az A session ment először, majd a B session elavult adatot küld vissza. Döntsd el, hol utazik a verzió, hol derül ki az eltérés, milyen HTTP eredmény születik, és mit lát ezután a felhasználó.

5.9. Tipikus hiba: csak a tranzakció előtt validálunk

Az alkalmazásszintű validáció jobb hibaüzeneteket ad, de a race-sensitive invariánsoknak továbbra is kell authoritative enforcement pont. Uniqueness, tulajdonosfüggő módosítás és optimistic concurrency akkor is maradjon helyes, ha két kérés szinte egyszerre fut.

5.10. Folyamatos mintaprojekt: Secure Notes

Használd az `examples/secure-notes/checkpoints/05-crud/main.vrn` compilerrel ellenőrzött snapshotot. Adj szerkesztést és törlést a Secure Notes-hoz. Legyen explicit a tulajdonosi ellenőrzés, szerkesztésnél használj optimistic lockingot, a módosítás maradjon a megfelelő tranzakciós határon belül, siker után pedig PRG-vel térj vissza.

6. fejezet

Adatbázis mélyebben: tranzakció, migráció és konkurencia

Tanulási út: Gyors út: alap. Tanuld meg a perzisztens alkalmazásokhoz szükséges séma- és query-invariánsokat.

Rust → Velran

Hozd Rustból: A függvényszignatúrák, `Result`, `Vec<T>`, typed paraméterek és a `?` ismerősek maradnak.

A Velran hozzáadja: A `##[query]`, compiler-owned SQL, typed bind paraméter, statikus sorbound és `Db/transaction capability` teszi felülvizsgálhatóvá az adatbázis-effectet.

Nem kell újratanulnod: Ne tanuld meg a stringből épített SQL-t Velran-technikaként. Az új fogalom a verified DB-határ, nem a queryk stringes összerakása.

Biztonsági invariáns: az adat nem SQL-szerkezet

Az SQL szerkezete compiler-owned marad. Felhasználó által befolyásolt érték csak typed bind paraméterként kerülhet a querybe, az eredmény mérete pedig explicit bounded; a validáció nem teszi megengedetté a stringből épített SQL-t.

Compiler diagnostic: SEC-A05-001

Jelentés: A forrás unsafe SQL-összeállítást próbál használni compiler-owned SQL és typed bindok helyett.

Javítás: Az SQL szerkezete maradjon statikus; a requestből érkező értékeket typed bind paraméterként add át.

Az előző fejezet bemutatta a typed SQL-t és a CRUD-ot. Itt azokra az adatbázis-szabályokra koncentrálunk, amelyek párhuzamos kérések és release-ek között védik az integritást. Első olvasáskor a query-szerződésre, typed bindokra, tranzakcióra, optimistic lockingra és migrationre figyelj; a kapacitás-, backup- és engine-választási részletek későbbi referencia.

6.1. Három külön felelősség

Velran alkalmazásnál érdemes három réteget szétválasztani:

alkalmazáskód statikus typed SQL queryket deklarál, bindokat ad át és tranzakciót kér;

runtime kapcsolatot pooloz, bindol, timeoutot és eredménykorlátot alkalmaz, majd a visszakapott sorokat a deklarált típushoz ellenőrzi;
operator DB credentialt, TLS-t, adatbázis-kapacitást, backupot és migrációs folyamatot birtokol.

Az alkalmazás tehát nem nyit tetszőleges socketet az adatbázishoz és nem épít saját connection poolt. A Db és a **Transaction** capability az a felület, amelyen keresztül adatbázis-művelet történhet.

A V1 data layer általános defense-in-depth korlátai között van a legfeljebb 32 poolkapcsolat, az 5 másodperces connection-acquire timeout, a 10 másodperces query timeout, a 10 000 soros backend eredményplafon és a 16 MiB-os eredményméret-plafon. A Velran modell-lista query ennél szigorúbb: a compiler literal LIMIT-et követel, és compiler/runtime szinten 1000 sor a felső korlát. Ezek biztonsági plafonok, nem pagination design; a helyes query ettől függetlenül legyen rövid, paginált és indexelhető.

Több szerverpéldánynál a DB connection budgetet az operatornak összesítve kell méreteznie:

```

ossz DB kapcsolat ~= Velran instance-ok szama
                    * instance-onkenti pool
                    + admin/maintenance tartalek

```

6.2. A query visszatérési típusa szerződés

Egy lekérdezés cardinalityje már a függvény típusában látszik:

```

#[query]
fn articleById(db: Db, id: i64)
  -> Result<Article, DbError> sql {
  SELECT id, slug, title, body, version
  FROM articles
  WHERE id = :id
}

#[query]
fn articleBySlug(db: Db, slug: Slug)
  -> Result<Option<Article>, DbError> sql {
  SELECT id, slug, title, body, version
  FROM articles
  WHERE slug = :slug
}

#[query]
fn recentArticles(db: Db, offset: i64)
  -> Result<Vec<Article>, DbError> sql {
  SELECT id, slug, title, body, version
  FROM articles
  ORDER BY id DESC
  LIMIT 100 OFFSET :offset
}

```

A három forma jelentése:

Típus	Elvárt eredmény	Cardinality sérülése
Article	pontosan egy sor	0 sor: 404; több sor: DB-hiba
Option<Article>	nulla vagy egy sor	több sor: DB-hiba
Vec<Article>	nulla vagy több sor	a runtime eredménylimitei érvényesek

Ez több egyszerű kényelmi jelölésnél: a lekérdezés eredményének alakja része a program futás közben is kikényszerített szerződésének. Az **Option<Article>** nem azt jelenti, hogy „vedd az első sort, ha van”: ha a query kettő vagy több sort ad, az invariáns sérült, ezért a runtime fail-closed DB-hibát ad. Ugyanezért egy **Article** query nulla sor esetén 404 **Not Found**, több sor esetén pedig nem választ önkényesen rekordot.

A **Vec<Model>** querynek az SQL-ben statikus literal sorlimitje is kell legyen. A bound a reusable query contract része, nem a hívó által adott **limit** paraméter; használj fix **LIMIT**-et, és bounded offsettel vagy más explicit hozzáférési mintával paginálj.

A row shape legyen explicit

A **SELECT** mezőinek neve és sorrendje egyezzen a modellel. Ne használj **SELECT ***-ot alkalmazásqueryben.

```
model Article {
  id: i64
  slug: Slug
  title: String
  body: String
  version: i64
}

// jo: ugyanaz az ot mezo, ugyanebben a sorrendben
SELECT id, slug, title, body, version
FROM articles
```

Az explicit projection két okból fontos. Először is a fordító ellenőrizni tudja a modell és a query kapcsolatát. Másodszor egy későbbi schema-bővítés nem változtatja meg észrevétlenül egy régi query result shape-jét.

6.3. Bindolj, ne SQL-t építs

A kliensből származó érték typed bindként kerüljön az SQL-be:

```
#[query]
fn findByEmail(db: Db, email: String)
-> Result<Option<User>, DbError> sql {
  SELECT id, email, display_name
  FROM users
  WHERE email = :email
}
```

A **:email** nem string-helyettesítés. A runtime backend-specifikus placeholderre fordítja és paraméterként bindolja.

Amit ne próbálj meg:

```
// rossz gondolkodasmod:
// "SELECT ... ORDER BY " + userOrder
// "WHERE " + userFilter
```

Oszlopnév, ORDER BY irány, SQL operator vagy teljes WHERE-fragment ne érkezzen user inputból. Ha több rendezést akarsz támogatni, az alkalmazáskódban válassz néhány statikus query közül.

6.4. Olvasás és mutáció authority-határa

Olvasó query Db capabilityt kap. Adatmódosító query Transaction-t:

```
#[query]
fn renameArticle(
  tx: Transaction,
  id: i64,
  title: String
) -> Result<(), DbError> mutates Article by id sql {
  UPDATE articles
  SET title = :title
  WHERE id = :id
}
```

A hívó action explicit tranzakciós blokkot nyit:

```
#[action]
fn rename(
  ctx: ActionContext,
  db: Db,
  id: i64,
  title: String
) -> Result<Redirect, PageError> {
  let article = articleById(db, id)?;
  authorize article authenticated;
  transaction db {
    renameArticle(tx, article.id, title)?;
  }
  return Ok(redirect(adminArticles()));
}
```

Normál best-effort alkalmazáskódban a sikeres blokk commitol, ismert DB hiba rollbackel. Retry-szenzitív vagy critical műveletnél viszont ne egyszerű siker/hiba állapotra szűkítsd az eredményt. Capture-öld explicit a transaction outcome-ot:

```
let article = articleById(db, id)?;
authorize article authenticated;
let outcome = transaction db {
  renameArticle(tx, article.id, title)?;
};

match outcome {
  Committed => { return Ok(redirect(adminArticles())); }
  RolledBack => { fail conflict; }
  CommitUnknown => { fail conflict; }
```

```
}

```

A `CommitUnknown` azt jelenti, hogy a runtime nem kapott elég bizonyítékot annak eldöntésére, megtörtént-e a commit. Ez nem válhat automatikus retry-já. Critical idempotens transactionnél a compiler kötelezővé teszi az outcome capture-t és exhaustive match-et. A `Transaction` továbbra sem escape-elhet a blokkból.

Egy üzleti művelet legyen egy tranzakció

Több, összetartozó módosítást ne külön actionök vagy külön tranzakciók végezzenek:

```
transaction db {
  Article.publishChanged(tx, article.id, version)?;
  audit Article article.id action publish;
    from article.status to ArticleStatus.Published
}
```

Ebben a mintában a cikk publikálása és a business audit trail együtt commitol. Ha az audit nem írható, a publikálás is rollbackel. Így nem jön létre olyan állapot, amelyben az üzleti módosítás megtörtént, de a hozzá kötelező audit rekord hiányzik.

6.5. Optimistic locking: ne írd felül más munkáját

CMS-ben és wikiben gyakori helyzet, hogy két szerkesztő ugyanazt a rekordot nyitja meg. A klasszikus

```
UPDATE articles SET title = :title WHERE id = :id
```

az utoljára mentő felhasználót teszi nyertessé, és csendben elveszhet az első szerkesztő módosítása.

Ehhez használható verziómező és `Changed`:

```
model Article {
  id: i64
  title: String
  version: i64
}

#[query]
fn updateArticle(
  tx: Transaction,
  id: i64,
  title: String,
  version: i64
) -> Result<Changed, DbError> mutates Article by id sql {
  UPDATE articles
  SET title = :title,
      version = version + 1
  WHERE id = :id AND version = :version
}
```

A call site előbb töltsse be és autorizálja a cikket, és a request ID helyett az `article.id` értéket

adja át ennek a mutációnak. Az edit form elküldi azt a `version` értéket, amelyet a felhasználó az oldal megnyitásakor látott. A `Changed` szerződése szigorú: a mutációnak **pontosan egy sort** kell módosítania. Egy módosított sor siker; nulla módosított sor `409 Conflict`; kettő vagy több módosított sor DB-hiba, mert a query a saját invariánsát sértette meg. `Changed` csak mutáló query visszatérési típusa lehet, és V1-ben nem használható `RETURNING` klauzussal.

Optimistic lockingnál, ha időközben más már mentett, a `WHERE id = :id AND version = :version` nulla sort talál, ezért a kérés konfliktusra fut. A helyes UX ilyenkor nem az automatikus újrapróbálás. Mutasd meg, hogy az adat közben változott, töltsd be az új verziót, és a felhasználó döntsön az összeolvasztásról.

Optimistic locking tipikus helyei:

- wiki oldalak és CMS cikkek;
- ticket vagy ügyfélrekord;
- workflow-állapotot tartalmazó üzleti objektum;
- ritkán módosított admin konfiguráció.

Append-only eseményekhez vagy adatbázis-oldali atomi számlálókhoz általában nincs rá szükség.

6.6. Indexet a hozzáférési mintához tervezz

A jó index nem attól jó, hogy sok van belőle, hanem attól, hogy a tényleges queryt szolgálja ki.

Egy CMS-ben például természetes hozzáférési minták lehetnek:

```
CREATE UNIQUE INDEX articles_slug_uq
  ON articles(slug);

CREATE INDEX articles_status_id_idx
  ON articles(status, id);
```

Ha a publikus oldal slug alapján kér le egy cikket, a slug legyen unique/indexelt. Ha az admin lista státusz szerint szűr és azon belül azonosító szerint lapoz, ehhez illeszkedő összetett index kellhet.

Néhány szabály:

- minden nagy listát paginálj;
- a rendszeresen használt lookup mező legyen indexelhető;
- az indexek sorrendje kövesse a `WHERE/ORDER BY` mintát;
- ne indexelj automatikusan minden oszlopot: az index írási és storage költség;
- teljesítményproblémát valós backend `EXPLAIN` tervével vizsgálj.

A Velran typed SQL-ja a query alakját védi; az adatbázis optimalizálóját nem helyettesíti.

6.7. Migration: a séma a release része

Productionban a server nem futtat automatikus migrációt startupkor. Ez szándékos: egy schema-változás külön operátori művelet, külön credentialdel és külön hibahatárral.

Tipikus könyvtár:

```
migrations/
```

```
0001_create_articles.sql
0002_articles_slug_index.sql
0003_articles_add_version.sql
```

A fájlnev alakja:

```
NNNN_name.sql
```

Már alkalmazott migration fájlt ne módosíts. A CLI SHA-256 checksumot tárol, és eltérésnél megáll.

A production parancsok a telepített CLI-val:

```
/usr/local/bin/velran-cli migrate status \
--dir /srv/myapp/current/migrations \
--db-url-file /run/secrets/migration-db-url

/usr/local/bin/velran-cli migrate verify \
--dir /srv/myapp/current/migrations \
--db-url-file /run/secrets/migration-db-url

/usr/local/bin/velran-cli migrate apply \
--dir /srv/myapp/current/migrations \
--db-url-file /run/secrets/migration-db-url \
--lock-timeout-secs 30
```

A migration credential magasabb jogosultságú lehet, mint az alkalmazás DML credentialje. Emiatt a server process ne kapja meg.

Mit garantál a migration CLI?

- ellenőrzi az alkalmazott fájl nevét és checksumát;
- nem enged észrevétlenül régi verziószám alá új migrationt beszúrni;
- adatbázis-szintű migration lockot kér;
- a state csak sikeres alkalmazás után halad tovább.

Backend-specifikus különbség viszont marad. PostgreSQL migrationonként tranzakciót használ; SQLite **BEGIN IMMEDIATE** writer lockkal a batch-et tranzakcióban tartja; MariaDB DDL nem minden esetben tranzakcionális, ezért részleges DDL-hibánál operator repair is szükséges lehet.

6.8. Expand/contract: tartsd meg a rollback lehetőségét

Egy gyors application rollback csak akkor egyszerű, ha az új schema a régi programmal is kompatibilis.

Ezért nagyobb változtatásnál használj expand/contract mintát:

1. **expand**: adj hozzá új nullable/defaultolt mezőt vagy új táblát úgy, hogy a régi app még működjön;
2. deployold az új alkalmazást;
3. szükség esetén végezz kontrollált backfillt;
4. csak későbbi release-ben **contract**: töröld a régi mezőt vagy constraintet, amikor a rollback window lezárult.

Az azonnali rename/drop, új kötelező mező vagy inkompatibilis adattípus-váltás könnyen forward-only release-t eredményez. Ezt release review során explicit döntésként kezeld.

6.9. Adatbázis-hiba kezelése

A `DbError` technikai hiba. A végfelhasználónak ne jeleníts meg driver message-et, SQL szöveget, hostnevet vagy connection stringet.

Alkalmazási szinten különítsd el legalább ezeket a helyzeteket:

nincs ilyen rekord normál domain-állapot, opcionális queryvel kezelhető;

409 konkurens módosítás optimistic-locking konfliktus, a felhasználó újratöltéssel/összevetéssel oldja fel;

adatbázis nem elérhető infrastruktúrahiba; readiness is jelezheti, a logban request ID-val kereshető;

constraint/domain hiba inkább validációval előzd meg, de az adatbázis constraint maradjon utolsó integritási védővonal.

Ne építs automatikus, korlátlan retry loopot egy teljes mutáló action köré. Egy megismételt kérésnek mellékhatása lehet, és a tranzakciós konfliktus sem feltétlen átmeneti hiba.

6.10. SQLite, PostgreSQL vagy MariaDB?

Velran V1 mindhárom backendet támogatja, de ez nem jelenti azt, hogy minden SQL dialektusfeature hordozható.

SQLite kiváló egyszerű vagy egygépes telepítéshez; az application DB fájl backupját SQLite-aware módszerrel készítsd, ne élő fájl egyszerű másolásával.

PostgreSQL erős alap többpéldányos és nagyobb production rendszerekhez; TLS-t használj remote kapcsolathoz.

MariaDB szintén támogatott, de különösen migrációknál számolj a DDL tranzakciós különbségeivel.

Portable alkalmazásnál a queryk közös SQL-részhalmozra épüljenek. Backend-specifikus funkciót csak tudatosan használj, és akkor a projekt dokumentációjában rögzítsd, hogy az alkalmazás mely backendhez kötött.

6.11. Backup szempont adatbázis-fejlesztőként

Adatbázis-fejlesztőként itt egy dolgot kell rögzíteni: a migráció és a backup két külön életciklus. A migration leírja, hogyan változik a séma; a backup/restore azt bizonyítja, hogy a tartós állapot visszaállítható. PostgreSQLnél, MariaDB-nél és SQLite-nál a backend saját konzisztens mentési eszközét használd, és ne tekints egy egyszerű fájlmásolást automatikusan érvényes backupnak.

A teljes alkalmazásállapot – adatbázis, AppFs, local-auth store, recovery manifest, recovery point objective (RPO), recovery time objective (RTO) és restore drill – a 19. fejezet témája. Itt azért utalunk rá, mert egy schema-változtatás tervezésekor már tudni kell, hogy rollbackhez vagy restore-hoz milyen állapot áll rendelkezésre.

6.12. Egy CMS adatbázis-rétegének mintája

Egy kisebb CMS-nél jó kiindulás:

```
Article
  id
  slug          UNIQUE
  title
  body
  status
  hero_image
  version
  created_at
  updated_at

migrations/
  0001_create_articles.sql
  0002_article_indexes.sql
  0003_add_version.sql
```

Az alkalmazási minták:

- publikus olvasás: `Option<Article>` slug alapján;
- admin lista: paginált `Vec<Article>`;
- szerkesztés: `Changed + version`;
- publish/archive: egy transactionben üzleti módosítás + audit;
- schema-változás: külön CLI migration;
- deploy előtt: migration verify + backup/restore readiness;
- deploy után: readiness, log, metrics és smoke test.

Ezzel a modellel a database layer nem különálló SQL-gyűjtemény lesz, hanem a webalkalmazás típus-, konkurencia-, release- és recovery-szerződésének része.

6.13. Fejezet végi ellenőrzőlista

Mielőtt egy adatbázisos funkciót késznek tekintesz, ellenőrizd:

- minden user érték typed bindként kerül SQL-be;
- nincs `SELECT *`; a projection egyezik a modellel;
- nagy lista paginált;
- a fontos lookup és rendezési minta mögött van megfelelő index;
- minden mutáció `Transaction` capabilityvel fut;
- több összetartozó módosítás egy tranzakcióban van;
- ember által párhuzamosan szerkeszthető rekordnál van optimistic locking;
- már alkalmazott migration immutable;
- a migration credential nincs az application servernél;
- destructive schema-változásnál ismert a rollback következménye;
- a DB hiba nem szivárogtat SQL-t vagy credentialt a kliensnek;
- a backuphoz tartozik rendszeresen bizonyított restore.

6.14. Gyakorlat: kezeld a query-t interfészként

Gyakorlási mód: támogatott. Használd az előszóban meghatározott támogatott módszert.

Minden feature query-jénél írd le az inputtípusokat, a projection oszlopait, a várt kardinalitást, a rendezést és az index-feltételezést. Ezt interfész-contractként kezeld, ne SQL-be rejtett implementációs részletként.

6.15. Tipikus hiba: az adatbázist csak tárolónak tekintjük

A production adatbázis invariánsokat és concurrency-szemantikát is kikényszerít. Ha a helyesség attól függ, hogy egy feltétel párhuzamos kérések mellett is igaz maradjon, az authoritative részt az adatbázisban vagy a tranzakciós tervben rögzítsd, ne csak egy korábbi alkalmazásszintű ellenőrzésben.

6.16. Folyamatos mintaprojekt: Secure Notes

Indulj az `examples/secure-notes/migrations/0001_init.sql` sémából. Vizsgáld felül a cím/törzs korlátokat, a publikációs állapot constraintjét, az időbélyegeket és az owner/id indexet mint adatbázisszintű invariantokat. Ezután adj lapozást úgy, hogy a determinisztikus rendezés és a principal szerinti szűrés megmaradjon; dokumentáld, mely garancia tartozik az SQL/séma réteghez, és melyik marad handler-level policy.

7. fejezet

Autentikáció, session és jogosultságkezelés

Tanulási út: Gyors út: alap. Válaszd külön az autentikációt, route policyt és objektumszintű authorizationt.

Rust → Velran

Hozd Rustból: Az enumok, typed azonosítók, **Result**, elágazások és normál domain predikátumok továbbra is alkalmasak policylogikára.

A Velran hozzáadja: A framework trusted principalt, route-szintű auth policyt és explicit authorization határt ad a védett objektumokhoz/effectekhez.

Nem kell újratanulnod: Ne cseréld le a domain rule-okat framework-szókincsre. Az authentication identitást ad; a typed business rule továbbra is authorityt dönt, a Velran pedig azt kényszeríti ki, hol kell ennek bizonyítva lennie.

Biztonsági invariáns: az identitás nem jogosultság

Az authentication azt állapítja meg, ki a hívó. Az authorization azt, hogy ez a hívó elvégezheti-e ezt a műveletet ezen az objektumon. Az authorization a protected read vagy mutation előtt fejeződjön be.

Verifier diagnostic: SEC-A01-022

Jelentés: A route identitása authenticated, de a role/permission proof nem elégíti ki a handler követelményét.

Javítás: Erősítsd a route policyt, vagy csak akkor módosítsd a handler követelményét, ha azt az üzleti authorization contract valóban megengedi.

Az authentication identitást bizonyít; az authorization azt dönti el, mit tehet a hívó. A session a trusted identityt viszi tovább a kérések között. A Velranban az alkalmazás route-policyja `.vrn`-ban, az identity- és session-policy pedig szerverkonfigurációban marad.

7.1. Hol helyezkedik el az authentication és authorization az életciklusban?

A request-életciklus segít különválasztani az identity és az objektum-authority kérdéseket:

```
request
-> parse / validate
-> authenticate: who is calling?
-> authorize: may caller act on this object?
-> query / transaction
-> response
```

A route policy a széles belépési kérdést válaszolja meg. Object-, tenant-, permission-, MFA- vagy critical-operation ellenőrzés ezt tovább szűkítheti: authenticated azt jelenti, hogy van trusted principal, nem azt, hogy a művelet engedélyezett.

A login authenticationt hoz létre: credential form és CSRF ellenőrzés, identity backend, session rotation, audit, redirect. A credential-policy az identity backend felelőssége, nem az objektumszintű authorizationé.

7.2. A teljes login útvonal

A beépített HTML login endpoint:

```
GET  /__velran/auth/login
POST /__velran/auth/login
POST /__velran/auth/logout
```

Ezek szerver-endpointok, nem az alkalmazás által definiált route-ok. A GET login oldal egy sessionhöz kötött CSRF tokenet ad. A POST login form pontosan a `_csrf`, `username`, `password` és `totp` mezőket várja.

A sikeres login magas szinten:

1. ellenőrzi a form szerkezetét és a CSRF tokenet;
2. kanonizálja a felhasználónevet;
3. alkalmazza a login rate limitet;
4. ellenőrzi a local-auth vagy LDAP credentialt;
5. ha szükséges, ellenőrzi a TOTP-t vagy local auth esetén a recovery kódot;
6. új, autentikált sessionre rotál;
7. audit eseményt ír;
8. 303 See Other válasszal a gyökérre irányít.

A session rotation fontos: sikeres autentikáció után nem marad használatban az anonim session azonosítója. Ez session-fixation elleni alapvédelmi elem.

7.3. Local auth: külön identity adatbázis

Single-node alkalmazásnál használható a local auth SQLite store. Ez szándékosan nem az application DB. A Velran program typed SQL queryjei így nem férnek hozzá a credential táblákhoz.

Secret file például:

```
/run/secrets/velran/local-auth-db-url
```

Tartalma:

```
sqlite:///srv/velran/auth/users.db?mode=rwc
```

Inicializálás:

```
/usr/local/bin/velran-cli auth init \  
--db-url-file /run/secrets/velran/local-auth-db-url
```

User létrehozásnál a jelszó nem CLI argumentumként szerepel, hanem fájlból olvasódik:

```
printf '%s\n' 'a-very-long-random-password' \  
> /run/secrets/velran/new-user-password  
chmod 600 /run/secrets/velran/new-user-password  
  
/usr/local/bin/velran-cli auth user-add \  
--db-url-file /run/secrets/velran/local-auth-db-url \  
--username alice \  
--password-file /run/secrets/velran/new-user-password \  
--role User \  
--role Editor
```

A local store Argon2id jelszóhash-t használ, egyedi random salttal. A jelenlegi implementáció legalább 12 és legfeljebb 1024 karakteres jelszót fogad el. Ezt ne keverd össze alkalmazásoldali form-validációval: a credential-policy az identity backend felelőssége.

7.4. Account lifecycle operátori CLI-val

A felhasználó állapotát nem kell közvetlen SQL-lel módosítani. A CLI erre explicit műveleteket ad:

```
velran-cli auth password-set --db-url-file ... \  
--username alice --password-file ...  
  
velran-cli auth disable --db-url-file ... --username alice  
velran-cli auth enable --db-url-file ... --username alice  
velran-cli auth roles-set --db-url-file ... --username alice \  
--role User --role Publisher
```

A disable nem törli a usert. Kívülről a sikertelen belépés nem árulja el, hogy rossz jelszó, nem létező user vagy tiltott account áll-e a háttérben: az ilyen különbségek account-enumeration csatornát nyithatnának.

7.5. Session invalidation: auth_generation

Local auth esetén credential- vagy authorization-változás növeli az `auth_generation` értékét. A session a létrehozáskori generációt tárolja; eltérésnél a régi session érvénytelen és új anonim session készül.

Ezért például az alábbi operátori műveletek nem csak a következő loginra hatnak:

- password change;
- role change;
- account disable/enable;

- TOTP enrollment vagy disable.

Ez különösen fontos role-visszavonásnál: nem kell megvárni a korábbi session természetes lejáratát.

7.6. Session store: memory vagy Redis

Development/test környezetben használható in-memory session store. Productionban, különösen több server instance esetén, Redis legyen a shared session backend.

```
[redis]
url_file = "/run/secrets/velran/redis-url"
allow_insecure = false
```

Redis a Velranban több rövid életű infrastruktúra-állapotot is hordozhat:

- shared session;
- TOTP replay védelem;
- login rate limiting;
- public cache.

Redis nem üzleti source of truth. Disaster recovery után elfogadható az üres session/cache state: a felhasználók újra belépnek, a cache újraépül.

7.7. Cookie policy

Normál production módban a session cookie neve:

```
__Host-velran_session
```

A szerver Path=/, HttpOnly, Secure attribútumokat használ. Alapesetben SameSite=Lax; credentialed cross-origin konfiguráció esetén SameSite=None; Secure.

Developmentben az explicit -insecure-dev-cookies escape hatch miatt a cookie neve velran_session, és nincs Secure. Ezt internetre kitett production deploymentben ne használj.

7.8. Logout

A logout szándékosan POST művelet:

```
<form method="post" action="/_velran/auth/logout">
  <input type="hidden" name="_csrf" value="{{ csrfToken }}">
  <button type="submit">Logout</button>
</form>
```

A szerver ellenőrzi a CSRF token, invalidálja a régi sessiont, létrehoz egy friss anonim sessiont, új cookie-t küld, majd auditálja a logout eseményt. Így a kijelentkezés nem egyszerű kliensoldali cookie-törlés.

7.9. LDAP / LDAPS vállalati identityhez

Shared vagy vállalati identity esetén használható LDAP backend. Productionban LDAPS URL-t használj, a service bind credential pedig secret file-ből jöjjön.

Példa konfiguráció:

```
[auth]
ldap_url = "ldaps://ldap.example.com:636"
ldap_search_base = "ou=people,dc=example,dc=com"
ldap_username_attribute = "uid"
ldap_service_bind_dn_file = "/run/secrets/velran/ldap-bind-dn"
ldap_service_bind_password_file = "/run/secrets/velran/ldap-bind-password"
require_totp = true
```

Local auth és LDAP ne legyen egyszerre elsődleges identity backend ugyanabban a processben. Ezzel nincs bizonytalan backend-prioritás vagy azonos username-ek közötti névütközés.

LDAP használatakor a role mapping trusted szerveroldali konfigurációból jöjjön. A kliens által küldött role, username vagy hasonló mező soha nem authorization authority.

7.10. TOTP és recovery code

Local auth userhez TOTP enrollment:

```
/usr/local/bin/velran-cli auth totp-enroll \
  --db-url-file /run/secrets/velran/local-auth-db-url \
  --username alice
```

A CLI megadja az enrollmenthez szükséges secret/URI adatot és recovery kódokat. A recovery kódok a local auth DB-ben csak hashként maradnak meg, és egy sikeresen felhasznált recovery kód elfogy.

TOTP kikapcsolása:

```
/usr/local/bin/velran-cli auth totp-disable \
  --db-url-file /run/secrets/velran/local-auth-db-url \
  --username alice
```

Ha `require_totp = true`, az a user sem léphet be, akinek egyébként helyes a jelszava, de nincs használható második faktora.

A login form `totp` mezőjébe hatjegyű TOTP vagy local auth esetén recovery code kerülhet. Redis backend mellett a TOTP replay állapot shared, ezért több server instance is ugyanazt a replay-védelmet látja.

7.11. Route authorization

Az alkalmazás route szinten három gyakori policyt deklarálhat:

```
route account GET "/account" auth user => account;
route admin GET "/admin" auth role Admin => admin;
route security GET "/security" auth mfa => security;
```

Jelentésük:

- **auth user:** bármely autentikált principal;
- **auth role Admin:** autentikált principal a trusted sessionben szereplő Admin role-lal;
- **auth mfa:** autentikált principal sikeresen igazolt második faktorrall.

HTML route-nál a nem autentikált kérés a login oldalra irányítható. JSON API-nál ugyanez nem HTML redirectként jelenik meg, hanem 401 **Unauthorized** válaszként; ez API kliensnél kiszámíthatóbb viselkedés.

7.12. A principal használata az oldalon

Az autholt oldal trusted sessionadatot kap:

```
#[page]
fn account(ctx: PageContext) -> Result<Html, PageError> {
  return Ok(html {
    <main>
      <h1>Account</h1>
      <p>Signed in as {{ authPrincipal }}</p>
      <form method="post" action="/__velran/auth/logout">
        <input type="hidden" name="_csrf" value="{{ csrfToken }}">
        <button type="submit">Logout</button>
      </form>
    </main>
  });
}

route account GET "/account" auth user => account;
```

A **authPrincipal**, a session role-jai és az MFA állapot szerveroldali trusted state. Ne próbáld ezeket hidden form mezőből vagy query parameterből visszaépíteni.

7.13. Route role nem mindig elég

A route-level role azt mondja meg, hogy a felhasználó beléphet-e egy műveletbe. Objektum-szintű authorizationnál ennél több kellhet.

Példa: egy szerkesztő csak a saját draftját módosíthatja. Ilyenkor nem elég az, hogy **auth role Editor**: az actionnek a DB-ből olvasott objektum tulajdonosát is össze kell vetnie a trusted session principaljával.

Az alapelv:

identity a sessionből, objektumállapot a DB-ből, döntés szerveroldalon

A kliens által küldött **owner**, **actor** vagy **role** mező legfeljebb adat, nem authority.

Az object authorization typed guard

Az **authorize guard** csak már betöltött, **nem optional** model recordon használható. A compiler azt is ellenőrzi, hogy az owner mező létezik és **String** típusú. Ezért az **Option<Article>** értéket visszaadó query eredményét előbb explicit módon kezelni kell; authorization nem értelmez „nincs rekord” állapotot.

```
let article = loadArticle(db, id)?;  
authorize article owner authorUsername or role Publisher;
```

Az ilyen guardot tartalmazó handler route-ja nem lehet public, és public cache sem kerülhet object authorization elé. A tulajdonos vagy a felsorolt trusted role engedélyez; minden más autentikált actor 403 Forbidden választ kap.

7.14. Login rate limiting

A login külön rate-limit policyt kap. Például:

```
[auth]  
login_max_attempts = 8  
login_principal_max_attempts = 16  
login_source_max_attempts = 64  
mfa_max_attempts = 5  
login_window_secs = 300
```

A szerver a trusted proxy szabályok után meghatározott effective client IP-t és a kanonikus usernevet használja a limiter kulcsához. Ezért reverse proxy mögött a **trusted_proxy_cidrs** auth security szempontból is fontos.

Sikertelen credential vagy második faktor nem ad részletes hibát a kliensnek. Rate-limit esetén 429 Too Many Requests, auth backend elérhetetlenségénél 503 Service Unavailable a megfelelő kategória.

A limiter source+principal, principal több source-on és source több principalon dimenziót is követ; MFA/recovery esetén szigorúbb stage érvényes. Ez lefedti a distributed guessinget és credential stuffingot. Shared limiter-store hiba fail closed.

7.15. CSRF, Origin és Fetch Metadata

Session-cookie alapú autentikációnál a böngésző automatikusan elküldi a cookie-t, ezért state-changing kérésnél CSRF-védelem kell. A Velran defense-in-depth modellje több réteget használ:

- session-bound CSRF token;
- same-origin Origin/Referer policy;
- Fetch Metadata kontroll;
- szigorú request parsing;
- trusted proxy policy.

A CORS nem helyettesíti a CSRF-et. Ha külön frontend originhez credentialed CORS-t engedélyezel, az külön security policy és HTTPS-et igényel.

7.16. Auth audit

A login/logout események audit logba kerülnek. Hasznos mezők például a request ID, actor, outcome és effective client IP. Az audit célja eseményrekonstrukció, nem credential-debugging.

Soha ne logold:

- a jelszót;

- TOTP secretet vagy recovery code-ot;
- session ID-t vagy teljes Cookie headert;
- CSRF tokenet;
- LDAP bind vagy DB/Redis credentialt.

A login eredménye lehet például siker, invalid credential, invalid second factor vagy rate limited; a kliensnek mutatott üzenet ettől szándékosan lehet kevésbé részletes.

7.17. Local auth vagy LDAP?

	Local auth	LDAP/LDAPS
Identity store	külön SQLite DB	vállalati directory
Tipikus deployment	single-node	shared/vállalati identity
User admin	Velran CLI	directory admin workflow
Role forrás	local auth store	trusted mapping
TOTP	local enrollment + recovery	server-side TOTP policy/secrets
Multi-instance identity	nem ez a cél	igen

Ha az alkalmazásnak saját publikus regisztráció, email-es password reset vagy önkiszolgáló MFA enrollment kell, azt ne próbáld a meglévő operátori CLI köré vékony route-tal odabarkácsolni. Ezek token-életciklust, email securityt, rate limitet és külön recovery threat modelt igényelnek.

7.18. Rich text és autentikált CMS

Az autholt CMS-szerkesztői felület sem indokolja raw HTML tárolását. A szerkesztői jogsultság és a content sanitization két külön védelmi réteg. Tartalomként Markdownot tárolj, megjelenítéskor pedig a biztonságos renderert használd:

```
<article class="article-body">
  @markdown(article.body)
</article>
```

Az @markdown nem enged raw HTML-t, és a link targeteket allowlist szerint kezeli.

7.19. Production ellenőrzőlista

Mielőtt autentikációt engedsz productionben, ellenőrizd:

- pontosan egy elsődleges identity backend aktív;
- local auth DB elkülönül az application DB-től;
- jelszó és service credential csak secret file-ból jön;
- production sessionhöz Redis van, ha több server instance fut;
- a cookie Secure/HttpOnly és nincs `-insecure-dev-cookies`;
- a `trusted_proxy_cidrs` csak a valódi Apache/Nginx proxy címét fedi;
- a login limiter be van állítva;
- a state-changing formok CSRF-et használnak;
- role helyett objektumszintű authorization is van, ahol szükséges;
- credential vagy session secret nem kerül logba;
- account disable/role/TOTP változás session-visszavonását tesztelted.

7.20. Ellenőrzött companion példák

A canonical futtatható companion példák:

- `examples/auth/app.vrn` – route-szintű autentikáció és role policy;
- `examples/object-authorization/app.vrn` – objektumszintű authority;
- `examples/mfa-elevation/main.vrn` – MFA elevation;
- `examples/critical-operations/main.vrn` – critical-operation contractok.

Így ezek az authority-mechanizmusok külön maradnak, nem mosódnak egyetlen általános „auth” fogalomká.

7.21. Gyakorlat: készíts autorizációs mátrixot

Gyakorlási mód: támogatott. Használd az előszóban meghatározott támogatott módszert.

Készíts kis táblázatot anonymous felhasználóval, hitelesített tulajdonossal, hitelesített nem tulajdonossal, privilegizált role-lal és letiltott accounttal. Sorold fel, melyik actor mit tehet egy jegyzettel. A táblázattal különítsd el a route-szintű access policy-t az objektumszintű autorizációtól.

7.22. Tipikus hiba: az authenticationt authorizationnek tekintjük

Az, hogy tudjuk, ki a hívó, nem bizonyítja, hogy hozzáférhet egy konkrét rekordhoz. A route policy adja a széles belépési szabályt; ownership-, tenant- vagy domain-specifikus ellenőrzést továbbra is a betöltött objektumon kell elvégezni.

7.23. Folyamatos mintaprojekt: Secure Notes

Használd az `examples/secure-notes/checkpoints/07-auth/main.vrn` snapshotot. Minden jegyzetművelethez követelj authenticationt. View, edit és delete útvonalon kényszerítsd ki az ownership szabályt. Ezután szándékosan teszteld a negatív esetet: egy érvényesen belépett felhasználó se férjen hozzá más jegyzetéhez pusztán egy azonosító kitalálásával.

8. fejezet

Fájlok, feltöltés és médiatár

Tanulási út: Gyors út: opcionális ág. Most olvasd, ha az alkalmazás uploadot fogad vagy alkalmazástulajdonú médiát szolgál ki.

Biztonsági invariáns: a kliens fájlmetaadata nem megbízható

A beküldött fájlnev és Content-Type leíró metaadat, nem bizonyíték biztonságos tárolásra vagy tartalomra. A storage maradjon AppFs/capability határ mögött, a kiszolgálási döntés pedig trusted detektálásból és policyből származzon.

Compiler diagnostic: SEC-FILE-001

Jelentés: Egy Image uploadból hiányzik a media-kiszolgálás előtti explicit `publish` transition.

Javítás: Használd a typed image upload formát `publish` jelöléssel; a raw/nem ellenőrzött upload maradjon `private`.

A feltöltés request-, erőforrás-, filesystem-, authorization-, persistence- és serving-határokat érint. A Velran ezért külön kezeli az általános, AppFs-be streamelt `Upload` értéket és a publikálható médiához való szigorúbb `Image` típust.

8.1. A mentális modell: három külön dolog

Feltöltésnél tartsd külön a következőket:

1. **kliens-metaadat:** eredeti fájlnev és a kliens által állított MIME-típus;
2. **tárolás:** a runtime által választott, AppFs-en belüli szerveroldali fájl;
3. **üzleti referencia:** amit az alkalmazás adatbázisban tárol és később felhasznál.

A kliens fájlneve nem storage path, a kliens `Content-Type` fejléce pedig nem biztonsági bizonyíték. Ezt a két értéket legfeljebb megjelenítési vagy audit-metaadatként kezeld.

8.2. Általános fájlfeltöltés: Upload

A route deklarációja megmondja a runtime-nak, hogy multipart fájlt vár, és melyik statikus AppFs alkönyvtárba helyezze azt:

```
route uploadFile POST "/upload"
  upload file<Upload> to "uploads"
  public => uploadFile;
```


Az action már nem a teljes fájlt kapja meg memóriában. Az `Upload` a feltöltés metaadatait adja át:

```
#[action]
fn uploadFile(ctx: ActionContext, file: Upload)
  -> Result<Redirect, PageError> {
  let savedPath = file.path;
  let originalName = file.filename;
  let contentType = file.contentType;
  let byteCount = file.bytes;
  return Ok(redirect(uploaded()));
}
```

A `redirect` typed `uploaded()` GET route helpert használ. Ennél fontosabb, hogy a raw `Upload` továbbra is private application data; attól még nem lesz public asset, hogy a handler látja a metadata-ját. Ez egyben fontos erőforrás-védelmi tulajdonság: a runtime streameli a multipart tartalmat, ezért egy nagyobb fájl nem válik automatikusan ugyanekkora request-szintű memóriaallokációvá.

8.3. AppFs: az alkalmazás írható fájltere

Az AppFs az alkalmazás számára kijelölt, korlátozott fájlrendszer-rész. Productionban külön data rootot adj neki; ne keverd a program binárisával, a statikus assetek könyvtárával vagy a rendszer más írható területeivel.

Példa szerverkonfiguráció:

```
[storage]
data_root = "/srv/velran/data"
fs_mode = "rwc"
max_upload_bytes = 16777216
max_image_pixels = 40000000
```

A rövid capability-jelölések jelentése:

- r** olvasás;
- w** írás;
- c** létrehozás és az upload commitjához szükséges műveletek.

Feltöltéshez tipikusan `rwc` szükséges. A byte-limit operátori hard limit: az alkalmazáskód nem emelheti meg tetszőlegesen egy route kedvéért.

Linuxon az AppFs confinement célja, hogy az alkalmazás a kijelölt root alól path traversal, symlink vagy magic-link trükkel se tudjon kitörni. Ettől még a normál Unix ownership és permission beállítások is szükségesek: a service user csak azt a könyvtárat kapja meg írhatónak, amelyre valóban szüksége van.

8.4. Mi történik egy feltöltés során?

A biztonságos feltöltési útvonal lépései fejlesztői szemmel:

1. a runtime ellenőrzi a request méretkorlátait és a route upload-sémáját;
2. a fájlt streaming módon egy confined staging helyre írja;
3. a storage kulcsot a szerver generálja, nem a kliens fájlnevéből képezi;

4. typed feltöltésnél további szerveroldali tartalomellenőrzés fut;
5. sikeres feldolgozás után a fájl atomikusan commitolható;
6. handler- vagy runtime-hiba esetén a runtime cleanupot kísérel meg.

Ne építs AppFs mellett párhuzamos fájlnev-szűrő írási útvonalat; az megkerülné a framework határát.

8.5. Képfeltöltés: preferáld az Image típust

CMS-ben, wikiben, hírportálon és adminfelületen a képekhez az általános Upload helyett az Image a megfelelő surface:

```
route saveHero POST "/admin/hero"  
  upload hero<Image> to "media" publish  
  auth role Editor  
=> saveHero;
```

Az Image ugyanazt a streaming multipart és AppFs útvonalat használja, de a szerver a tényleges fájl tartalmát is ellenőrzi. V1-ben PNG és JPEG fogadható el. A kliens által küldött MIME-típus nem authority.

Az SVG szándékosan nem általános képfeltöltési formátum: aktív tartalmat és script/XSS felületet hordozhat. A GIF, WebP vagy AVIF támogatását se feltételezd, ha az adott runtime-verzió dokumentációja nem mondja ki.

8.6. Byte-limit és pixel-limit nem ugyanaz

Képnél két külön erőforrás-probléma van. Egy fájl lehet kevés byte, miközben a dekódolt kép óriási pixelszáma. Ezért a max_upload_bytes mellett külön max_image_pixels limit is van.

```
max_upload_bytes = 16777216  
max_image_pixels = 40000000
```

A második limit a width * height értéket korlátozza. A production konfigurációban mindkettőt tudatosan állítsd be az alkalmazás feladatához; a „16 MB-os upload limit” önmagában nem kép-dekódolási védelem.

8.7. Kép referencia az adatbázisban

Az Image first-class scalar, ezért model mezőként és adatbázisban is tárolható canonical reprezentációval. Az alkalmazásnak nem kell – és nem is érdemes – a belső storage path részeit kézzel összeraknia vagy szétszednie.

Tipikus üzleti modell:

```
model Article {  
  id: i64  
  slug: Slug  
  title: String  
  body: String  
  hero: Image  
}
```

A lényeg az, hogy az adatbázisban *referencia* legyen, ne kliens fájlnev és ne kézzel gyártott publikus URL. Így a tárolási és a HTTP-kiszolgálási szabályok a runtime kontrollja alatt maradnak.

8.8. Biztonságos megjelenítés HTML-ben

Egy validált képet nem közvetlen string-interpolációval jelenítünk meg, hanem az erre szolgáló renderelővel:

```
#[page]
fn article(ctx: PageContext, article: Article)
  -> Result<Html, PageError> {
  return Ok(html {
    <article>
      <h1>{{ article.title }}</h1>
      @image(article.hero, article.title)
      @markdown(article.body)
    </article>
  });
}
```

Az `@image(image, alt)` a validált kép-referenciából készít HTML-t. Az `alt` szöveget escape-eli, a validált méreteket felhasználja, és a beépített read-only media endpointot célozza. Az `Image` közvetlen `{ image }` interpolációja compiler error, így a typed referencia nem csúszhat át véletlenül raw URL vagy markup szerepbe.

8.9. A beépített media endpoint

A runtime fenntartja a következő URL-prefixet:

```
/__velran/media/
```

Az alkalmazás ezzel ütköző route-ot nem deklarálhat. A media endpoint nem teszi HTTP-n böngészhetővé a teljes `data_root` könyvtárat: csak olyan AppFs könyvtárból szolgál ki képet, amelyet a program valamely `upload ...<Image> to "..."` `publish` route-ja deklarált.

Kiszolgáláskor a szerver a tartalmat ismét a képdetektor alapján kezeli, nem a kliens eredeti MIME-jára hagyatkozik. A válaszhoz biztonsági és cache headerek társulnak; a média kiszolgálása nem futtat alkalmazás-route-ot és nem hoz létre sessiont.

8.10. CMS-példa: hero kép feltöltése

Egy szerkesztői felületnél a gyakorlati folyamat általában két üzleti lépésből áll:

1. a szerkesztő feltölti a validált képet;
2. az alkalmazás az így kapott typed képreferenciát az adott cikkhez rendeli.

A route legalább autentikált legyen, admin/szerkesztő funkciónál pedig role policy is indokolt:

```
route saveHero POST "/admin/articles/:id<i64>/hero"
  upload hero<Image> to "media" publish
  auth role Editor
  => saveHero;
```

Ne abból indulj ki, hogy „csak kép, tehát veszélytelen”. Az upload route ugyanúgy state-changing POST: auth, CSRF/origin védelem, request-limit és auditálható üzleti művelet tartozhat hozzá.

8.11. Adatbázis és AppFs együtt alkothat üzleti állapotot

Ha DB-sor AppFs-képre hivatkozik, a DB és AppFs együtt alkot helyreállítandó üzleti állapotot. Kontrollált stop/drain mellett készíts összerendelt application DB + local-auth DB + AppFs snapshotot; egymástól független mentések nem automatikusan konzisztensek. A Redis session/cache/rate-limit állapota nem üzleti source of truth.

8.12. Statikus asset és feltöltött fájl nem ugyanaz

A programhoz tartozó CSS, JavaScript, ikon és egyéb verziózott asset más életciklusú, mint a felhasználói feltöltés. A static root legyen read-only deploy-artifact; az AppFs data root pedig külön, kontrolláltan írható runtime adat.

Ez a szétválasztás több hibát megelőz:

- deploy nem írja felül a felhasználói médiát;
- upload nem módosíthat alkalmazáskódot vagy verziózott assetet;
- backupnál egyértelmű, mi üzleti adat és mi újratelepíthető release-artifact;
- a reverse proxy szabályai is tisztábban kialakíthatók.

8.13. Mit ne csinálj?

- Ne használd a kliens fájlnevét közvetlen filesystem pathként.
- Ne bízz a multipart `Content-Type` értékében.
- Ne tedd publikussá a teljes AppFs data rootot Apache/Nginx aliasszal.
- Ne interpolálj `Image` értéket kézzel URL-be vagy HTML-be.
- Ne kezeld a DB-t és az AppFs-t egymástól független recovery state-ként, ha vannak köztük referenciák.
- Ne kerül meg az operátori upload- és pixel-limiteket alkalmazáskódból.

8.14. A jelenlegi media surface határai

A V1 media felülete tudatosan kicsi. Ne építs dokumentálatlan feltételezésekre. A jelenlegi alap nem ígér automatikus thumbnail/resize pipeline-t, WebP/AVIF konverziót, EXIF-normalizálást vagy metadata strippinget, teljes admin media browsert, S3/object-storage backendet vagy általános Markdown image syntaxot.

Ha az alkalmazásnak ezek valamelyike szükséges, azt külön architekturális feladatként kezeld, saját erőforrás-, security- és lifecycle-szabályokkal.

8.15. Fejlesztői ellenőrzőlista feltöltéshez

- | | |
|--------------------------------|---|
| 1. Auth/authorization rendben? | 2. AppFs root elkülönített és nem publikus? |
| 3. Ésszerű byte-limit? | 4. Képnél pixel-limit is? |
| 5. Képhez <code>Image</code> ? | 6. Filename/MIME csak metaadat? |
| 7. DB-ben typed referencia? | 8. HTML-ben <code>@image</code> ? |
| 9. DB + AppFs együtt mentve? | 10. Nincs reverse-proxy data-root alias? |

8.16. Ellenőrzött companion példák

A pontos aktuális szintaxisához használd:

- `examples/upload/app.vrn` – általános upload-határ;
- `examples/media/app.vrn` – validált media felület;
- `examples/typed-multipart/main.vrn` – típusos multipart dekódolás.

8.17. Gyakorlat: threat-modellezz egy feltöltést

Gyakorlási mód: támogatott. Használd az előszóban meghatározott támogatott módszert.

Egy képfeltöltésnél sorold fel a byte-méretet, pixelszámot, fájlnévet, MIME metaadatot, dekódolt formátumot, tárolási helyet, autorizációt, backupot és serving útvonalat. Döntsd el, mely érték lesz megbízható csak framework-validáció után, és mely marad örökre nem megbízható metaadat.

8.18. Tipikus hiba: uploadot teszünk a static root alá

A static asset deployment artefaktum; az upload módosuló alkalmazásadat. A kettő összekeverése gyengíti a rollback és backup szemantikát, és könnyen megkerülhet autorizációs vagy tartalomkezelési szabályokat.

8.19. Folyamatos mintaprojekt: Secure Notes

Opcionálisan csatolj egy képet a jegyzethez a típusos media útvonalon. A fájl maradjon a static release-fán kívül, az adatbázis csak a szükséges típusos referenciát tárolja, és ellenőrizd, hogy egy jegyzet törlése vagy cseréje nem teheti elérhetővé más felhasználó médiáját.

9. fejezet

JSON API, CORS és böngészős kliensintegráció

Tanulási út: Gyors út: API ág. Most olvasd, ha JSON/API klienst vagy külön frontend origint használsz.

Rust → Velran

Hozd Rustból: A typed skalárok, structok/modellek, collectionök, `Result` és újrahasznosítható business függvények ugyanazt az alkalmazásmagot adják, mint HTML route-oknál.

A Velran hozzáadja: A JSON input mód, zárt request séma, explicit public projection, content negotiation és CORS policy definiálja az API-határt.

Nem kell újratanulnod: Ne építs második business réteget csak a JSON kedvéért. Ugyanazt a typed logikát használd, csak a reprezentáció és a boundary contract változik.

Biztonsági invariáns: a CORS nem authorization

A CORS azt szabályozza, mely böngészős originekből indítható vagy olvasható cross-origin kérés. Nem hitelesíti a hívót, nem autorizál objektumot, és nem helyettesíti a normál request-életciklust.

Security diagnostic: SEC-DATA-004

Jelentés: Egy model explicit public projection nélkül kerülne kiadásra, ezért belső mezők véletlenül átléphetnék a response határt.

Javítás: Definiáld és add vissza a szándékolt public projectiont a belső model közvetlen szerializálása helyett.

A Velran nem kényszerít választásra a klasszikus szerver-renderelt HTML és a JSON API között. Ugyanabban az alkalmazásban lehetnek `Html`-t visszaadó oldalak, formot feldolgozó actionök és typed JSON endpointok. Ez hasznos akkor is, ha csak egy kisebb JavaScript komponens kér adatot, és akkor is, ha külön frontend alkalmazás használja a Velran szerveret API-ként.

A fejezet célja három működési modell tiszta szétválasztása:

1. **same-origin HTML + API:** a böngésző ugyanarról az originről tölti az oldalt és hívja az API-t;
2. **cross-origin publikus API:** külön origin hívja az API-t, de nincs session-cookie;
3. **cross-origin session auth:** külön frontend origin használ cookie-alapú sessiont, ezért CORS és CSRF policy együtt szükséges.

A legfontosabb alapelv: **a CORS nem autentikáció és nem CSRF-védelem.** A CORS böngészőoldali hozzáférési policy; az auth azt dönti el, ki a felhasználó, a CSRF pedig azt védi, hogy egy autentikált session ne legyen más originről jogosulatlanul felhasználható állapotmódosításra.

9.1. Ugyanez az életciklus érvényes az API-kra is

A JSON a reprezentációt változtatja meg, nem a trust modellt. Egy állapotmódosító API requestet ugyanabban a sorrendben érdemes végiggondolni:

```
HTTP request
-> parse: closed typed JSON schema
-> validate: bounds/refinements
-> authenticate: session or machine identity if required
-> authorize: object/permission/tenant decision
-> query / transaction
-> response: explicit JSON projection or stable error envelope
```

A CORS e mellett az életciklus mellett helyezkedik el, nem valamelyik lépés helyett. Azt dönti el, hogy egy böngésző-origin a frontend kód számára hozzáférhetővé teheti-e a választ. Nem hoz létre identityt, objektum-authorityt, CSRF-biztonságot vagy adatbázis-integritást. Ha a CORS-t a core authorization láncon kívül tartod, elkerülhető a gyakori tervezési hiba: egy engedélyezett origint engedélyezett felhasználónak tekinteni.

9.2. Az első typed JSON endpoint

Egy JSON endpoint handlerének visszatérési típusa `Json`. A `json(...)` a támogatott runtime értéket JSON reprezentációvá alakítja.

```
#[page]
fn statusApi(ctx: PageContext) -> Result<Json, PageError> {
  let healthy = true;
  return Ok(json(healthy));
}

route statusApi GET "/api/status" public => statusApi;
```

A sikeres válasz reprezentációja JSON, például:

```
HTTP/1.1 200 OK
Content-Type: application/json; charset=utf-8

true
```

A `json(value)` nem tetszőleges objektum-serializáló. Scalar érték közvetlenül kódolható, de database/domain model publikus JSON boundaryn explicit `expose(...)` projectionön keresztül léphet át. Így egy új model field—különösen egy sensitive mező—nem jelenhet meg észrevétlenül egy meglévő API-ban. A response shape tehát tudatos szerződés, nem egy runtime objektum automatikus lenyomata.

9.3. Adatbázisból JSON lista

A typed SQL és a JSON response közvetlenül összekapcsolható. A query továbbra is pontos model shape-et ad vissza, a JSON réteg pedig ezt reprezentálja.

```
model Product {
  id: i64
  name: String
  price: i64
}

#[query]
fn listProducts(db: Db, offset: i64)
  -> Result<Vec<Product>, DbError> sql {
  SELECT id, name, price
  FROM products
  ORDER BY id
  LIMIT 100 OFFSET :offset
}

#[page]
fn productsApi(ctx: PageContext, db: Db,
               offset: i64)
  -> Result<Json, PageError> {
  let products = listProducts(db, offset)?;
  return Ok(json(expose(products, id, name, price)));
}

route productsApi GET "/api/products"
  query offset<i64>
  validate offset range 0 1000000
  public => productsApi;
```

A fontos rész nem pusztán a JSON encoding. A reusable list query saját statikus row boundot hordoz; a kliens csak a bounded offsetet adja, nem az SQL cardinality authorityját. A route input typed és validált marad, a response pedig explicit public projectiont használ. Így egy később hozzáadott model field nem válik észrevétlenül az API részévé.

9.4. Typed JSON POST body

JSON request bodyhoz a route `json` input módot deklarál.

```
#[action]
fn echoApi(ctx: ActionContext,
           name: String,
           age: i64,
           active: bool)
  -> Result<Json, PageError> {
  return Ok(json(name));
}

route echoApi POST "/api/echo"
  json name<String> age<i64> active<bool>
  validate name length 1 100 age range 0 150
  auth user
  => echoApi;
```

Egy megfelelő kérés:

```
POST /api/echo HTTP/1.1
Content-Type: application/json
Accept: application/json
X-CSRF-Token: <session CSRF token>

{"name": "Alice", "age": 42, "active": true}
```

A JSON input schema **zárt**. A runtime hibának tekinti többek között az ismeretlen vagy hiányzó kulcsot, a duplikált kulcsot, a rossz scalar típust, valamint a nem támogatott nested object, array, float vagy null inputot. Ezek request-contract hibák, ezért **400 Bad Request** választ adnak. Hibás vagy hiányzó **Content-Type** JSON route-on **415 Unsupported Media Type**. A mező- és request-méretlimitek ugyanúgy érvényesek, mint más request body esetén.

A `json`, `form` és `upload` body módok ugyanazon POST route-on kölcsönösen kizárják egymást, és ezt már a compiler ellenőrzi. JSON route `application/json`, egyszerű form route `application/x-www-form-urlencoded`, upload route pedig `multipart/form-data` requestet vár. Egy endpointnak legyen egyértelmű request contractja.

9.5. Content negotiation és a 406 válasz

A kliens az `Accept` headerrel jelezheti, milyen reprezentációt tud fogadni. JSON route-nál támogatott például:

```
Accept: application/json
Accept: application/*
Accept: */*
```

Ha a handler JSON választ adna, de a kliens kizárólag `text/html` típust fogad el, a szerver **406 Not Acceptable** választ ad. Ez jobb annál, mintha a szerver csendben olyan reprezentációt küldene, amelyet a kliens explicit elutasított.

9.6. API hibák

JSON route-on a runtime által klasszifikált alkalmazáshibák ugyanazt az egyszerű envelope-ot használják: egyetlen stabil **error** kódot adnak vissza, miközben a részletes diagnosztika a szerveroldali logban marad. A kliens ne angol hibaüzenetek szövegét parse-olja, hanem a HTTP státuszt és ezt a kódot kezelje.

JSON-return route felismerése után a hibák JSON error envelope-ban érkeznek. Például:

```
HTTP/1.1 401 Unauthorized
Content-Type: application/json; charset=utf-8

{"error": "unauthorized"}
```

Tipikus error kódok:

- `bad_request`;
- `unauthorized`;
- `forbidden`;
- `csrf_failed`;
- `unsupported_media_type`;
- `database_unavailable`;
- `resource_limit`;
- `internal_error`.

Ez különösen fontos böngészős kliensnél. Egy autentikált JSON route névtelen kérésre 401-et ad, nem HTML login redirectet. A frontend így státuszkód alapján tud dönteni, nem kell HTML választ API hibaként értelmeznie.

9.7. Same-origin: az alapértelmezett és legegyszerűbb modell

Ha a HTML és az API ugyanazon scheme + host + port kombinációról érhető el, a böngésző szempontjából same-origin alkalmazásról beszélünk.

```
https://app.example/
https://app.example/api/products
```

Ilyenkor általában nincs szükség CORS konfigurációra. Egy egyszerű kliens:

```
const response = await fetch("/api/products?limit=20&offset=0", {
  headers: { Accept: "application/json" }
});

if (!response.ok) {
  throw new Error(`HTTP ${response.status}`);
}

const products = await response.json();
```

Ha nincs valódi igény külön frontend originre, maradj same-origin modellen: egy trust boundaryval és a CORS-bonyolultság nagy részével kevesebb.

9.8. CORS: exact origin allowlist

Cross-origin böngészős hozzáférés alapból nincs engedélyezve. A trusted server configban konkrét origineket kell felsorolni; wildcard origin nincs.

```
[web]
cors_origins = ["https://frontend.example"]
cors_allow_credentials = false
```

A CLI megfelelője:

```
velran-server --cors-origin https://frontend.example ...
```

Minden engedélyezett origin-t explicit sorolj fel; tetszőleges request `Origin` értéket soha ne tükrözz vissza.

9.9. Preflight

Bizonyos cross-origin requestek előtt a böngésző `OPTIONS` preflight kérést küld. A Velran csak akkor enged preflightot, ha:

- az `Origin` konfigurált;
- a cél GET/POST route létezik;
- a kért request header támogatott.

Támogatott például a `Content-Type`, az `Accept` és az `X-CSRF-Token`; a preflight a route/origin policy része, nem általános allow-all `OPTIONS` endpoint.

9.10. Credentialed CORS session-cookie-val

Ha a frontend külön originről használ Velran session authot, credentialed CORS szükséges:

```
[web]
cors_origins = ["https://frontend.example"]
cors_allow_credentials = true
```

Productionben ez csak HTTPS mellett engedélyezett. A session cookie ilyen konfigurációban `SameSite=None`; `Secure` tulajdonságot kap, mert a böngészőnek cross-site requestben is el kell küldenie.

A JavaScript oldalon a fetch kérésnek is explicit credential mód kell:

```
const response = await fetch("https://api.example/api/account", {
  credentials: "include",
  headers: { Accept: "application/json" }
});
```

A fetch `credentials` beállítása csak a megfelelő cookie-k elküldését engedi; alkalmazás-jogosultságot nem ad.

9.11. CSRF JSON API-n

Cookie-alapú sessionnél a böngésző a session cookie-t automatikusan kezeli, ezért állapotmódosító JSON requestnél ugyanúgy CSRF-védelem kell, mint HTML formnál.

Form POST esetén a token body mező:

```
_csrf=<token>
```

JSON POST esetén header:

```
X-CSRF-Token: <token>
```

Külön frontend originhez készíthető autentikált token endpoint:

```
#[page]
fn csrfApi(ctx: PageContext) -> Result<Json, PageError> {
    return Ok(json(csrfToken));
}

route csrfApi GET "/api/csrf" auth user => csrfApi;
```

A token ugyanahhoz a sessionhöz tartozik; a CORS nem helyettesíti a CSRF-validációt.

9.12. Teljes böngészős POST folyamat

Külön session-auth frontendnél a böngésző session cookie-t és sessionhöz kötött CSRF token-t küld; a runtime az üzleti handler előtt ellenőrzi az origint, authenticationt, CSRF-et és typed inputot.

Példa kliens:

```
const csrf = await fetch("https://api.example/api/csrf", {
  credentials: "include",
  headers: { Accept: "application/json" }
}).then(async r => {
  if (!r.ok) throw new Error(`CSRF HTTP ${r.status}`);
  return r.json();
});

const result = await fetch("https://api.example/api/echo", {
  method: "POST",
  credentials: "include",
  headers: {
    "Content-Type": "application/json",
    Accept: "application/json",
    "X-CSRF-Token": csrf
  },
  body: JSON.stringify({ name: "Alice", age: 42, active: true })
}).then(async r => {
  if (!r.ok) throw new Error(`API HTTP ${r.status}`);
  return r.json();
});
```

9.13. Reverse proxy és origin

Apache vagy Nginx mögött különösen fontos, hogy a public scheme/host és a trusted proxy konfiguráció összhangban legyen. Az alkalmazás ne a kliens által szabadon küldött forwarded headerekből találja ki, hogy milyen origin tekinthető biztonságosnak. A trusted proxy boundary alapját a telepítési fejezet mutatja be; a teljes `server.toml` kulcsreferencia a könyv végén található.

A CORS allowlistbe mindig a **böngésző által látott frontend origint** írd, például:

```
https://frontend.example
```

nem pedig a belső proxy vagy container címet.

9.14. Mit ne tegyél?

- Ne legyen permissive vagy reflektált CORS; explicit origineket használj.
- Ne kezeld a CORS-t authentication vagy CSRF-védelem helyett, és productionben ne küldj session credentialt HTTP-n.
- Egy POST route egy typed body módot használjon, az API input pedig legyen bounded.
- JSON kliens API-státuszokat, például 401/403-at kezeljen, ne HTML login redirectet.

9.15. Melyik architektúrát válaszd?

Helyzet	Ajánlott modell
szerver-renderelt oldal + kisebb JS	same-origin HTML + JSON API
azonos domainen futó SPA	same-origin API, CORS nélkül
külön publikus frontend, nincs session	exact-origin CORS credentials nélkül
külön frontend + session auth	exact-origin credentialed CORS + HTTPS + CSRF
server-to-server kliens	CORS nem releváns; az auth és hálózati policy a lényeg

9.16. Kipróbálható példa

A repository teljes mintája: `examples/json-api/app.vrn`.

Indulj same-originból; külön frontend origint és CORS-t csak akkor adj hozzá, ha a deployment ezt ténylegesen megköveteli.

9.17. Ellenőrzött companion példák

A repository companion példák:

- `examples/json-api/app.vrn` – JSON API route-flow;
- `examples/typed-query-form/main.vrn` – típusos query/form structok;
- `examples/json-typed-response/app.vrn` – típusos JSON response-határ.

Teljes request/response példához ezeket használd az izolált könyvrészlet helyett.

9.18. Gyakorlat: válaszd külön az API- és a CORS-tervet

Gyakorlási mód: támogatott. Használd az előszóban meghatározott támogatott módszert.

Írd le a JSON request/response contractot anélkül, hogy böngészőt említenél. Ezután külön dönts el, szüksége van-e egy másik originról futó böngészőnek engedélyre a híváshoz. Így nem keveredik össze a CORS az authenticationnel vagy az API-autorizációval.

9.19. Tipikus hiba: alaptól permissive CORS-t kapcsolunk be

A CORS böngészős origin-policy, nem általános hálózati security mechanizmus. Ahol ésszerű, preferáld a same-origin telepítést; ha kell cross-origin hozzáférés, az exact originöket és credential-viselkedést tudatosan sorold fel.

9.20. Folyamatos mintaprojekt: Secure Notes

Indulj az `examples/secure-notes/checkpoints/09-json-api/main.vrn` snapshotból és annak ellenőrzött JSON companion példáiból. Adj read-only JSON reprezentációt a hitelesített felhasználó jegyzeteiről. Ugyanazokat az autorizációs szabályokat tartsd meg, mint a HTML page esetén. CORS-t csak akkor adj hozzá, ha szándékosan külön frontend origint telepítesz; egyébként maradjon same-origin az API.

10. fejezet

Statikus assetek és frontend-integráció

Tanulási út: Gyors út: böngészős ág. Most olvasd, ha static assetet vagy progresszív böngészős enhancementet adsz hozzá.

A webalkalmazásnak nem csak dinamikus HTML- és JSON-válaszai vannak. CSS, ikon, kép, font, source map vagy egy külön frontend build artifactjai más életciklusú fájlok: a release részeként érkeznek, tartalmuk ideális esetben immutable, és a böngésző agresszíven cache-elheti őket.

A Velran ezért a statikus asseteket külön, **read-only static rootból** szolgálja ki. Ez nem ugyanaz a terület, mint az AppFs írható data rootja, és nem ugyanaz a fogalom, mint a felhasználó által feltöltött média.

Első olvasás: tanuld meg a három fájléletről, a fingerprintelt cache-elést és a V1 JavaScript/CSP határt. A szerver-vs-proxy kiszolgálási példák deployment referenciaként maradhatnak későbbre.

10.1. Három külön fájléletről

Egy production alkalmazásban legalább három fájlcsoporthoz érdemes külön kezelni:

	Példa	Írhatóság	Életről
release	CSS, font, logo	read-only	deploykor cserélődik
runtime data	export, generált fájl	AppFs policy szerint	alkalmazás futása közben
user media	feltöltött kép	kontrolláltan írható	üzleti adattal együtt

Tipikus könyvtárszerkezet:

```
/usr/local/bin/velran-server
/usr/local/bin/velran-cli
/srv/myapp/releases/2026-09-04/
  main.vrn
  migrations/
  public/
    css/
    img/
```

```
    fonts/  
/srv/myapp/data/  
/run/secrets/velran/
```

A `public/` legyen deployment artifact. A `data/` ezzel szemben runtime állapot. Ha a kettőt összekevered, egy upload felülírhat release-fájlt, vagy egy rollback véletlenül felhasználói adatot cserélhet vissza.

10.2. A static root konfigurálása

A server configban a statikus namespace külön szekció:

```
[static_assets]  
root = "/srv/myapp/current/public"  
url_prefix = "/assets/"  
max_asset_bytes = 8388608  
max_age_secs = 300  
immutable_max_age_secs = 31536000  
precompressed = true
```

Ugyanez CLI-ből is beállítható. A fontosabb opciók:

```
--static-root  
--static-url-prefix  
--max-static-asset-bytes  
--static-max-age-secs  
--static-immutable-max-age-secs
```

A static prefix rezervált route namespace. Ha az URL prefix `/assets/`, az application ne deklaráljon ezzel ütköző dinamikus vagy statikus route-ot. Az ütközést a szerver fail-closed módon kezelje: jobb startupkor hibázni, mint requestenként bizonytalan route-választást végezni.

10.3. Fingerprint: a hosszú cache kulcsa

A statikus asset tartalma és URL-je legyen összekötve. Például:

```
public/  
  css/app.01234567.css  
  img/logo.a91f028c.svg  
  fonts/ui-f1e2d3c4.woff2
```

A Velran fingerprinteltnek tekinti azt a fájlnévet, amelyben legalább 8 hexadecimális karakterből álló, ponttal, kötőjellel vagy underscore-ral határolt token szerepel. Ilyen fájlnál a response hosszú immutable cache-t kaphat:

```
Cache-Control: public, max-age=31536000, immutable
```

Nem fingerprintelt fájlnál a biztonságosabb alap rövid és revalidálható:

```
Cache-Control: public, max-age=300, must-revalidate
```

A lényeg: **ne ugyanazon URL mögött cseréld le a tartalmat egyéves cache mellett.** Új CSS buildhez új fingerprintelt fájlnev tartozzon. A régi fájl akár maradhat is a release-ben addig, amíg a régi HTML-reprezentációk kifuthatnak.

10.4. Egy minimális asset build pipeline

A Velran nem akar frontend build rendszert előírni. A pipeline feladata egyszerűen az, hogy determinisztikus outputot készítsen a `public/` könyvtárba.

Egy kis alkalmazásnál a forrás lehet például:

```
assets-src/  
  app.css  
  logo.svg  
public/
```

A release build lépései:

1. tiszta `public/` könyvtár létrehozása;
2. CSS/font/kép build vagy másolás;
3. content hash képzése;
4. fingerprintelt fájlnev létrehozása;
5. opcionális Brotli/gzip siblingek készítése;
6. az alkalmazásban a fingerprintelt URL használata.

A hash nem security token. A feladata az, hogy a fájlnev a tartalom verzióját azonosítsa.

10.5. ETag és conditional GET

Minden statikus reprezentáció saját **ETag** értéket kap. A böngésző később küldhet:

```
If-None-Match: "velran-..."
```

Ha a reprezentáció nem változott, a szerver `304 Not Modified` választ adhat body nélkül. GET mellett HEAD is támogatott.

A static request nem hoz létre sessiont és nem küld session cookie-t. Ez fontos teljesítmény- és security tulajdonság: egy CSS vagy font letöltése ne váljon session-lifecycle eseménnyé.

10.6. Precompressed Brotli és gzip

A Velran server request közben nem tömöríti a statikus fájlt. A release pipeline opcionálisan előre elkészítheti:

```
app.01234567.css  
app.01234567.css.br  
app.01234567.css.gz
```

A kliens **Accept-Encoding** értéke alapján a szerver a Brotli, majd gzip, végül az eredeti reprezentáció közül választ. Tömörített válasznál a header például:

```
Content-Encoding: br  
Vary: Accept-Encoding
```

Ennek két előnye van: nincs requestenkénti compression CPU-költség, és a web request nem nyit dinamikus tömörítési erőforrás-felületet.

10.7. MIME, nosniff és path confinement

A server ismert webes kiterjesztésekhez MIME típust rendel, ismeretlen extensionnél pedig `application/octet-stream` típust használ. A válasz mindig kap:

```
X-Content-Type-Options: nosniff
```

A static path szándékosan szigorú. Jó:

```
/assets/css/app.01234567.css
```

Tiltott többek között a `...`, encoded traversal, backslash, üres path komponens és control karakter. A V1 static pathban percent-encoding sincs; a build pipeline használjon URL-safe fájlneveket.

Linuxon a static root ugyanahhoz a confinement szemlélethez igazodik, mint az AppFs: a kiszolgálás nem válhat tetszőleges host filesystem olvasássá symlink vagy path traversal segítségével.

10.8. CSS, kép és font a Velran HTML-ben

A V1 CSP same-origin style, image, font és media asseteket enged. Egy release-ben ezért természetes minta:

```
/assets/css/app.01234567.css  
/assets/img/logo.a91f028c.svg  
/assets/fonts/ui-f1e2d3c4.woff2
```

A HTML-ben mindig a release-hez tartozó fingerprintelt URL jelenjen meg. A sablon ne felhasználói inputból állítson elő asset URL-t; az assetnév a build része, nem request adat.

10.9. Fontos V1-határ: JavaScript kiszolgálható, de Velran HTML-ben nem fut

A static server ismeri a `.js` és `.mjs` MIME típust, ezért ilyen build artifact fizikailag kiszolgálható. Ez azonban **nem jelenti azt, hogy a Velran által renderelt HTML scriptet futtathat.**

A V1 HTML security modellben:

- a CSP nem nyit `script-src` engedélyt;
- a compiler a `<script>` markupt elutasítja;
- nincs inline-script vagy „unsafe” escape hatch.

Ez tudatos security határ. A könyv előző fejezetének JavaScript `fetch` példái ezért kétféleképpen értelmezendők:

1. külön frontend alkalmazás fut a saját originjén, és Velran JSON API-t hív;

2. a kód a browser API kliens viselkedését szemlélteti, nem azt, hogy V1 Velran HTML-be script tag írható.

Ha az alkalmazás klasszikus Velran szerver-renderelt oldal, építs HTML + CSS + typed form/action modellre. Ha teljes JavaScript frontend szükséges, kezeld azt külön frontendként, és a Velran legyen typed JSON backend.

10.10. Velran szolgálja ki vagy Apache/Nginx?

Két működő deployment modell van.

1. Velran-owned static namespace

Ez legyen az alapértelmezett. A reverse proxy minden requestet a Velran szervernek ad, és a Velran kezeli az `/assets/` namespace-t is:

```
Internet
|
Apache / Nginx :443
|
127.0.0.1:8080 Velran
|-- application routes
`-- /assets/*
```

Előnye, hogy a fingerprint-, ETag-, precompressed-, nosniff- és path-policy egy helyen marad. Kis és közepes alkalmazásnál ez általában bőven elég.

2. A reverse proxy szolgálja ki közvetlenül

Nagy forgalomnál dönthetsz úgy, hogy az Apache vagy Nginx közvetlenül a read-only release public/ könyvtárból szolgál. Ekkor viszont **neked kell reprodukálnod** a fontos contractot:

- read-only, külön static root;
- traversal/symlink elleni biztonságos path-kezelés;
- helyes MIME + **nosniff**;
- fingerprintelt fájlokra hosszú immutable cache;
- nem fingerprintelt fájlokra rövid/revalidálható cache;
- Brotli/gzip esetén helyes **Vary**;
- a namespace ne ütközzön application route-tal.

Ez már operátori optimalizáció, nem szükséges ahhoz, hogy az alkalmazás helyes legyen.

10.11. Nginx minta közvetlen static kiszolgálásra

Egy egyszerű kiindulás:

```
server {
    listen 443 ssl;
    server_name app.example;

    location /assets/ {
        alias /srv/myapp/current/public/;
        try_files $uri =404;
        add_header X-Content-Type-Options nosniff always;
    }

    location / {
        proxy_pass http://127.0.0.1:8080;
        proxy_set_header Host $host;
        proxy_set_header X-Forwarded-Proto https;
    }
}
```

Ez csak topológiai alaplanta. A production konfigurációban a cache policy-t, precompressed fájlokat és symlink/release szabályokat is ugyanahhoz az immutable deployment modellhez kell igazítani.

10.12. Apache minta közvetlen static kiszolgálásra

Apache esetén a gondolat ugyanaz:

```
Alias /assets/ /srv/myapp/current/public/

<Directory /srv/myapp/current/public/>
    Require all granted
    Options -Indexes
    AllowOverride None
    Header always set X-Content-Type-Options "nosniff"
</Directory>

ProxyPass          / http://127.0.0.1:8080/
ProxyPassReverse   / http://127.0.0.1:8080/
```

Ha Apache közvetlenül szolgálja az asseteket, az `Alias` szabálynak a proxy catch-all előtt kell érvényesülnie, és a cache headereket is explicit konfigurálni kell.

10.13. Deploy: asset és alkalmazáskód ugyanahhoz a release-hez tartozzon

A legkönnyebben elkerülhető assethiba az, amikor az új HTML már új CSS fájlnevet hivatkozik, de a static rootban még a régi release van – vagy fordítva.

Ezért a helyes sorrend:

```
build application
-> build/fingerprint assets
-> optional .br/.gz
-> immutable release directory
-> preflight
```

```
-> atomic current switch
-> controlled restart/reload
```

Az alkalmazáskód és a `public/` könyvtár ugyanazon release artifact része legyen. Így rollback-kor mindkettő együtt áll vissza.

10.14. Ellenőrzés curl-lel

Deploy után legalább ezeket érdemes ellenőrizni:

```
curl -I https://app.example/assets/css/app.01234567.css
curl -I -H 'Accept-Encoding: br' \
  https://app.example/assets/css/app.01234567.css
curl -I -H 'If-None-Match: "velran-..."' \
  https://app.example/assets/css/app.01234567.css
```

Ellenőrizd a `Content-Type`, `Cache-Control`, `ETag`, `X-Content-Type-Options`, valamint tömörített reprezentációnál a `Content-Encoding` és `Vary` headert.

10.15. Gyakorlati döntési szabály

Helyzet	Ajánlott megoldás
szerver-renderelt Velran oldal	Velran static root + CSS/image/font
kis/közepes production	reverse proxy mindent Velrannak továbbít
nagy statikus forgalom	Apache/Nginx direct static, azonos policy-vel
teljes JavaScript frontend	külön frontend deployment + Velran JSON API
user upload	AppFs/media pipeline, nem static root

A legfontosabb szabály: **a statikus asset deployment artifact, nem alkalmazásadat**. Ha ezt a határt megtartod, a cache, rollback, backup és security modell egyszerre lesz egyszerűbb.

10.16. Ellenőrzött companion példa

Az aktuális static-asset companion az `examples/static-assets/app.vrn`. A release asseteket tartsd read-only módon, az AppFs/runtime állapottól elkülönítve, a fejezet deployment modelljének megfelelően.

10.17. Gyakorlat: osztályozd a frontend minden fájlját

Gyakorlási mód: támogatott. Használd az előszóban meghatározott támogatott módszert.

Egy page által használt minden fájlj sorolj immutable deployment asset, generált HTML, JSON response vagy módosuló alkalmazásadat kategóriába. Ellenőrizd, hogy a static root alatt kizárólag immutable deployment asset található.

10.18. Tipikus hiba: a frontend-kényelem eltünteti a trust boundaryt

Egy JavaScript frontend nem változtatja meg a szerveroldali autorizáció követelményeit. A kliensoldali validáció és elrejtett vezérlők UX-eszközök; minden authoritative validáció, autorizáció és módosítási szabály a szerver határán marad.

10.19. Folyamatos mintaprojekt: Secure Notes

Adj kis stylesheetet és szükség esetén progresszív JavaScript enhancementet a jegyzet UI-hoz. Az alkalmazás maradjon helyes kliensoldali ellenőrzésekbe vetett bizalom nélkül is, a felhasználói tartalom pedig maradjon a static asset fán kívül.

11. fejezet

Nagyobb alkalmazások szervezése

Tanulási út: Teljes út / skálázódó kódszerkezet. Akkor olvasd, amikor az egymodulos alkalmazást már nehéz biztonságosan módosítani.

Rust → Velran

Hozd Rustból: A modulokra bontás, qualified nevek, explicit függőségek és összetartozó kód együtt tartása ugyanazokat a Rustos tervezési ösztönöket használja.

A Velran hozzáadja: A Velran application-root source graphot old fel, domain object namespace-t ad, és a HTTP route policyt külön kezeli a forrás/modul struktúrától.

Nem kell újratanulnod: Nem kell újratanulnod a modularitást. Használd a megszokott modulhatárokat; csak a source location, domain namespace és külső HTTP exposure különbségét tanuld meg.

Ahogy nő az alkalmazás, a change locality fontosabb lesz a fájlszámnál. A `mod` a forrásszerkezetet, az `object` a domain API névterét szervezi; a HTTP exposure maradjon `route`-ban.

11.1. Három külön kérdés

A nagyobb alkalmazás szerkezetét érdemes három külön réteggént kezelni:

Eszköz	Feladata
<code>mod</code>	Melyik fájlban, illetve forrásmodulban található a kód?
<code>object</code>	Melyik üzleti fogalomhoz tartozik a model, query, page vagy action?
<code>route</code>	Milyen HTTP felület és milyen auth/cache/rate-limit policy teszi elérhetővé?

Ez a szétválasztás tudatos. Egy domain object nem kap automatikusan adatbázist, current usert, fájlrendszert vagy hálózati hozzáférést, és egy modul sem jelent HTTP policy-t.

11.2. A `mod` nem szöveges `include`

Egy kisebb, többfájlos projekt már így nézhet ki:

```
myapp/  
|-- main.vrn  
|-- models.vrn
```

```
|-- queries.vrn
|-- components.vrn
|-- pages.vrn
|-- actions.vrn
|-- migrations/
`-- public/
```

A belépési pont:

```
mod models;
mod queries;
mod components;
mod pages;
mod actions;
```

A `mod` nem C-szerű szövegbeillesztés, és nem önti a deklarációkat globális névtérbe. A compiler az explicit source graphot tölti be, ellenőrzi és egyetlen programmá fordítja. Egy modul saját névtérrel bír: például a `queries.vrn` deklarációira más modulból `queries::name` alakban kell hivatkozni. A modulútvonal application-root relatív; nested forrásfájlt explicit, például `mod article::admin;` deklarációval kell felvenni. Nincs automatikus almappa-discovery vagy relatív `../` modulútvonal.

11.3. Mikor válts feature-alapú szerkezetre?

A `models.vrn`, `queries.vrn`, `pages.vrn` típus szerinti bontás jó első lépés, de nagyobb alkalmazásnál könnyen azt eredményezi, hogy egyetlen feature módosításakor öt-hat fájlt kell párhuzamosan megnyitni. Ilyenkor jobb üzleti terület szerint bontani:

```
main.vrn
article.vrn
article/
|-- admin.vrn
`-- components.vrn
account.vrn
account/
`-- profile.vrn
invoice.vrn
invoice/
`-- admin.vrn
routes.vrn
migrations/
public/
```

A `main.vrn` ilyenkor inkább alkalmazás-kompozíció:

```
mod article;
mod article::admin;
mod article::components;
mod account;
mod account::profile;
mod invoice;
mod invoice::admin;
mod routes;
```

A cél a *change locality*: egy üzleti változtatás kódja maradjon többnyire egy feature könyvtáron belül.

11.4. Domain object: üzleti névtér, nem OOP-osztály

A globális nevek számát az *object* csökkenti. Például:

```
object Article {
  model {
    id: i64
    slug: Slug
    title: String
    body: String
    authorUsername: String
  }

  #[query]
  fn bySlug(db: Db, slug: Slug)
    -> Result<Article, DbError> sql {
    SELECT id, slug, title, body, authorUsername
    FROM articles
    WHERE slug = :slug
  }

  #[page]
  fn show(ctx: PageContext, db: Db, slug: Slug)
    -> Result<Html, PageError> {
    let article = Article.bySlug(db, slug)?;
    return Ok(html {
      <article>
        <h1>{{ article.title }}</h1>
        @markdown(article.body)
      </article>
    });
  }
}
```

A használat így beszédes:

```
let article = Article.bySlug(db, slug)?;
```

Később ugyanilyen természetesek a rövid domainnevek: `Invoice.approve`, `Invoice.cancel`, illetve `UserAccount.disable`.

Mit tartalmazhat egy *object*?

A framework domain object egy model köré rendezheti a hozzá tartozó műveleteket. Az alábbi blokk csak szerkezeti váz, nem önmagában fordítható Velran forrás:

```
object Article {
  model { ... }
  #[query]
  fn bySlug(...) ...
  #[page]
  fn show(...) ...
```

```

    #[action]
fn publish(...) ...
    #[component]
fn Card(...) ...
    #[layout]
fn EditorLayout(...) ...
}

```

Pontosan egy model blokk tartozik egy framework domain **object**-hez. Ez a konstrukció framework-owned model/query/page/action neveket rendez egybe; külön fogalom a normál pure/domain értékekhez használt Rust-szerű **struct** + inherent **impl** surface-től. A framework object továbbra sem jelent öröklést, rejtett capabilityt, virtual dispatchot vagy reflectiont. Rust-szerű értékeknél támogatott az associated konstruktor (például **Type::new**) és az immutable **&self** metódus; az általános mutable/owned receiver továbbra is szándékosan fail-closed.

11.5. Package-határok újrahasznosítható alkalmazáskódhoz

Ha kódot több alkalmazás között kell újrahasznosítani vagy stabil library-határ mögé kell zárni, ne növelj korlátlanul az application modulgráfot: emeld lokális package-be. Minden package saját **velran.toml**-lal, alpból privát API-val és korlátozott lokális dependency-gráffal rendelkezik. A közvetlen dependency explicit path dependency; a tranzitív dependency enkapszulált marad, nem válik ambient application névtérre.

A library surface tervezéséhez **pub struct**, **pub fn**, publikus inherent metódus és a szűk package-root modul-re-export használható. A framework authority maradjon az alkalmazásban: egy reusable library **pub** hatására sem kaphat HTTP route-ot, actiont, adatbázis-/hálózati authorityt vagy más webes capabilityt. Így a package-dekompozíció architekturális határ, nem capability-eszkalációs mechanizmus.

11.6. A dependency továbbra is explicit

A domain namespace nem rejt el capabilityket. Ez jó:

```

#[query]
fn byId(db: Db, id: i64)
    -> Result<Article, DbError> sql {
    SELECT id, slug, title, body, authorUsername
    FROM articles
    WHERE id = :id
}

```

Az action ugyanilyen explicit módon kapja meg a szükséges capabilityket. A következő blokk szándékosan rövidített szerkezeti példa; az állapotváltás konkrét typed queryjét a CRUD és üzleti-audit fejezetek mutatják be:

```
#[action]
fn publish(ctx: ActionContext, db: Db, id: i64)
  -> Result<Redirect, PageError> {
  let article = Article.byId(db, id)?;
  authorize article owner authorUsername or role Publisher;
  // ... állapotváltás tranzakcióban ...
  return Ok(redirect(adminArticles()));
}
```

A handler aláírásából továbbra is látszik, hogy adatbázishoz fér hozzá. Ugyanez igaz AppFs-re, authra és named hálózati egressre is: a domain object nem válhat rejtett service locatorrá.

11.7. Miért maradjon a route top-level?

A route tudatosan nem olvad bele teljesen az objectbe:

```
route articleShow GET "/cikk/:slug<Slug>"
  public cache public ttl 60
  => article::Article.show;

route articlePublish POST "/admin/cikk/:id<i64>/publish"
  auth role Publisher
  invalidate cache articleShow
  => article::Article.publish;
```

Így egy route-registry vagy egy code review során egy helyen látszik:

- az URL és a HTTP method;
- az authentication/authorization követelmény;
- a cache policy és invalidáció;
- a rate limit vagy resource profile;
- a logokban és auditban megjelenő globális route-név.

A domain member neve lehet rövid és helyi jelentésű, például `Article.publish` a saját modulján belül; külön `routes.vrn` fájlból `article::Article.publish` szükséges. A route neve viszont maradjon globálisan beszédes, például `articlePublish`.

11.8. Ajánlott névkonvenció

Elem	Forma	Példa
object	PascalCase üzleti főnév	Article, Invoice, UserAccount
member	camelCase, domainen belül rövid	bySlug, publish, approve
route	globálisan auditálható név	articleShow, invoiceApprove
module	feature vagy technikai egység	article, account, routes

11.9. Egy CMS lehetséges szerkezete

Egy közepes CMS-nél jó kompromisszum lehet:

```
main.vrn
routes.vrn
article.vrn          # Article object + alap queryk
article/
|-- admin.vrn        # szerkesztes/publikalas
`-- components.vrn   # Card, listak
media.vrn
media/
`-- admin.vrn
account.vrn
migrations/
public/
`-- app.css
```

A publikus és admin HTTP policy továbbra is a route-rétegben marad. Az article feature viszont együtt tartja a cikk üzleti modelljét és műveleteit.

11.10. Mit ne csinálj?

Ne készíts minden deklarációból külön fájlt

A túl apró modulok több navigációt okoznak, mint amennyi szerkezetet adnak. Egy ötven soros feature-nek nem kell nyolc fájl.

Ne építs rejtett „service” réteget

Ha egy művelet DB-t, AppFs-t vagy auth contextet használ, az maradjon látható a typed aláírásban. A Velran egyik értéke éppen az, hogy a capability-határ explicit.

Ne duplikáld a route policy-t a handlerben

A route-level auth, cache, rate limit és resource policy deklaratív szerződés. A domain-specifikus objektumszintű authorization természetesen a handlerben maradhat, mert az üzleti rekordtól függ.

Ne használj szabad Stringet üzleti állapotnak

Ha egy domain Draft, Review, Published vagy Cancelled állapotokból áll, használd az enumot. A projektstruktúra akkor segít igazán, ha a domain contract is typed.

11.11. Refaktorálási út: egy fájlból nagyobb alkalmazás

Nem kell a projekt első napján végleges könyvtárhierarchiát tervezni. Jó evolúciós út:

1. Kezdj egy `main.vrn` fájlal, amíg az alkalmazás kicsi.
2. Válaszd külön a `model/query/page/action` csoportokat, amikor a fájl már nehezen áttekinthető.
3. Ha egy feature több technikai fájlra szétszóródik, válts feature-alapú modulokra.
4. Ha sok globális üzleti név keletkezik, rendezd őket `object` namespace-be.

11.12. Ellenőrzött companion példák

Modulstruktúrához használd az `examples/module-namespaces/main.vrn` példát, általános többfájlos kiindulóponthoz az `examples/starter-project/main.vrn` fájlt, a könyv folyamatos mintaprojektjéhez pedig az `examples/secure-notes/main.vrn` példát. Mindhárom compilerrel ellenőrzött, ezért jobb copy/paste forrás a rövidített architektúráis vázlatoknál.

11.13. Gyakorlat: refaktorálj változási lokalitás alapján

Gyakorlási mód: önálló. Használd az előszóban meghatározott önálló módszert.

Válassz egy feature-t, és azonosíts minden fájlt, amely megváltozik a feature módosításakor. Ha egy kis üzleti változás sok, egymástól távoli technikai réteg szerkesztését kívánja, fontold meg a feature vagy domain objektum körüli csoportosítást. Csak akkor refaktorálj, amikor a dependency-irány már világos.

11.14. Tipikus hiba: csak a fájlszám miatt modularizálunk

A hasznos modul koherens contractot birtokol és csökkenti az egyszerre fejben tartandó fogalmakat; kerülj a monolit fájlokat és a mesterséges egyfüggvényes modulokat.

11.15. Folyamatos mintaprojekt: Secure Notes

Mozgasd át a Secure Notes alkalmazást a kezdeti egyfájlos alakból olyan struktúrába, ahol a jegyzethez tartozó modellek, query-k, page/action elemek és policy-k könnyen megtalálhatók. A külső route-ok és security-viselkedés közben ne változzon.

III. rész

Alkalmazott fejlesztés, integráció és assurance

12. fejezet

Gyakorlati alkalmazások: wiki és CMS

Tanulási út: Teljes út / domain példa. Használd összetettebb workflow- és publication-state esettanulmányként.

A wiki és CMS példák a korábbi feature-öket domain workflow-vá állítják össze. Compilerrel ellenőrzött forrásuk az `examples/wiki/main.vrn` és `examples/news-site/`; rövidített listingnél ezeket használd source of truthként.

12.1. Egyszerű wiki

Egy wiki minimális domainje oldalból áll: stabil azonosító, slug, cím és Markdown törzs. Az alábbi minta a nyelvi elemek összekapcsolását mutatja.

Modell

```
model WikiPage {
  id: i64
  slug: Slug
  title: String
  body: String
}
```

Lekérdezés

```
#[query]
fn wikiBySlug(db: Db, slug: Slug)
  -> Result<WikiPage, DbError> sql {
  SELECT id, slug, title, body
  FROM wiki_pages
  WHERE slug = :slug
}
```

Publikus oldal

```
#[page]
```

```
fn wikiShow(ctx: PageContext, db: Db, slug: Slug)
  -> Result<Html, PageError> {
  let page = wikiBySlug(db, slug)?;
  return Ok(html {
    @layout(Main, page.title) {
      <article>
        <h1>{{ page.title }}</h1>
        @markdown(page.body)
      </article>
    }
  });
}

route wikiShow GET "/wiki/:slug<Slug>" public => wikiShow;
```

Szerkesztő form

```
form WikiForm {
  title<String>
  body<String>
  validate title length 2 200 body length 1 300000
}
```

A módosító route legyen autentikált, például `auth role Editor`. A write query `Transaction` capabilityt kapjon. A cím alapján generált slugot ne írd át vakon minden szerkesztéskor, ha a régi URL-ek stabilitása fontos.

Canonical slug és alias

Érettebb wikiben külön slug-alias/history tábla tartja a régi címeket. A runtime canonical route-ból tud 301-es redirectet készíteni, így nem kell tetszőleges redirect URL-t építeni.

Mit adj hozzá egy valódi wikihez?

- optimistic locking a konkurens szerkesztésekhez;
- üzleti audit trail a módosításokhoz;
- objektumszintű authorization, ha nem minden oldal azonos jogosultságú;
- backup/restore policy az adatbázisra és az esetleges AppFs adataira.

12.2. CMS és híroldal

A repository `examples/news-site/` példája már egy kis CMS magját mutatja: model, typed SQL, layout, Markdown, szerkesztői action és public cache.

Projektstruktúra

```
main.vrn
models.vrn
queries.vrn
components.vrn
pages.vrn
actions.vrn
```

Cikkmodell

```
model Article {
  id: i64
  slug: Slug
  title: String
  body: String
}
```

A szerkesztői input külön named form-sémát kap:

```
form ArticleForm {
  title<String>
  body<String>
  validate title length 2 200 body length 1 300000
}
```

Publikus cikkoldal

```
#[page]
fn article(ctx: PageContext, db: Db, slug: Slug)
  -> Result<Html, PageError> {
  let article = articleBySlug(db, slug)?;
  return Ok(html {
    @layout(Main, article.title) {
      <article>
        <h1>{{ article.title }}</h1>
        @markdown(article.body)
      </article>
    }
  });
}

route article GET "/cikk/:slug<Slug>"
  public cache public ttl 60
  => article;
```

Public cache csak olyan oldalra való, amely nem session- vagy userfüggő.

Szerkesztői létrehozás

```
#[action]
fn create(ctx: ActionContext, db: Db,
  title: String, body: String)
  -> Result<Redirect, PageError> {
  let articleSlug = slug(title);
  transaction db {
    insertArticle(tx, articleSlug, title, body)?;
  }
  return Ok(redirect(adminArticles()));
}

route articleCreate POST "/admin/articles"
  form ArticleForm
  auth role Editor
```

```
invalidate cache article
=> create;
```

A fenti invalidáció route-szintű generationváltás: nem csak az új slug egyetlen cache-kulcsát törli, hanem az `article` route korábbi generationjét teszi elérhetetlenné. Egyszerű mintában ez helyes és kiszámítható, de nagy forgalmú CMS-nél érdemes tudatosan mérlegelni, mely publikus route-okat kell valóban invalidálni egy adott módosítás után.

CMS-ből production rendszer

A következő lépések adják a gyakorlati production CMS gerincét:

- publikálási és domain-invariánsok külön validációja;
- draft/published állapot enumként;
- canonical slug history;
- optimistic locking szerkesztéskor;
- business audit trail;
- media library és AppFs a képekhez;
- role-based szerkesztő/publisher policy;
- rate limit és resource profile a drágább route-okra;
- backup, restore drill és kontrollált deployment.

A CMS-ben az eredeti Markdown legyen a source of truth; ne tárolj alkalmazásból származó raw renderelt HTML-t.

12.3. Szerveroldali Markdown renderelés és cache

Publikált tartalomnál alapértelmezetten szerveroldali Markdown renderelést használj. Az authoritative útvonal:

```
SQL-ben tarolt Markdown
-> typed query stabil id/slug alapján
-> @markdown(...)
-> biztonságos HTML válasz
-> opcionális route-szintű public cache
```

Az adatbázisban az eredeti Markdown maradjon a source of truth. Ne tárold el az alkalmazás által generált HTML-t pusztán azért, hogy megspórold a renderelést. A szerveroldali renderer nem enged raw HTML-t, és a link targeteket saját policy szerint kezeli, így a renderelés a framework HTML-security boundaryján belül marad ahelyett, hogy a bizalmat böngészős Javascriptre helyezné át.

Publikus, nem személyre szabott dokumentumnál route cache használható, ha a freshness contract ezt megengedi. A cache a kész választ tárolja; a cache-clause elhagyásával ugyanaz a renderelési útvonal marad cache nélkül. Egy minimális pár:

```
route markdownDocumentLive GET "/documents/:id<i64>"
  public
  => markdownDocument;

route markdownDocumentCached GET "/documents-cached/:id<i64>"
  public
  cache public ttl 120
  => markdownDocument;
```

A canonical end-to-end példa az `examples/markdown-sql-cache/`. SQLite-, PostgreSQL- és MariaDB-sémaváltozatot is tartalmaz, és megmutatja, hogyan táplál egy id-alapú typed SQL olvasás @markdown renderelést. Böngészőoldali Markdown renderelést csak szerkesztői preview-hoz használj, ha javítja az interakciót; a publikált reprezentációt továbbra is a szervertől készítesd, hogy a sanitization, cache-policy, CSP-feltételezések és JavaScript nélküli kliensek szempontjából a szervertől maradjon authoritative.

12.4. Demo CMS-től a szerkesztőségi workflow-ig

Production tartalomkezelőhöz nem elég a CRUD; workflow is kell. Jó határvonal, ha a publikus renderelés, a szerkesztői mutáció, a concurrent-edit védelem, a média publikálása és az audit külön mechanizmus marad, amelyeket explicit módon komponálunk.

Konkurens szerkesztés

Emberi szerkesztő gyakran percekig nyitva tart egy formot. Adj `version` oszlopot a rekordhoz, és update-nél használj `Changed` eredményt, hogy egy régi szerkesztés conflictot kapjon ahelyett, hogy felülírná az újabb munkát. Az edit oldal typed form inputként viszi a jelenlegi verziót; a mutáció atomikusan ellenőrzi az `id + version` párost, és siker esetén növeli a verziót.

Fontos állapotváltásnál, például publish vagy archive esetén, a mutáció és a business audit rekord ugyanabban az adatbázis-tranzakcióban fusson. Az audit az autentikált actort és a statikus üzleti actiont rögzíti; authorizationt nem helyettesít.

```
transaction db {
  Article.publishChanged(tx, id, version)?;
  audit Article id action publish
    from article.status to ArticleStatus::Published;
}
```

Stale verziónál a végrehajtás az audit statement előtt megáll, így sikertelen publikálásból nem lesz hamis success audit rekord.

A média state machine-hoz tartozik

A böngésző által küldött filename vagy client MIME type ne legyen trusted path/type. A nyers Upload privát marad. Böngészőből elérhető képhez explicit typed `Image` upload és `publish` kell; a platform a byte-okat privát stagingbe teszi, ellenőrzi a támogatott képformátumot és méreteket, majd csak sikeres handler-végrehajtás után publikál.

Ez hasznos tartalmi szabályt ad: adatbázis-metadata hivatkozhat publikált médiaobjectre, de ellenőrizetlen multipart byte nem válik publikussá csak azért, mert a formot a rendszer elfogadta.

A publikus oldal és az editor preview eltérő cache-domain

Publikus cikk route csak akkor használjon public cache-t, ha a response nem session- vagy userfüggő. Editor preview, draft oldal, permission-sensitive view és személyre szabott navigáció ne ossza meg ezt a cache-contractot. A publikus route maradjon szándékosan szűk, és a renderelt cikket módosító mutáció után annak generationjét invalidálja.

12.5. Gyakorlati CMS-rétegmodell

Nagyobb projektnél tartsd külön a felelőségeket:

1. **domain model:** article/page azonosító, status, slug és version;
2. **queryk:** compiler-owned SQL és typed bindok;
3. **editor boundary:** named formok, validáció és autentikált route-ok;
4. **authorization:** role/permission és szükség esetén object-level ellenőrzés;
5. **tranzakciós mutáció:** állapotváltás, optimistic locking és business audit;
6. **presentation:** layoutok, componentek és Markdown renderelés;
7. **media:** staged inspection és explicit publication;
8. **public delivery:** canonical URL, public cache és szűk invalidáció;
9. **operations:** migration, backup/restore, monitoring és release evidence.

Ez a rétegzés növekvő CMS-ben is külön, review-zható határon tartja az authorizationt, renderelést, mutationt és cache-viselkedést.

12.6. CMS review-kérdések

Mielőtt késznek tekintesz egy content feature-t, kérdezd meg:

- Két szerkesztő felülírhatja-e egymást csendben?
- Minden védett állapotváltás külön autorizált-e, a validációtól függetlenül?
- A publish/archive döntések rekonstruálhatók-e business auditból, ha ez követelmény?
- A régi slugok canonical redirectet kapnak-e arbitrary redirect input helyett?
- Publikussá válhat-e ellenőrizetlen upload byte?
- Bekerülhet-e sessionfüggő response public cache-be?
- A mutáció a lehető legkisebb helyes public-cache scope-ot invalidálja-e?
- A content DB és az AppFs/media állapot visszaállítható-e ismert konzisztens pontra?

12.7. Ellenőrzött companion példák

A vizsgált kérdéshez a következő compiler-checked companionokat használd:

- `examples/wiki/main.vrn` – fókuszált wiki-flow;
- `examples/news-site/main.vrn` – többfájlos CMS és editorial workflow;
- `examples/markdown-sql-cache/` – fókuszált typed SQL-szerveroldali Markdown renderelés, opcionális public response cache-sel.

Együtt azt ellenőrzik, hogy az editorial workflow, a típusos SQL, a Markdown renderelés, az autorizáció, a cache és a modulstruktúra együtt is működik az aktuális compilerrel.

12.8. Gyakorlat: definiálj editorial state machine-t

Gyakorlási mód: önálló. Használd az előszóban meghatározott önálló módszert.

Sorold fel egy publikálható objektum állapotait és azokat az actorokat, akik az állapotok között átmenetet végezhetnek. Legyen legalább egy tiltott átmenet is. Ezután dönts el, mely átmenetekhez kell erősebb autorizáció, audit evidence vagy concurrency-védelem.

12.9. Tipikus hiba: a workflow-t szabad szöveggént tároljuk

Az editorial state üzleti állapot. Zárt, típusos halmazként modellezd, az átmeneti szabályokat pedig tartsd expliciten. Ellenkező esetben az érvénytelen kombinációk idővel adatjavítási problémává válnak ahelyett, hogy tervezési döntések lennének.

12.10. Folyamatos mintaprojekt: Secure Notes

Használd az `examples/secure-notes/checkpoints/12-publication/main.vrn` snapshotot. Bővítsd a Secure Notes-ot Draft és Published állapottal. A publikálás maradjon explicit state transition, az ownership szabályok maradjanak meg, a kívülről látható nézet pedig a típusos állapottól függjön, ne ad-hoc string összehasonlítástól.

13. fejezet

Külső szolgáltatások és egress policy

Tanulási út: Teljes út / integráció. Olvasd el, mielőtt outbound hálózati integrációt adsz az alkalmazáshoz.

Biztonsági invariáns: az egress explicit capability

Az alkalmazáskód nem kap tetszőleges hálózati hozzáférést. A kifelé irányuló kommunikáció névvel ellátott, konfigurált capability-határon halad át, így a destination, credential, limit és audit policy felülvizsgálható marad.

Compiler diagnostic: SEC-EFFECT-002

Jelentés: Egy handler hálózati effectet végez anélkül, hogy a megfelelő integration capability szerepelne a `uses` listában.

Javítás: Deklaráld explicit a named integration capabilityt a handler signature-ben; ne helyettesítsd ambient vagy raw hálózati hozzáféréssel.

Az outbound HTTPS külön trust boundary, mert a kapcsolatot maga az alkalmazás kezdeményezi. A Velran ezért explicit capabilityként kezeli, nem ambient hálózati jogosultsággént.

13.1. A jelenlegi Velran-határ

A Velran szándékosan nem ad korlátlan `http.get(userUrl)` jellegű API-t. Ehelyett a source named integrationt deklarál, a handler pedig explicit capabilityt kap:

```
integration Billing {
    egress payments
}

#[action]
fn charge(
    ctx: ActionContext,
    amount: i64
) -> Result<Json, PageError>
    uses Billing
{
    let status = Billing.postJson("/v1/charges", amount)?;
    return Ok(json(status));
}
```

A deklaráció csak trusted *nevet* tesz láthatóvá. Host, IP, port, TLS és hálózati tartomány nem lesz az alkalmazáskód authorityje. A compiler a hívást `net.payments` effectként rögzíti; a handler a megfelelő `uses Billing` capability nélkül nem indíthatja el.

A biztonságos út:

```
Velran business intent
    |
    v
named integration capability
    |
    v
trusted operator egress policy
    |
    v
bounded HTTPS transport
```

13.2. Miért nem reprezentálható arbitrary URL?

A user-controlled fetch target klasszikus SSRF kockázat. Loopback, link-local metadata service vagy belső infrastruktúra nem válhat normál alkalmazási céllá:

```
http://127.0.0.1/...
http://169.254.169.254/...
http://internal-db.example/...
http://[::1]/...
```

A Velran source a deklarált integráción belül relatív pathot választhat, de scheme-et, hostot, IP-t vagy portot nem. A path compiler által ismert rooted relative literal; absolute, protocol-relative és dinamikus destination URL tiltott.

13.3. Named target: a hálózati authority konfigurálása

A trusted egress policy köti a nyelvi target-nevet a konkrét transport authorityhez:

```
[[target]]
name = "payments"
hosts = ["api.payments.example"]
cidrs = ["203.0.113.0/24", "2001:db8:1200::/48"]
ports = [443]
tls_required = true
max_dns_answers = 8
max_sent_bytes = 262144
max_received_bytes = 2097152
connect_timeout_ms = 5000
total_timeout_ms = 15000
```

A typed Velran outbound surface által használt targetet a startup policy egyértelműen a támogatott endpoint formára oldja fel. Outbound effectet használó program hiányzó vagy inkonzisztens egress policyval nem indul el.

13.4. A policy több réteget ellenőriz

Az egress policy több egy hostname allowlistnél.

Host, port és TLS

A destination host és port trusted policyből jön, a TLS identity verification pedig a konfigurált hostname-et használja. Alkalmazási input nem kapcsolhatja ki a TLS-t és nem választhat másik portot.

DNS és CIDR

Minden A és AAAA választ ellenőrizni kell az engedélyezett tartományok ellen. Ha egyetlen answer sérti a policyt, a feloldás fail-closed módon elutasított. Ez DNS rebinding és hibás split-horizon konfiguráció ellen is véd.

Peer verification connect után

TCP kapcsolat után a tényleges peer IP újra ellenőrzött. A DNS-validáció nem maradandó proof.

13.5. IPv4 és IPv6 ugyanazt az authority modellt követi

A policy dual-stack. Az IPv4-mapped IPv6 cím normalizálódik, ezért IPv4 tiltás nem kerülhető meg IPv6 reprezentációval. Loopback, link-local, ULA, multicast és unspecified tartomány implicit trustot nem kap.

13.6. Typed outbound műveletek

Az első Velran call surface szándékosan kicsi:

```
let status = Billing.get("/v1/status")?;  
let status = Billing.postJson("/v1/charges", amount)?;
```

Fontos korlátok:

- page handler outbound GET-et végezhet, outbound POST side effectet nem;
- JSON POST action capability;
- generic outbound JSON sink nem fogad **Secret**, **Sensitive**, credential-purpose vagy upload értéket;
- server-side egress nem ad browser network authorityt – a generált CSP **connect-src** **'none'** marad, amíg nincs külön browser capability;
- a jelenlegi typed call HTTP status code-ot ad vissza, nem automatikusan trusted response body-t.

Ez utóbbi tudatos döntés. Külső response data csak untrusted adatként, typed validation/refinement boundaryn keresztül kerülhetne domain értékké; arbitrary upstream JSON nem válik csendben trusted adattá.

13.7. Méret-, protokoll- és időkorlátok

Az upstream lehet hibás vagy ellenséges. A transport ezért DNS answer-, request byte-, response byte- és időkorlátot alkalmaz. Az operator target limitjein felül a typed Velran status-call surface saját hard upstream-body ceilingt is használ, és elutasítja a nem támogatott transfer/content encodingot.

A teljes request részt vesz a cumulative external-I/O budgetben is. Több külön-külön megengedett network operation így sem állhat össze egyetlen korlátlan kéréssé.

13.8. Szekretek külső szolgáltatáshoz

A külső credential külön secret-store boundary mögött marad:

```
/run/secrets/velran/  
payments-bearer
```

A secret név nem válhat arbitrary filesystem pathszá. Credential nem kerülhet response-ba, normál logba, security eventbe vagy audit payloadba. A typed `Secret<T>` és a trusted Rust secret store ugyanazt a határt erősíti két külön rétegen.

13.9. Retry, idempotencia és exceptional outcome

Side-effecting POST automatikus retry-ja szándékosan nem default. Retry-szenzitív critical operation a Velran idempotency contractját használja, ne ad-hoc retry loopot.

Critical database worknél a transaction outcome explicit:

```
let outcome = transaction db {  
  charge(tx)?;  
};  
  
match outcome {  
  Committed => { return Ok(json(true)); }  
  RolledBack => { fail conflict; }  
  CommitUnknown => { fail conflict; }  
}
```

Idempotens critical transactionnek kötelező az outcome capture és exhaustive kezelés. A `CommitUnknown` nem jelentheti csendben azt, hogy „retry POST”, mert a side effect már commitolhatott.

13.10. A webhook az ellenkező irányú boundary

Outbound integration és inbound webhook két külön probléma. A webhook route deklarált verification policyt, raw-body signature verificationt, timestamp/replay-window ellenőrzést és atomic replay claimet használ még a handler authority előtt. Webhookot ne modellezz egyszerű public JSON POST-ként.

13.11. Szinkron request vagy háttér munka?

Jelenleg nincs first-class background-job subsystem. Rövid, bounded integration maradhat request/response ciklusban. Hosszú vagy megbízhatóan retry-zandó workflowhoz explicit job/outbox design kell; ne ad-hoc thread, subprocess vagy korlátlan request-idő.

13.12. Minimum tesztelés

Bizonyítsd legalább:

- unknown integration vagy hiányzó **uses** capability compile-time reject;
- dynamic és absolute destination path reject;
- page-side POST reject;
- allowed host + IPv4 és IPv6 működik;
- mixed DNS answer egyetlen tiltott címmel fail closed;
- forbidden port, TLS identity mismatch és peer-IP mismatch fail closed;
- request/response byte limit és timeout enforced;
- unsupported upstream transfer/content encoding reject;
- cumulative external-I/O exhaustion resource-limit failure;
- secret nem folyhat generic outbound JSON-ba.

13.13. A fejezet lényege

A külső HTTPS **named, typed, bounded capability**; a destination authority trusted deployment policy marad. Az SSRF-védelem, TLS policy és resource limit így a secure path része.

13.14. Ellenőrzött companion példa

A minimális natív outbound-capability példa az `examples/outbound-native/app.vrn`. Szándékosan kicsi: a lényegi contract az, hogy a forrás csak named integration capabilityt nevez meg, míg a tényleges destination authority a trusted deployment policy tulajdona.

13.15. Gyakorlat: írd egress contractot

Gyakorlási mód: önálló. Használd az előszóban meghatározott önálló módszert.

Egy külső szolgáltatáshoz rögzítsd a destination identity-t, az engedélyezett műveletet, timeoutot, retry policy-t, credential-forrást, várt response shape-et és failure-viselkedést. Ezeket az integráció contractjának tekintsd, ne mellékes kliensbeállításnak.

13.16. Tipikus hiba: tetszőleges kimenő destinationt engedünk

Egy integráció ne alakítsa a felhasználó által vezérelt inputot tetszőleges hálózati célponttá. Az egress maradjon named, bounded és reviewable, hogy egy generikus HTTP helperből ne alakulhasson ki SSRF-jellegű viselkedés.

13.17. Folyamatos mintaprojekt: Secure Notes

Adj egy opcionális named integrációt, például publikációs értesítés küldését. Ne tedd a core persistence útvonal kritikus részévé: egy sikertelen külső hívás hatása legyen expliciten megtervezve, ne hagyja a jegyzetet félkész állapotban.

14. fejezet

Tesztelés, hibamodellek és fejlesztői diagnosztika

Tanulási út: Gyors út: alap production előtt. Alakítsd a compiler-, verifier- és runtime-hibákat ismételhető hibakeresési workflow-vá.

Biztonsági invariáns: az elutasítást is bizonyítsd

Security szabályt nem bizonyít önmagában az, hogy a helyes program sikeresen fut. Legyen negatív teszt is, amely megmutatja, hogy a tiltott változatot a rendszer elutasítja; a váratlan elfogadás regresszió.

A fordítás source- és policy-korlátokat bizonyít, nem üzleti helyességet vagy környezeti egészséget. A tesztelésnek ezért ki kell terjednie az alkalmazásállapokra, valós dependencykre, migrációkra, jogosultsági elvárásokra és a deployolt topológiára.

14.1. Security diagnostic olvasása négy menetben

Amikor a compiler vagy verifier stabil security kóddal elutasít egy programot, ne a check megkerülésével kezd. Négy menetben olvasd a diagnosticot: azonosítsd a kódot, nevezd meg a megsértett invariantot, keresd meg az érintett authority- vagy adathatárt, majd végezd el a legkisebb forrásmódosítást, amely helyreállítja az invariantot. Futtasd újra az ellenőrzést, és security-szempontról fontos szabálynál tartsd meg az elutasított változatot negatív tesztként.

Rögzíts négy dolgot: *kód, határ, az elutasítás oka, és mi bizonyítja a javítás elfogadhatóságát.*

Ezért a Velranban érdemes a tesztelést több egymásra épülő réteggé kezelni:

1. **source check:** a program nyelvi és policy szempontból érvényes-e;
2. **negatív compile teszt:** a veszélyes vagy tiltott program valóban elutasításra kerül-e;
3. **alkalmazás- és adatbázisteszt:** az üzleti művelet a várt állapotot hozza-e létre;
4. **integration teszt:** a valódi DB, Redis, LDAP, TLS, AppFs vagy más dependency együtt helyesen működik-e;
5. **system/smoke teszt:** a telepített rendszer HTTP-n, a tényleges konfigurációval is működik-e.

A cél nem az, hogy minden teszt ugyanazt bizonyítsa, hanem az, hogy minden hibafajta a lehető legolcsóbb rétegben bukjon el.

14.2. Az első kapu: `velran-cli check`

Fejlesztés közben a leggyorsabb ellenőrzés:

```
velran-cli check main.vrn
```

Siker esetén a CLI összefoglalja a felismert programot, például a modellek, query-k, page-ek, actionök és route-ok számát. A check nem indít HTTP servert és nem módosít adatbázist: a source contractot ellenőrzi.

A fordító többek között itt tudja megfogni:

- a típusegyeztési hibákat;
- a query eredményalak és a `model` eltérését;
- a hibás route vagy form deklarációt;
- a tiltott HTML/script konstrukciókat;
- a veszélyes SQL összeállítást vagy bind hibát;
- az auth/cache/policy egymással össze nem férő kombinációit;
- a capability-határok bizonyos megsértéseit.

Ez az egyik legfontosabb szemléleti különbség egy általános scripting megoldáshoz képest: a hibák jelentős részét nem HTTP kérés közben akarjuk felfedezni.

14.3. A negatív teszt ugyanolyan értékes, mint a pozitív

Biztonsági invariánsnál nem elég azt tesztelni, hogy a helyes program lefordul. Azt is bizonyítani kell, hogy a tiltott program *nem* fordul le.

A repository release-ellenőrzése ezért ilyen fixture-öket is futtat:

```
# Ennek sikeresen kell fordulnia:
velran-cli check examples/auth/app.vrn

# Ennek viszont hibával kell visszatérnie:
velran-cli check \
  tests/fixtures/security/sql-injection-rejected.vrn
```

Tipikus negatív teszt:

- SQL injectionre alkalmas dinamikus SQL;
- raw/script jellegű XSS-kísérlet;
- optional érték ellenőrzés nélküli használata;
- hibás form/schema validáció;
- userfüggő route public cache-sel;
- public route objektumszintű authorization guarddal;
- tiltott upload path vagy JSON/form body-mode keverés.

Egy biztonsági szabály akkor erős, ha a projektben van rá **pozitív és negatív bizonyíték** is.

A repository szerkezetében érdemes ezt a szerepkülönbséget láthatóvá tenni. Az olvasásra és futtatásra szánt alkalmazások az `examples/` alatt maradnak; a szándékosan elutasítandó compiler/security inputok a `tests/fixtures/` alatt vannak; a verification tooling által használt listák pedig a `tests/manifests/` alatt. Egy pozitív példa attól még részt vehet a `verify.sh`-ban, de alkalmazáskódként is érdemes legyen elolvasni. Egy rejection fixture-nek viszont nem

kell jó alkalmazásnak látszania: az a feladata, hogy egyetlen invariánst izoláljon, és a kívánt okból bukjon el.

14.4. A hibák négy külön világa

Hibakereséskor először azt dönts el, melyik rétegben történt a hiba. Ugyanaz a felhasználói tünet teljesen más javítást kívánhat.

Compile-time hiba A source contract hibás. A `velran-cli check` vagy `build` áll meg. Ezt kódban kell javítani.

Startup/config hiba A program érvényes, de a szerver konfigurációja, secretje, TLS kulcsa vagy egy hard policy hibás. Itt a `-check-config`, a server log és az effective config a fő eszköz.

Request/policy hiba A szerver fut, de egy kérés input-, auth-, authorization-, rate-limit- vagy content-negotiation szabály miatt elutasítódik. Itt az HTTP státusz, request ID, access és audit log a bizonyíték.

Dependency/runtime hiba DB, Redis, LDAP, filesystem, hálózat vagy resource limit okozza a hibát. Itt readiness, metrics, server log és valódi integration teszt kell.

A rossz diagnosztikai reflex az, hogy minden 500/503 körüli tünetre a source kódot kezdjük átírni. Először a réteget azonosítsd.

14.5. Konfigurációt külön tesztelj

A source check nem helyettesíti a server konfiguráció ellenőrzését:

```
velran-server \  
  --config /usr/local/etc/velran/server.toml \  
  --check-config
```

Ha deploy előtt a ténylegesen összeálló értékeket is látni akarsz:

```
velran-server \  
  --config /usr/local/etc/velran/server.toml \  
  --print-effective-config
```

A `-check-config` nem dependency integration teszt. Egy szintaktikailag helyes DB URL mellett a DB még lehet elérhetetlen; ezért van külön readiness és deployment smoke teszt.

14.6. Migráció: status, verify, apply, verify

Adatbázis-változásnál a tesztelési folyamat része maga a migrációs állapot is:

```
velran-cli migrate status \  
  --dir migrations \  
  --db-url-file /run/secrets/velran/migration-db-url  
  
velran-cli migrate verify \  
  --dir migrations \  
  --db-url-file /run/secrets/velran/migration-db-url
```

Deploymentben a jó sorrend:

```
source/config check
-> migration verify
-> migration apply
-> migration verify
-> application rollout
-> readiness
-> smoke test
```

A már alkalmazott migration fájlt ne módosítsd azért, hogy a teszt újra zöld legyen. Új változást új migrationben vezess be.

14.7. Üzleti teszt: állapotot bizonyíts, ne implementációt

Egy CRUD vagy CMS action tesztjének ne az legyen a lényege, hogy „meghívódott-e ez a belső függvény”. A kívülről megfigyelhető állapotot ellenőrizd.

Például cikkpublikálásnál:

1. hozz létre draft cikket;
2. autentikálj Publisher szerepkörrel;
3. küldd el a publish actiont;
4. ellenőrizd a redirectet;
5. olvasd vissza a cikket és ellenőrizd a `Published` állapotot;
6. ellenőrizd a business audit rekordot;
7. ellenőrizd, hogy a publikus oldal már az új állapotot adja;
8. cache-elt route esetén ellenőrizd az invalidációt is.

Ez a teszt akkor is értékes marad, ha később belül átszervezed a query-ket vagy a modulstruktúrát.

14.8. Adatbázisteszt: SQLite nem bizonyít mindent

A gyors fejlesztői tesztben a külön ideiglenes SQLite adatbázis kényelmes. De ha production PostgreSQL vagy MariaDB, legalább a release/integration körben a valódi backendet is teszteld.

Különösen érzékeny területek:

- tranzakciós viselkedés;
- constraint és unique conflict;
- konkurens módosítás és optimistic locking;
- index/lekérdezési terv;
- migration DDL;
- timeout és connection availability.

A backend-független typed query contract sok hibát kizár, de a backend működését nem teszi azonossá.

14.9. Auth és authorization tesztmátrix

Egy védett actionnél legalább a következő eseteket érdemes explicit tesztelni:

Eset	Elvárt eredmény
------	-----------------

Nincs session	login/auth elutasítás
Érvényes user, nincs megfelelő role	authorization deny
Megfelelő role	sikeres művelet
MFA-köteles route, nincs MFA	MFA elutasítás
Lejárt vagy visszavont session	új autentikáció szükséges
Hibás CSRF token state change-nél	kérés elutasítva
Objektum nem a useré	object authorization deny
Admin/engedélyezett actor	sikeres művelet

LDAP/LDAPS vagy Redis használatakor legyen valódi dependency integration teszt is. Egy mock nem bizonyít TLS hostname validationt, hálózati timeoutot vagy Redis fail-closed viselkedést.

14.10. Fájlf, média, API és cache tesztek

Ezeknél a happy path különösen kevés.

Upload és média

Teszteld legalább:

- a megengedett kis fájl;
- byte-limit túllépést;
- kép pixel-limit túllépést;
- hibás/támadó fájlnevet;
- path traversal/symlink escape elutasítást;
- félbeszakadt upload cleanupot.

JSON/CORS

Teszteld a helyes JSON mellett a hibás content type-ot, duplikált kulcsot, ismeretlen mezőt, **Accept** negotiationt, preflightot és credentialed state change esetén a CSRF/CORS együttműködését.

Cache és rate limit

Teszteld, hogy userfüggő oldal nem válik public cache-é, módosítás után a cache invalidálódik, több instance esetén a shared state tényleg közös, és Redis kiesésnél a security szempontból választott fail-closed viselkedés érvényesül.

14.11. HTTP hibakód nem egyenlő programhibával

A kliensnek adott státusz a szerződés része. Tipikus esetek:

Helyzet	HTTP
Hibás request-contract vagy malformed input	400
Nincs érvényes autentikáció JSON/API route-on	401
Az actor/policy nem engedélyezi a műveletet	403
Nem található erőforrás	404
Rossz HTTP method	405
Nem elfogadható representation	406

Konkurens módosítás / Changed nulla sor	409
Túl nagy request/upload	413
Rossz vagy hiányzó body content type	415
Host/trust boundary mismatch	421
Named form mezőhiba, újrarendeléssel	422
HTTPS szükséges	426
Rate limit	429
Túl nagy request header	431
Belső, nem klasszifikált serverhiba	500
Dependency vagy request resource átmenetileg nem elérhető	503

A kliensnek soha ne küldj SQL-t, secretet, fizikai filesystem pathot, session tokenet vagy stack trace-et. A részletes diagnosztika a serveroldali logban marad, request ID-val korrelálva.

A státusz és a válasz formátuma külön szerződés

Ugyanaz az üzleti helyzet eltérő reprezentációt kaphat HTML és JSON felületen. Ez nem inkonzisztencia: a státusz a hibakategóriát, a reprezentáció pedig a klienssel kötött felületi szerződést fejezi ki.

Helyzet	HTML route	JSON route
Nincs session	303 See Other a beépített login oldalra	401 + JSON error envelope
Authorization tiltás	403	403 + JSON error envelope
Optimistic-lock konfliktus	409 konfliktusoldal, no-store	409 + JSON error=conflict
Named form validation	422, a form újrarendelve mezőhibákkal	nem JSON body-mód; a body-contractok kölcsönösen kizártak
Nem található record	404	404 + JSON error envelope

A sikeres mutató HTML action tipikusan 303 **See Other** választ használ a Post/Redirect/Get mintához. A 303 redirect tehát nem hiba, hanem a böngészős vezérlési folyamat része.

14.12. Exceptional state-ek exhaustív tesztelése

A Velran zárt sum type-jai security-releváns exceptional state-et compile-time tesztelhetővé tesznek. A **match** nem ad catch-all wildcardot, ezért új variant hozzáadása minden érintett match site review-ját kikényszeríti.

Critical transactionnál legyen teszt mindhárom outcome-ra:

- **Committed**: success path és stabil replay state;
- **RolledBack**: a művelet nem történt meg, a public outcome biztonságos;
- **CommitUnknown**: nincs automatikus side-effect retry, az unknown állapot explicit kezelt.

Negatív compiler teszt bizonyítsa azt is, hogy idempotens critical transaction outcome capture vagy exhaustive match nélkül reject.

14.13. Request ID-val hibakeresni

Ha egy HTTP kérés hibázik, a legjobb első diagnosztikai adat a szerver által generált request ID. A kliens válaszából vagy az access logból indulva:

```
grep 'velran-...' /var/log/velran/access.log
grep 'velran-...' /var/log/velran/server.log
grep 'velran-...' /var/log/velran/audit.log
```

A három log más kérdésre válaszol:

- access: mi érkezett és milyen státusszal végződött;
- server: milyen technikai/runtime esemény történt;
- audit: security vagy felhasználói policy döntés történt-e.

Ezt a diagnosztikai láncot érdemes automatizált teszt failure outputban is megőrizni.

14.14. Deployment smoke teszt

A CI zöld állapota nem bizonyítja, hogy a frissen telepített service ténylegesen elindult. Rollout után legalább:

```
curl --fail --silent \
  https://example.test/health/live

curl --fail --silent \
  https://example.test/health/ready

curl --fail --silent \
  https://example.test/
```

A smoke teszt legyen kicsi és nem destruktív. Ha state-changing végpontot is ellenőrzöl, használj erre elkülönített tesztadatot és idempotens cleanupot; ne a production tartalmon kísérletezz.

14.15. Mit jelent a repository verify.sh?

A Velran saját forrásfájában a teljes release-evidence belépési pontja:

```
./verify.sh
```

Egyesíti a source checket a workspace Rust tesztjeivel, az `examples/` olvasásra szánt pozitív programjaival, a `tests/fixtures/` rejection/security fixture-jeivel, a `tests/manifests/` manifest-vezérelt corpus ellenőrzéseivel, valamint migrációs/auth/storage/runtime checkekkel és release artifact invariánsokkal.

Ha magát a Velran implementációját módosítod, ez a release minimum. Ha csak egy saját `.vrn` alkalmazást fejlesztesz, a napi ciklus kisebb lehet, de a release folyamatodnak akkor is tartalmaznia kell source checket, migration/config ellenőrzést, a releváns integration tesztet és deploy utáni smoke tesztet.

14.16. Ajánlott fejlesztői ciklus

Egy hétköznapi feature fejlesztésére jó alap:

```
1. domain/model valtozas megtervezese
2. migration megirasa, ha kell
3. velran-cli check main.vrn
4. gyors alkalmazasteszt
5. negativ/security esetek
6. migration verify
7. relevans integration teszt
8. velran-server --config server.toml --check-config
9. deploy stagingre
10. readiness + HTTP smoke
11. production rollout
12. readiness + smoke + metrics/log ellenorzes
```

A sorrend lényege: a gyors és determinisztikus hibák bukjának el korán, a drága környezeti tesztek csak azután fussanak.

14.17. Hibakeresési döntési sorrend

Amikor „nem működik”, haladj következetesen:

1. Lefordul? Futtasd a `velran-cli check` parancsot.
2. Érvényes a config? Futtasd a `-check-config` ellenőrzést.
3. Elindult a service? Nézd a service és server logot.
4. Él a process? Nézd a liveness endpointot.
5. Elérhetőek a dependency-k? Nézd a readiness endpointot.
6. Eljut a kérés a route-ig? Nézd az access logot és request ID-t.
7. Policy dobja el? Nézd az audit logot és a trust-boundary státuszt (például forwarding 400, 401/403/421/426/429).
8. Runtime/dependency hibázik? Nézd a server logot és metrics-et.
9. Üzleti eredmény rossz? Reprodukálj izolált tesztadattal és ellenőrizd a tartós állapotot.

Ez a sorrend sokkal gyorsabb, mint egyszerre kódot, proxy-konfigurációt és adatbázist módosítani.

14.18. Minimum release checklist alkalmazáshoz

- `velran-cli check` zöld;
- migration history verify zöld;
- server config check zöld;
- a kritikus business flow-k pozitív és negatív tesztje zöld;
- auth/authorization/security regressziós esetek zöldek;
- productionnal azonos DB/Redis/TLS irányú integration teszt zöld;
- backup/restore readiness ismert;
- staging readiness és smoke zöld;
- rollout után readiness, smoke, log és metrics ellenőrzött.

A tesztelés célja nem a százszázalékos sorlefedettség, hanem az, hogy a rendszer fontos szerződéseire legyen ismételterő bizonyíték: **mit engedünk, mit tiltunk, milyen állapotot hozunk létre, és hogyan viselkedünk dependency-hiba esetén.**

14.19. Gyakorlat: vezess le negatív tesztek a határokból

Gyakorlási mód: önálló. Használd az előszóban meghatározott önálló módszert.

Minden megbízható átmenethez írd legalább egy tesztet, ahol az átmenetet el kell utasítani: hibás input, hiányzó autorizáció, stale verzió, elfogyott budget vagy elérhetetlen dependency. A negatív teszt akkor értékes, ha magát a határt bizonyítja, nem csak egy hibaüzenetet.

14.20. Tipikus hiba: csak sikeres handlereket tesztelünk

Egy security-first framework értékének nagy része abból jön, amit nem hajlandó végrehajtani. A compile-time rejection, verification failure és runtime failure tesztek kezelését külön, hogy egy regresszió ne toljon csendben későbbi és gyengébb fázisba egy védelmet.

14.21. Folyamatos mintaprojekt: Secure Notes

Készíts kompakt tesztmátrixot create, view, edit, publish és delete műveletekre. Legyen benne másik felhasználó azonosítója, stale edit verzió, érvénytelen input bound és integration failure. A projekt így már nemcsak a sikert, hanem az elvárt visszautasítást is bizonyítja.

15. fejezet

Security hardening: a jelenlegi secure-by-construction modell

Tanulási út: Gyors út: kötelező production előtt. Fogd össze az egész alkalmazásban kötelező security-invariánsokat.

A Velran security nem opcionális middleware: az authority a nyelv, compiler, runtime és server között oszlik meg. Amit lehet, compiler-invariánsként bizonyíts; különben legyen secure platform default vagy stabil diagnostic-kal fail closed.

Első olvasás. Fókuszálj az authority/access control, input/browser boundary, resource safety és production security gate részekre. A többi szakasz hardening referencia.

15.1. Mit dönt el a secure modell – és mit nem?

A Velran reprezentálhatatlanná tehet raw SQL-t, arbitrary redirect URL-t, ambient DB/network authorityt, unbounded list queryt, hamis redactiont és több más veszélyes állapotot. Azt is bizonyíthatja, hogy egy mutation authorization proofot használt. Azt viszont nem döntheti el helyetted, hogy az üzleti szabály szerint „az Editor publikálhatja-e ezt a konkrét cikket”. Ez továbbra is az alkalmazás permission/object/tenant policyjének feladata.

Hasznos review-szabály: *a mechanizmus legyen compiler/runtime-owned, az üzleti intent maradjon explicit alkalmazáskód.*

15.2. Authority és access control

Minden route explicit access policyt kap; nincs implicit public endpoint. Az authorization réteges:

1. trusted authentication létrehozza a principalt;
2. route policy eldönti, beléphet-e a handlerbe;
3. object authorization proofot ad a konkrét betöltött rekordhoz;
4. mutation authorization bizonyítja, hogy UPDATE/DELETE kulcsa autorizált object authorityból származik;
5. named permission és MFA a handler contract része lehet;
6. tenant-scoped adat aktív membership proofot igényel;
7. effect/capability metadata láthatóvá teszi a DB és outbound network authorityt.

Validation proof soha nem authorization proof. Egy valid `ArticleId` nem bizonyítja, hogy a principal módosíthatja az adott cikket.

15.3. Critical operation, idempotency és transaction outcome

A critical contract egyetlen helyen fogja össze a security kötelezettségeket:

```
critical Payment {  
    permission BillingWrite  
    mfa  
    transaction  
    idempotency  
    audit PaymentCharged  
}
```

Az idempotency a replay key-t request fingerprinthez és principal scope-hoz köti. Ugyanaz a key más payloadhoz conflict, nem második side effect.

A DB commit visszaigazolása is explicit. Captured transaction zárt `Committed` | `RolledBack` | `CommitUnknown` sum type-ot ad, a `match` exhaustive és nincs wildcard fallthrough. Idempotens critical transactionnek kötelező az outcome capture és kezelés; a `CommitUnknown` nem tűnhet el generic retryable DB hibában.

15.4. Input, injection és browser boundaryk

A fő injection felületek strukturális kontroll alatt vannak:

- SQL compiler-owned statement és typed bind; string-interpolált raw SQL reject;
- HTML/template output strukturált és sensitivity-aware;
- redirect typed local route call, nem string URL;
- response header typed név és bounded validált érték;
- filename, media type és Content-Disposition typed;
- outbound call named integration és relative literal path, nem arbitrary URL;
- generált CSP least-privilege defaultot használ, többek között `connect-src 'none'`;
- compressed inbound HTTP body reject, amíg nincs külön bounded decompression surface.

15.5. Trust, domain típusok és disclosure

A request-boundary érték untrusted állapotból indul. Domain validation refined nominal type-ot adhat, de sensitivityt nem mos le és authorizationt nem teremt.

A `Secret<T>` és `Sensitive<T>` metadata hívásokon keresztül is követett. Teljes database/domain model nem generic módon serializálódik; public output explicit projection. Így egy új model field nem jelenhet meg észrevétlenül régi API-ban.

Loghoz/monitoringhoz a `Redacted<T>` nem hamisítható type annotationnel. Csak a trusted `redact(...)` builtin hozza létre, amely eldobja az eredeti reprezentációt és biztonságos redacted értéket ad.

15.6. Authentication és session hardening

A browser session platform-owned. Login authority rotationt végez, logout invalidál, auth-generation változás pedig security-releváns account change után régi sessiont érvénytelenít. Cookie flaget az alkalmazás nem gyengíthet.

Online auth abuse egyszerre három dimenzióban limitált:

- source + principal;
- principal több source-on keresztül;
- source több principalon keresztül.

MFA/recovery külön szigorúbb stage limitet kap. Limiter-store hiba fail closed, a kliens credential hibája pedig szándékosan generic az account enumeration csökkentésére.

15.7. Credential, token és kriptográfia

Purpose-specific típusok zárják ki a password, token hash, session token, reset token, CSRF token és crypto key véletlen keresztfelhasználását.

Jelszóhoz platform-owned Argon2id policy tartozik. Bearer token purpose-specific hashként perzisztálódik, expiry-aware verificationnel; reset grant, session revoke és session rotation atomic lifecycle contractot használ.

Crypto keynek purpose és lifecycle állapota van. User-data encryption high-level primitive fix AES-256-GCM-et, platform-generált nonce-ot és versioned authenticated envelope-ot használ:

Active	-> encrypt + decrypt
Retiring	-> decrypt only
Retired	-> neither

Az alkalmazás nem választhat gyengébb cipher-t és nem adhat saját nonce-ot.

15.8. Verified webhook és safe upload

A webhook nem autentikáció nélküli public POST. Külön webhook authority van: raw-body signature verification, timestamp/replay-window check és atomic replay claim történik a handler előtt.

A file upload állapotgép:

multipart bytes
-> private staged file
-> byte-authoritative inspection
-> verified image
-> handler success
-> atomic publish

Client filename és MIME metadata, nem storage vagy file-type authority. A public media endpoint csak verified image publish route által deklarált könyvtárból szolgál.

15.9. Resource safety mint security invariant

A Velran egyedi műveletet és teljes requestet is korlátoz:

- request/parser hard limit;
- route resource profile platform ceiling alatt;
- instruction/allocation budget és hard request deadline;
- DB list query statikus row bound + runtime row cap;
- bounded request collection;
- upload byte/pixel limit;
- bounded outbound request/response body;
- cumulative external-I/O accounting.

Unrestricted generic collection growth szándékosan nincs kitéve addig, amíg ugyanez a cardinality guarantee nem tartható meg.

15.10. A production konfiguráció security contract

Az alkalmazás strict production követelményt deklarálhat:

```
production {
  https required;
  debug disabled;
  hsts required;
  database tls required;
}
```

Startup fail closed, ha a deployment nem tudja teljesíteni. Production policy tiltja az insecure development cookie-t, unsafe source reloadot, missing-Origin engedményt és insecure remote DB transportot. Reload nem változtathatja meg csendben a deployment security contractot; ehhez restart és operator review kell.

15.11. Supply-chain state a release része

Három artifact összetartozik:

```
Cargo.lock
SUPPLY-CHAIN-CAPABILITIES.txt
VELRAN-SUPPLY-CHAIN.lock
```

A direct external dependency edge explicit security capabilityt kap, például network, filesystem, crypto vagy parser. A jelenlegi provenance policy csak checksummed crates.io package-et enged. A supply-chain envelope a Cargo lockot és capability policyt együtt sealeli, release előtt pedig kötelező a `cargo metadata -locked` konzisztencia.

15.12. Security event, redaction és alert

Critical operation typed audit eseményt ad. A platform automatikusan strukturált security eventet generál auth abuse, MFA failure, access-policy denial, CSRF/origin/CORS rejection, webhook verification failure, idempotency conflict és resource exhaustion esetén.

Az event schema nem fogad arbitrary request payloadot. Principal/source emitted eventben redacted; bounded in-memory correlation használhat raw kulcsot úgy, hogy logba ne kerüljön. Platform-owned burst threshold és cooldown alertet ad anélkül, hogy az alerting maga log-flood DoS-sá válna.

15.13. Production security gate

Production átadás előtt legalább:

1. valódi toolchain gate: format, locked metadata, check, test és `./verify.sh`;
2. production policy és startup compatibility zöld;
3. HTTPS, HSTS, secure cookie és trusted proxy topology helyes;
4. runtime DB credential least privilege és remote DB TLS enforced;
5. auth/session/rate-limit shared state megfelel a deployment topológiának;
6. tenant/object/mutation authorization negatív tesztek zöldek;
7. critical/idempotent operation explicit kezeli a `CommitUnknown` állapotot;
8. webhook verification/replay és upload publish-state tesztek zöldek;
9. outbound integration csak review-zott named egress policyből oldódik fel;
10. resource/deadline limitek konfiguráltak és megfigyeltek;
11. security-event redaction és burst alert ellenőrzött;
12. supply-chain és release-manifest állapot aktuális és verified.

15.14. Öt review-kérdés minden új feature-höz

1. **Mi jön kívülről?** Melyik mező, fájl, identifier vagy upstream data untrusted?
2. **Hol válik typed/validated adattá?** Melyik boundary hozza létre a proofot?
3. **Ki az authority?** Session, DB record, tenant membership, trusted config vagy named capability?
4. **Mi történik bizonytalan hibánál?** Fail closed, rollback, cleanup vagy explicit unknown state?
5. **Mi korlátozza az erőforrást?** Byte, row, item, idő, rate, CPU vagy cumulative I/O?

Ha bármelyik válasz homályos, a feature security modellje nincs kész.

15.15. Ellenőrzött companion példák

A security hardening végrehajtható fixture-ök mellett könnyebben review-zható. A canonical companion példák itt az `examples/resource-profile/app.vrn`, `examples/rate-limit/app.vrn` és `examples/typed-security-events/main.vrn`.

15.16. Gyakorlat: rendelj kontrollokat threat pathokhoz

Gyakorlási mód: önálló. Használd az előszóban meghatározott önálló módszert.

Válassz három konkrét fenyegetést, és kövesd végig, mely egymástól független kontrollok állítják meg vagy korlátozzák őket. Rétegzett választ keress: typed input + autorizáció + tranzakciós invariáns erősebb, mint egyetlen validáció, amelytől mindent várunk.

15.17. Tipikus hiba: a hardeninget végső deployment fázisnak tekintjük

Az alkalmazásstruktúrától függő hardeninget production előtt kell megtervezni. Route policy, capability boundary, object authorization, bounded input és named egress architektúra, nem release utáni konfigurációs csiszolás.

15.18. Folyamatos mintaprojekt: Secure Notes

Nézd át a Secure Notes-ot támadó szemmel: azonosítók, form mezők, publikációs átmenetek, media referenciák, JSON endpointok és kimenő integráció. Dokumentáld, mely határ utasítja el az egyes érvénytelen útvonalakat, és mely eseménynek kell operátori szinten láthatónak lennie.

IV. rész

Production és üzemeltetés

16. fejezet

Cache, Redis, rate limiting és többpéldányos működés

Tanulási út: Teljes út / skálázás. Akkor olvasd, amikor mérés alapján valóban cache-re, Redisre vagy horizontális skálázásra van szükség.

Első olvasás. Tanuld meg, mi cache-elhető, hogyan invalidál biztonságosan a generation modell, és hogyan viselkedik fail closed módon a shared session/rate limit. A backend tuning és topológiai részletek referenciaanyagok.

Több instance esetén a session state, rate limit és public cache ownershipja legyen explicit. A Velran a Redis runtime-owned infrastruktúraként kezeli: az alkalmazáskód policyt deklarál, általános key/value API-t nem kap. A business source of truth továbbra is DB/AppFs.

16.1. Miért nem általános Redis kliens?

A V1 nyelvben nincs olyan művelet, amellyel a program tetszőleges Redis kulcsot írhatna vagy olvashatna. A Redis compiler/runtime-owned infrastruktúra:

Felhasználás	Ki birtokolja a kulcsot és a szabályokat?
session	runtime/auth réteg
TOTP replay	runtime/auth réteg
login rate limit	runtime/auth réteg
route rate limit	runtime + trusted policy config
public page cache	compiler/runtime
üzleti adat	nem Redisben; application DB-ben/AppFs-ben

Ezért egy wiki cikkét, CMS workflow állapotát vagy számlázási rekordját ne Redisbe tedd. Az ilyen adat üzleti source of truth, ezért a typed SQL-lel elért application DB-be tartozik. Redis elvesztése után a rendszernek újraépíthető állapotból kell indulnia: a sessionök megszűnhetnek, a cache újraépülhet, a rate-limit ablakok újraindulhatnak.

16.2. Production Redis konfiguráció

A Redis URL secret, ezért fájlból érkezzon:

```
[redis]
```

```
url_file = "/run/secrets/velran/redis-url"
allow_insecure = false
```

A secret file például TLS-es URL-t tartalmazhat:

```
rediss://velran:SECRET@redis.internal.example/0
```

Productionben az `allow_insecure = false` legyen az alap. A Redis hálózati elérhetősége önmagában nem jogosultsági modell: korlátozd hálózati ACL-lel/firewallal is, és a credentialt csak a Velran service user olvashassa.

A runtime külön namespace-eket használ a különböző célokra. A public cache például `velran-cache` namespace-ben él; az alkalmazás nem választ tetszőleges key prefixet.

16.3. Shared session több szerverpéldánnyal

Az in-memory session store fejlesztési kényelem, nem multi-instance production modell:

```

      +--> Velran A ---+
Browser -> proxy |           |--> Redis
      +--> Velran B ---+
```

Ha a session csak A-ban él, egy B-re routolt request elveszíti az authentication állapotot. Shared session backend kell; a sticky session nem failover- vagy rollout-stratégia.

Redis production környezetben többek között az alábbi auth state-et osztja meg:

- sessionök;
- TOTP replay védelem;
- login rate-limit állapot.

Így nincs szükség arra, hogy ugyanaz a böngésző mindig ugyanarra a példányra kerüljön.

A local auth fontos kivétele

A session shared lehet, de a V1 local-auth identity store SQLite és single-node identity megoldás. Ezért több application instance esetén a jelenlegi shared identity backend az LDAP/LDAPS. Ne téveszd össze a két réteget:

Redis session megmondja, hogy egy már autentikált request milyen trusted principalhoz és auth state-hez tartozik;

identity backend a login során ellenőrzi a credentialt és az account/role állapotot.

Több instance-nál shared Redis session mellett is közös identity forrás kell; V1-ben erre az LDAP a production út.

16.4. Public page cache

A public cache célja, hogy nyilvános, felhasználótól független GET oldalnál ne fusson minden requestre ugyanaz a query és HTML render.

Deklaráció:

```
route frontPage GET "/"
```

```
public cache public ttl 60
=> frontPage;
```

A fejlesztő csak a cache-elési szándékot és a route TTL-jét adja meg. A kulcsot a runtime készíti canonical, hash-elt formában többek között a route nevéből, route generationból, pathból, rendezett typed query inputból és reprezentációból.

Az alkalmazás tehát nem írhat ilyet:

```
cache.set("homepage:" + userInput, rendered)
```

Ilyen általános cache API nincs. Ez megakadályozza a kulcsnév-konvenciók szétcsúszását, a véletlen PII-keyeket és több invalidációs stratégia keverését.

16.5. Mit szabad public cache-be tenni?

A `cache public` név szó szerinti. Csak olyan GET route legyen cache-elt, amelynek válasza nem függ a felhasználó sessionjétől vagy request-specifikus security state-től.

A compiler tiltja többek között az olyan függéseket, mint:

```
csrfToken
authPrincipal
authMfaVerified
```

Ez azért kritikus, mert sikeres cache-elt válasz HTTP szinten is public cache-elhető:

```
Cache-Control: public, max-age=<ttl>
```

Vagyis ugyanaz a response reverse proxy vagy CDN cache-be is kerülhet. Account oldal, admin felület, személyre szabott navigáció vagy flash üzenetet fogyasztó oldal ezért nem public cache.

16.6. Operator által szabott cache-határok

A route nem állíthat tetszőlegesen nagy TTL-t. A trusted server config kemény plafont ad:

```
[cache]
max_ttl_secs = 3600
max_entries = 10000
max_bytes = 67108864
allow_memory = false
```

A `max_ttl_secs` a route által kérhető TTL maximuma. A memória backend entry- és byte-limitjei bounded development fallbackot adnak, nem production skálázási stratégiát.

Ha az alkalmazás public cache-t használ, de Redis nincs konfigurálva, a server production default mellett nem indul el. Fejlesztéshez explicit engedélyezhető a memory cache, de ezt ne vidd át észrevétlenül productionbe.

16.7. Cache invalidation: generation, nem kulcskeresés

State-changing route megjelölheti, mely cache-elt route-ok válnak érvénytelenné:

```
route publish POST "/publish"  
  public invalidate cache frontPage article  
  => publish;
```

Az invalidation nem Redis `SCAN` és nem kulcslista törlése. A runtime route-generationt növel. Az ezután érkező request már új generationból készíti a cache key-t, ezért a korábbi entry automatikusan elérhetetlenné válik az új forgalom számára.

Ennek két fontos következménye van:

- az invalidation $O(1)$ jellegű, nem függ a korábban létrejött cache key-ek számától;
- shared Redis esetén minden instance ugyanazt az új generationt látja, ezért az invalidáció cluster-safe.

A régi entry fizikailag egy ideig még Redisben maradhat, de logikailag már nem része az aktuális cache namespace-nek és TTL alapján kifut.

16.8. Stampede védelem és annak határa

Cache miss után több azonos request egyszerre érkezhet. A Velran ugyanazon instance-en ugyanarra a cache keyre single-flight jelleggel csak egy requestet enged újraépíteni; a többiek várnak, majd újra cache-t olvasnak.

Ez fontos, de a határát is ismerni kell: V1-ben nincs distributed single-flight lease. Két külön Velran instance ugyanannak a lejárt kulcsnak az újraépítését egyszerre is elkezdheti. Ezért a cache-elhető page queryje önmagában is legyen ésszerű költségű és jól indexelt. A cache nem mentség drága, kontrollálatlan adatbázis-műveletre.

16.9. Redis hiba cache-elt request közben

A public cache backend hibája nem változhat csendben más konzisztenciamodellre. Cache backend elérhetetlenségénél a Velran `cache_unavailable` hibával szolgáltatáshibát adhat a cache-elt útvonalon ahelyett, hogy implicit, korlátlan fallbackként minden requestet az adatbázisra engedne.

Ez productionben hasznos fail-closed viselkedés: Redis kiesés ne okozzon észrevétlen DB thundering herdőt. A readiness probe DB/Redis függőségi hibánál 503-at adhat, ezért a load balancer ki tudja venni a példányt a forgalomból.

16.10. Route rate limiting

A route nem numerikus limitet ír a forráskódba, hanem trusted policy nevet kér:

```
route api GET "/api/status"  
  public rate publicApi  
  => api;
```

A konkrét számok operátori configban élnek:

```
[policy.publicApi]  
limit = 120  
window_secs = 60
```

```
scope = "ip_route"
```

Ez tudatos elválasztás. Az alkalmazás megmondhatja, hogy az endpointot védeni kell a `publicApi` policy-val, de nem tudja önkényesen felülrni a production erőforrás- és abuse policy-t.

16.11. Rate-limit scope-ok

A V1 scope-ok:

Scope	Bucket alanya
<code>ip</code>	effective client IP
<code>route</code>	minden kliens közös route bucketje
<code>ip_route</code>	effective client IP + route
<code>user</code>	autentikált canonical principal
<code>user_route</code>	canonical principal + route

A `user` és `user_route` public route-on startup error, mert nincs trusted autentikált principal, amelyből a bucket képződhetne.

IP-scope esetén a Velran nem vakon bíz az `X-Forwarded-For` fejlécben. Előbb a trusted proxy policy alapján állapítja meg az effective client IP-t. Emiatt Apache/Nginx mögött a proxy CIDR helyes konfigurálása rate-limit security kérdés is.

16.12. Fixed-window modell több instance-nál

A route limiter fixed-window bucketet használ. Redisben minden instance ugyanazt a bucketet növeli atomikusan, ezért a limit nem példányonként értendő.

Például két szerver mellett:

```
limit = 120 / 60 sec

nem: 120 request A + 120 request B
hanem: 120 request a shared bucketben
```

A subject stabil hashként kerül a Redis kulcsba, így a raw username vagy IP nem jelenik meg közvetlenül keyként.

Limit túllépésekor a válasz:

```
HTTP/1.1 429 Too Many Requests
Retry-After: 60
```

Redis limiter hiba esetén a policy fail-closed:

```
503 rate_limiter_unavailable
```

Ez biztonságosabb annál, mintha Redis hiba idején automatikusan korlátlanra nyílna az endpoint.

16.13. Memory backend csak fejlesztési kivétel

A trusted config production baseline-ja:

```
[rate_limit]
policies_file = "/usr/local/etc/velran/rate-limits.toml"
allow_memory = false

[cache]
allow_memory = false
```

A memory rate limiter és memory cache explicit development escape hatch. Több process vagy több host esetén minden példány külön memóriát látna, tehát sem a limit, sem a cache-invalidation globális szemantikája nem lenne ugyanaz, mint Redis mellett.

Ha lokálisan memory backenddel tesztelsz, deployment előtt mindig fuss át a production configon:

```
velran-server --config /usr/local/etc/velran/server.toml --check-config
```

16.14. Egy tipikus többpéldányos topológia

```

      +---> Velran A  --+---> application DB
Internet -> proxy --|           |
      +---> Velran B  --+---> Redis
              |
              +-----> shared AppFs (*)
```

A csillag fontos: ha az alkalmazás feltöltött fájlokat/AppFs üzleti állapotot használ, akkor több instance esetén annak tárolását is olyan deploymenttel kell megoldani, amely minden példány számára konzisztensen elérhető. A V1 AppFs API confinementet és biztonságos fájlműveleteket ad; attól még az infrastruktúra feladata eldönteni, hogyan lesz a data root több példány számára helyesen megosztott.

A komponensek szerepe:

- **reverse proxy/load balancer:** TLS, routing és instance-választás;
- **Velran instance:** statelesshez közelítő alkalmazásprocessz, lokális bounded runtime state-tel;
- **application DB:** tartós strukturált üzleti source of truth;
- **AppFs:** tartós fájl/mediális üzleti állapot, ha az alkalmazás használja;
- **Redis:** újraépíthető shared session/cache/rate-limit/replay infrastruktúra.

16.15. Rolling restart és cache generation

Új release rolloutjánál nem kell a régi cache kulcsokat kézzel törölni. A cache key része a route generation, ezért a release/compiler által meghatározott új generation új namespace-t eredményez. Explicit üzleti invalidáció továbbra is a state-changing route deklarációjából jön.

Rolling restartnál a proxy csak ready instance-nak küldjön forgalmat. A SIGTERM graceful drain útvonalat használd. Az orchestrator leállítási türelmi ideje legyen nagyobb a szerver konfigurált shutdown grace értékénél.

Sticky session nem követelmény, ha a shared session backend helyesen konfigurált.

16.16. Mit monitorozz?

A cache és rate limit önmagában nem láthatatlan optimalizáció. Minimum production monitoring:

- `velran_cache_hits_total` és `velran_cache_misses_total`;
- 429 válaszok és `velran_rate_limit_denials_total`;
- `rate_limiter_unavailable` események;
- readiness 503 események Redis/DB dependency hibánál;
- Redis latency, connection és memória állapot az infrastruktúra monitoringjában;
- az alkalmazás DB latency-je cache miss időszakokban.

Metric labelként továbbra se használj raw IP-t, username-et, session ID-t, tenantot, request ID-t vagy teljes URL-t. A későbbi observability fejezet bounded-cardinality szabálya itt is érvényes: metric label ne legyen felhasználónként vagy kérésenként korlátlanul változó.

16.17. Redis recovery szerepe

A Redis ebben az architektúrában runtime state: session, cache, rate-limit és replay-védelmi állapot élhet benne, de nem ez az üzleti source of truth. Emiatt recovery során a fő kérdés nem a cache visszatöltése, hanem az, hogy a Redis elvesztése után a rendszer biztonságosan és kiszámíthatóan építse újra ezt az állapotot.

A teljes backup/restore sorrendet és azt, hogy mely állapotot kell ténylegesen menteni, a 19. fejezet tárgyalja.

16.18. Gyakorlati példa: cache-elt híroldal publikálással

A publikus címlap userfüggetlen, ezért cache-elhető:

```
#[page]
fn frontPage(ctx: PageContext, db: Db)
  -> Result<Html, PageError> {
  let articles = latestArticles(db)?;
  return Ok(html {
    <main>
      @for article in articles {
        <a href="/cikk/{{ article.slug }}">
          {{ article.title }}
        </a>
      }
    </main>
  });
}

route frontPage GET "/"
  public cache public ttl 60
  => frontPage;
```

A publikálás autentikált state change, ezért nem cache-elhető, viszont invalidálja a címlapot:

```
route publish POST "/admin/publish/:id<i64>"
```

```
auth role Publisher
invalidate cache frontPage
=> publish;
```

A mentális modell így egyszerű marad:

1. az adatbázis a cikk igazságforrása;
2. a publikus HTML cache csak gyors másolat;
3. a publish tranzakció után az új generation miatt a következő GET friss oldalt épít;
4. ezt bármelyik Velran instance megteheti;
5. az elkészült response shared Redis cache-be kerül.

16.19. Ellenőrzőlista

Többpéldányos production deployment előtt ellenőrizd:

- Redis URL secret file-ből jön, TLS/security policy megfelel;
- shared session miatt nincs szükség sticky sessionre;
- local auth SQLite-ot nem kezeled shared multi-instance identity store-ként;
- public cache csak valóban userfüggetlen GET route-on van;
- minden releváns state-changing route explicit cache invalidációt deklarál;
- a cache TTL nem lépi túl az operator maximumot;
- productionben `allow_memory = false` cache-hez és rate limiterhez;
- rate-limit scope trusted identityből/effective client IP-ből képződik;
- Redis outage esetére 503/readiness viselkedést tesztelted;
- cache hit/miss és rate-limit denial metrikák monitorozva vannak;
- Redis állapotot nem tekinted üzleti backup kötelező részének.

16.20. Gyakorlat: osztályozd a cache-elhető adatot

Gyakorlási mód: önálló. Használd az előszóban meghatározott önálló módszert.

Minden cache-jelölnél nevezd meg az authoritative source-ot, key scope-ot, lifetime-ot, invalidation trigger-t és authorization contextet. Ha ezek közül bármelyik bizonytalan, halaszd a cache-t addig, amíg a contract explicit nem lesz.

16.21. Tipikus hiba: consistency előtt cache-elünk

A cache a stale vagy cross-user adatot is sokkal gyorsabbá teheti. A performance optimalizáció őrizze meg ugyanazokat az ownership és visibility szabályokat, mint a nem cache-elt út, az invalidation pedig szűken kapcsolódjon az authoritative állapotváltozáshoz.

16.22. Folyamatos mintaprojekt: Secure Notes

Csak olyan read pathot cache-elj, amelynek visibility szabályai már stabilak. Bizonyítsd, hogy publish, edit, delete vagy authorization-változás után sem maradhat más felhasználó számára stale vagy rosszul scope-olt reprezentáció.

17. fejezet

Naplózás, audit, metrics és hibakeresés

Tanulási út: Teljes út / üzemeltetési mélység. Akkor olvasd, ha production diagnosztikáért, telemetriáért és audit evidence-ért felelsz.

Biztonsági invariáns: a telemetry nem adatdump

A log és trace csak az üzemeltetéshez és vizsgálathoz szükséges minimális bizonyítékot őrizze meg, és ne váljon secretok, credentialök, tokenek vagy nyers request bodyk második adattárává.

Production hibakeresésnél össze kell kapcsolni a requestet, route-ot, státuszt, security döntést és az esetleges tartós business audit eseményt.

Első olvasás. Fókuszálj a request ID-ra, naplóosztályokra, redaction/cardinality szabályokra, healthre és a hibakeresési folyamatra. A proxy-, rotation- és retention-részletek operátori referenciaanyagok.

A Velran ehhez négy külön felületet ad:

- strukturált **server log** a technikai és lifecycle eseményekhez;
- strukturált **access log** a HTTP kérésekhez;
- strukturált **audit log** a security és felhasználói aktivitáshoz;
- külön listen socketen elérhető **Prometheus metrics** felületet.

Ehhez társul az alkalmazás adatbázisában tárolható, tranzakciós **business audit trail**. Fontos: az audit log és a business audit nem ugyanaz.

17.1. A három naplóosztály szerepe

Ajánlott production konfiguráció:

```
[observability]
metrics_listen = "127.0.0.1:9090"
allow_public_metrics = false
access_log = true

[logging]
server_file = "/var/log/velran/server.log"
```

```
access_file = "/var/log/velran/access.log"
audit_file = "/var/log/velran/audit.log"
stderr = true
```

A fájlok célja eltérő.

server.log Indulás, leállítás, dependency-hibák, technikai és belső runtime események. Ezt nézd először, ha a szerver nem indul, readiness hibás, vagy egy háttérkomponens problémát jelez.

access.log Egy rekord HTTP kérésenként. Ez adja a forgalmi és latency nyomot: method, route, státusz, időtartam, kliens IP, be- és kimenő byte-ok, request ID.

audit.log Biztonsági és felhasználói aktivitás: login/logout, auth vagy MFA failure, policy deny, rate-limit deny, resource-profile startup audit és autentikált state-changing action.

Az `access_log = false` csak az access logot kapcsolja ki. A security/user audit ettől még megmarad.

17.2. Request ID: egy kérés végigkövetése

Minden sikeresen parse-olt application, static vagy health request szerver-generált azonosítót kap:

```
X-Request-Id: velran-...
```

Az access log például ilyen lehet:

```
{"event":"http_request","request_id":"velran-...",
 "method":"GET","route":"articleShow","status":200,
 "duration_ms":7,"client_ip":"203.0.113.5",
 "bytes_in":0,"bytes_out":812}
```

A kliens által küldött `X-Request-Id` nem authority. A Velran saját request ID-t generál, így a támadó nem kényszerítheti rá a szerverre a saját korrelációs azonosítóját.

A hibakeresés alapművelete ezért:

```
# 1. keresd meg a request ID-t az access logban
grep 'velran-...' /var/log/velran/access.log

# 2. keresd ugyanazt a technikai logban
grep 'velran-...' /var/log/velran/server.log

# 3. security esetben az audit logban is
grep 'velran-...' /var/log/velran/audit.log
```

A három napló együtt adja a teljes technikai képet.

17.3. Mit szabad és mit tilos logolni?

A log nem debug adatlerakó. Productionban különösen tilos érzékeny request tartalmat automatikusan naplózni.

A Velran logfelületére nem kerülhet:

- `Authorization` header;
- Cookie vagy session ID;
- CSRF token;
- TOTP secret és recovery code;
- DB, Redis vagy LDAP credential;
- secret-file tartalma;
- teljes request body vagy form payload.

Tartós üzleti bizonyítékhoz explicit, szűk auditmezőket használj request-body naplózás helyett.

17.4. Security audit és HTTP elutasítások

A trust-boundary elutasításoknál a Velran külön `security_audit` eseményt képez ugyanazzal a request ID-val. Ide tartozik a hibás/untrusted forwarding metadata 400-as elutasítása, a 401/403/421/426/429 security döntések, valamint a CSRF/CORS/origin tiltások finomabb kategorizálása. Így elkülöníthető például:

- `proxy/invalid_forwarding`: hibás vagy nem trusted forwarding metadata;
- `auth/unauthorized`: autentikáció hiánya API route-on;
- `csrf/denied`, `cors/denied`, `origin/denied`: böngészős trust-boundary tiltás;
- `policy/forbidden`: route- vagy object-authorization deny;
- `host/mismatch`, `transport/https_required`: Host/TLS boundary sérülése;
- `rate_limit/denied`: limiter miatti elutasítás.

A fejlesztő számára fontos különbség: a 403 nem feltétlen programhiba. Lehet helyes security-policy döntés is. Hibakereséskor mindig nézd meg az audit eseményt és a route policyt is.

17.5. Business audit: tartós üzleti változástörténet

Az operációs log eseményt ír le; a tartós business state change tranzakciós business audit rekordba való.

Példa:

```
enum ArticleStatus {
    Review
    Published
}

transaction db {
    Article.publishChanged(tx, id, version)?;
    audit Article id action publish;
        from article.status
        to ArticleStatus.Published
}
```

Az audit ugyanabban a DB-tranzakcióban fut, mint a módosítás. Ha az audit rekord nem írható be, a business módosítás is rollbackel. Ez erősebb garancia, mint egy külön log sor kiírása.

A Velran a `_velran_business_audit` runtime táblába többek között az alábbiakat írja:

- runtime-generált eseményazonosító és UTC timestamp;

- HTTP request ID;
- canonical autentikált actor;
- source action;
- objektumtípus és objektumazonosító;
- statikus üzleti action név;
- opcionális előző és új domain állapot.

A business audit statement authorizationt nem helyettesít:

```
let invoice = Invoice.byId(db, id?);
authorize invoice owner ownerUsername or role Accountant;

transaction db {
  Invoice.approve(tx, id, version?);
  audit Invoice id action approve;
    from invoice.status
    to InvoiceStatus.Approved
}
```

A helyes sorrend: **azonosítás -> authorization -> módosítás -> ugyanazon tranzakción belüli audit.**

17.6. Platform security event, redaction és burst alert

A security monitoring strukturált platform boundary. Automatikus event keletkezik authentication/MFA abuse, policy/browser-boundary denial, webhook failure, idempotency conflict és resource exhaustion esetén.

Az event schema szándékosan nem fogad arbitrary request payloadot vagy message textet. Emitted principal/source mező redacted. Ha alkalmazáskódnak érzékeny szövegből biztonságos monitoring reprezentáció kell, a trusted `redact(...)` builtint használd; `Redacted<T>` annotationnel vagy casttal nem hozható létre.

A server bounded in-memory correlationt és platform-owned burst thresholdokat/cooldownt is használ. Threshold átlépéskor strukturált `threshold_exceeded` alert event keletkezik. Raw correlation key átmenetileg memóriában lehet, de security eventbe nem kerül.

Ez a monitoring layer nem helyettesíti az enforcementet. Rate limiting, authorization, replay prevention és resource limit továbbra is blokkol; az alert detection és incident-response evidence-et ad.

17.7. Prometheus metrics külön management socketen

A metrics nem application route. Külön listeneren fut:

```
[observability]
metrics_listen = "127.0.0.1:9090"
allow_public_metrics = false
```

Lekérdezés ugyanazon a hoston:

```
curl http://127.0.0.1:9090/metrics
```

A listener csak a `/metrics` GET/HEAD kérést szolgálja ki. Nem fut rajta alkalmazás-route, session vagy auth handler.

Tipikus metrikák:

```

velran_requests_total
velran_responses_5xx_total
velran_auth_failures_total
velran_csrf_failures_total
velran_policy_denials_total
velran_request_timeouts_total
velran_runtime_budget_exceeded_total
velran_rate_limit_denials_total
velran_readiness_failures_total
velran_request_bytes_in_total
velran_response_bytes_out_total
velran_active_connections
velran_log_fallback_total
velran_request_duration_ms_bucket
velran_route_requests_total{route="..."}
velran_route_5xx_total{route="..."}
velran_route_duration_ms_sum{route="..."}

```

A `velran_policy_denials_total` a security-policy tiltásokat követi: a 403/421/426 döntések mellett a kifejezetten forwarding metadata hibaként felismert 400-as elutasítás is ide számít. A normál malformed request 400 nem security policy denial.

A legfontosabb szabály a **bounded cardinality**. A `route` label compiler-owned route név vagy fix reserved név. Raw URL, path paraméter, IP, username, tenant, session ID és request ID soha ne legyen metric label. Ellenkező esetben egy támadó korlátlan idősor-számot hozhatna létre.

17.8. Apache vagy Nginx mellett ki scrape-elje a metricset?

A publikus webes proxy és a metrics listener külön trust boundary. Tipikus elrendezés:

```

Internet
  |
  v
Apache/Nginx :443
  |
  v
127.0.0.1:8080  Velran server

Prometheus agent
  |
  v
127.0.0.1:9090  Velran metrics

```

A metrics portot ne proxyd ki automatikusan ugyanarra a publikus domainre. Ha távoli Prometheus szervernek kell elérnie, használj privát management hálózatot, ACL-t vagy külön autentikált monitoring proxyt.

17.9. Health: liveness és readiness nem ugyanaz

A Velran beépített szerver endpointjai:

```
GET /health/live
HEAD /health/live

GET /health/ready
HEAD /health/ready
```

A liveness csak azt jelzi, hogy a szerver event loop él. Nem hív DB-t, Redis-t vagy LDAP-ot.

A readiness azt mutatja, hogy a kiszolgáláshoz szükséges konfigurált runtime dependency-k elérhetők. DB vagy Redis hiba/timeout esetén 503 **Service Unavailable** lehet a válasz.

Ezért load balancernél:

- a **liveness** alapján ne vedd ki rögtön az instancet egy rövid DB kiesés miatt;
- a **readiness** alapján viszont ideiglenesen kivehető a forgalomból.

A health endpoint nem üzleti monitoring API: szándékosan minimális státuszt közöl.

17.10. Logrotate és SIGHUP

A Velran file logging append módban ír. Productionban a logrotációt az operációs rendszerre bíz. A repository példája: `examples/logrotate/velran`.

Egyszerűsített minta:

```
/var/log/velran/server.log /var/log/velran/access.log {
    daily
    rotate 14
    compress
    delaycompress
    missingok
    notifempty
    create 0640 velran velran
    sharedscripts
    postrotate
        systemctl kill -s HUP --kill-who=main velran.service || true
    endscript
}
```

A **SIGHUP** jelentése tudatosan korlátozott. Behind-proxy módban újraindítja a logfájlokat és tranzakciósan újraépíti a domain/application hosting állapotot; nem általános process-szintű `server.toml` rekonfiguráció. Az alkalmazásforrás változását normál esetben az automatikus source-reload supervisor kezeli.

Konfigurációmódosításnál:

```
/usr/local/bin/velran-server \
  --config /usr/local/etc/velran/server.toml \
  --check-config

systemctl restart velran.service
```

Ez auditálhatóbb és kiszámíthatóbb, mint egy részlegesen sikerülő hot reload.

17.11. Mi történik, ha a logfájl nem írható?

A file logging bounded writer queue-t használ. Ha a queue megtelik, a file sink kiesik vagy a writer nem érhető el, a Velran stderr fallbacket használ és növeli a következő metrikát:

```
velran_log_fallback_total
```

Ezt érdemes riasztási feltételként kezelni. Ha növekszik, lehet:

- jogosultság- vagy filesystem-hiba;
- megtelt lemez;
- hibás logrotate/reopen;
- tartós log-processing nyomás.

A logfájl útvonalának parent directory-ja létezzen. Unix rendszeren a runtime a logfájlt 0640 móddal, symlink-követés nélkül nyitja meg; az operatornak ettől még helyesen kell kialakítania a könyvtárjogosultságokat.

17.12. Gyakorlati production hibakeresési folyamat

Egy 5xx vagy „lassú az oldal” bejelentésnél ne találgass. Használj reprodukálható sorrendet.

1. **Liveness:** fut-e maga a process?
2. **Readiness:** elérhető-e DB/Redis?
3. **Access log:** melyik route, státusz, duration és request ID érintett?
4. **Server log:** ugyanahhoz a request ID-hoz tartozik-e dependency/runtime hiba?
5. **Audit log:** security policy vagy rate limit utasította-e el?
6. **Metrics:** egyedi eset vagy trendszerű 5xx/latency növekedés?
7. **Reverse proxy:** a hiba az Apache/Nginx előtt, benne vagy a Velran mögött keletkezik?

Példa gyors ellenőrzésre:

```
curl -i https://www.example.com/health/live
curl -i https://www.example.com/health/ready
# Proxy deploymentnel plusz belso upstream-proba:
curl -i http://127.0.0.1:8080/health/ready
curl -s http://127.0.0.1:9090/metrics | grep velran_responses_5xx_total
journalctl -u velran.service --since "10 min ago"
```

Ha a loopback Velran endpoint jó, de a publikus domain rossz, a következő vizsgálati réteg az Apache/Nginx, TLS vagy forwarded-header policy. Ha a loopback readiness is 503, előbb a backend dependency-ket vizsgáld.

17.13. Minimum monitoring baseline

Egy kisebb production telepítésnél is legyen legalább riasztás az alábbiakra:

- readiness tartósan nem ready;
- 5xx arány vagy számláló hirtelen növekedése;
- request latency érdemi romlása;
- auth/policy/rate-limit denial szokatlan kiugrása;

- `velran_runtime_budget_exceeded_total` növekedése;
- `velran_log_fallback_total` növekedése;
- process restart loop vagy váratlan shutdown.

A konkrét thresholdot nem a nyelvnek kell hardcode-olnia. A forgalom, SLO és üzleti kritikalitás alapján az üzemeltető állítja be.

17.14. Retention, SIEM és auditvédelem

A retention szervezeti és jogi policy. A Velran nem kényszerít rá univerzális megőrzési időt.

A helyi `audit.log` fontos operációs nyom, de önmagában nem erős tamper-evident archívum, mert a service host adminisztrátora vagy megfelelő filesystem jogosultságú szereplő módosíthatja. Magasabb assurance esetén továbbítsd centralizált, hozzáférés-védett vagy append-only log/SIEM rendszerbe.

A business audit adatbázis-tábla ugyancsak nem véd a DB-admin közvetlen módosítása ellen. A két réteg együtt ad jó alkalmazási nyomot, de compliance környezetben a külső, független logarchívum marad fontos.

17.15. Ellenőrzőlista

Production indulás előtt ellenőrizd:

- külön `server/access/audit` log útvonal van;
- a logkönyvtár jogosultságai szűkek;
- request body és secret nincs logolva;
- a metrics listener nem publikus véletlenül;
- a Prometheus labelk bounded cardinality-jűek;
- a liveness és readiness külön van kezelve;
- logrotate után SIGHUP történik, configmódosítás után restart;
- `velran_log_fallback_total` monitorozva van;
- üzleti workflow esetén a business audit ugyanabban a DB-tranzakcióban fut;
- az audit log központi továbbításáról és retentionjéről van dokumentált döntés.

17.16. Gyakorlat: tervezz evidence trailt

Gyakorlási mód: önálló. Használd az előszóban meghatározott önálló módszert.

Egy fontos kérdéshez sorold fel azokat az operátori tényeket, amelyekből rekonstruálható, mi történt anélkül, hogy secretet vagy teljes érzékeny payloadot logolnál. Legyen correlation/request identity, eredmény, latency, policy rejection és releváns domain-audit azonosító.

17.17. Tipikus hiba: események helyett adatot logolunk

A nagyobb logvolumen nem automatikusan jobb observability. Strukturált eseményeket részesíts előnyben, amelyek döntéseket és eredményeket magyaráznak. Credentialt, tokenet, request body-t vagy érzékeny üzleti rekordot ne másolj logba csak azért, mert elérhető.

17.18. Folyamatos mintaprojekt: Secure Notes

Definiálj observable eseményeket sikertelen autorizációra, publish átmenetre, integration failure-re és váratlan szerverhibára. A business audit és az operational telemetry maradjon külön, de legyen korrelálható.

18. fejezet

Production deployment és service lifecycle

Tanulási út: Gyors út: production alap. Tanuld meg a release/activation contractot és a biztonságos deployment workflow-t.

Biztonsági invariáns: aktiválás előtt ellenőrzés

A teljes candidate generációt fordítsd és validáld, mielőtt aktívvá válhatna. Ha a verification vagy activation elbukik, az előző known-good generáció szolgáljon tovább.

Production diagnostic: SEC-PROD-002

Jelentés: Az alkalmazás hiányos production policyt deklarál, ezért a szigorú deployment contract nem bizonyítható.

Javítás: Deklaráld mind a négy kötelező invariantot: HTTPS required, debug disabled, HSTS required és database TLS required.

Ez a fejezet nem ismétli meg a konfigurációs referencia és az observability részleteit. A production szempontból azt foglalja össze, hogyan lesz egy ellenőrzött release-ből futó service, hogyan történik a release-váltás, és hogyan marad kontrollált a process lifecycle. A teljes backup/restore és disaster recovery a következő fejezet önálló témája.

18.1. Az alkalmazás production contractja

A production security egy része magában a Velran programban deklarált:

```
production {  
    https required;  
    debug disabled;  
    hsts required;  
    database tls required;  
}
```

Ez nem második secret/config rendszer. Követelményt deklarál; certificate/path/trusted network továbbra is server/operator config. Másképp fogalmazva: a Velran source azt mondja meg, *minek kell igaznak lennie*, a trusted deployment config pedig azt, *hogyan teszi ezt igazzá az adott környezet*. Startup elbukik, ha a valós topology nem teljesíti a contractot. Source

reload nem változtathatja meg csendben; production security requirement módosítás restartot és explicit operator review-t igényel.

18.2. Release útvonal

Ajánlott sorrend:

```
./verify.sh
-> cargo build --locked --release
-> immutable release directory
-> velran-server --config ... --check-config
-> velran-cli migrate verify
-> approved velran-cli migrate apply
-> velran-cli migrate verify
-> atomic current symlink switch
-> controlled restart
-> /health/live
-> /health/ready
-> log / metric / audit review
```

A buildelt **velran-server** és **velran-cli** bináris, az alkalmazásforrás és a migration legyen ugyanahhoz a release-azonosítóhoz köthető. Ne egy élő **/srv/velran/current** könyvtár tartalmát módosítsd fájlanként; új immutable release könyvtárat készíts, majd atomikusan válts symlinket.

18.3. systemd service

A **velran-server** hosszú életű service process. Az **velran-cli** nem daemon: migration, auth-admin vagy más operátori művelet idejére indul és kilép.

```
[Service]
User=velran
Group=velran
ExecStart=/usr/local/bin/velran-server \
  --config /usr/local/etc/velran/server.toml
Restart=on-failure
NoNewPrivileges=true
PrivateTmp=true
```

A service fusson nem privilegizált felhasználó alatt. systemd esetén a process-szintű cgroup felső korlátok – például **MemoryMax**, **CPUQuota** és **TasksMax** – tulajdonosa a systemd legyen; a Velran ne próbálja ugyanazokat a cgroup kontrollokat párhuzamosan írni. Minden erőforrás-határhoz egyértelmű authority tartozzon.

Közvetlen TLS esetén a 80/443 portokhoz csak a bind-service capabilityt (**CAP_NET_BIND_SERVICE**) add meg. Ha a Velran reverse proxy mögött, például 127.0.0.1:8080-on figyel, erre nincs szükség.

18.4. Controlled restart és readiness

Process-szintű konfigurációmódosítás után előbb fusson **-check-config**, majd controlled restart. Behind-proxy módban a SIGHUP a logok újrainvitása mellett tranzakciósan újraépíti

a domain/application hosting állapotot, de listener, DB/Redis/auth kapcsolat, cgroup policy, logging sink és más process-szintű beállítás továbbra is restartot igényel.

Az application source változása az alább részletezett candidate-activation útvonalat használja: request handlingtől elkülönített compile/validation után atomikus domaincsere történik. Elutasított candidate esetén a korábbi generáció marad aktív.

Rolling deploymentnél csak ready instance kapjon forgalmat: indítsd el az újat, várd meg a readiness állapotot, majd vedd ki a régit. A health diagnosztika az observability fejezetben szerepel.

18.5. Migration és rollback nem ugyanaz

Az alkalmazás binárisának vagy forrásának visszaváltása egyszerű lehet, az adatbázis-séma visszaállítása viszont nem. Ezért production migrationnél preferáld az expand/contract mintát: az új release előbb backward-compatible sémát vezessen be, az inkompatibilis takarítás csak későbbi release-ben történjen.

Ne kapcsolj automatikus reverse SQL-t az alkalmazás symlink visszaállításához. Ha a migration adatot alakított át vagy törölt, a valódi recovery pont a bizonyított backup lehet.

18.6. Recovery gate a deployment előtt

Destruktív vagy schema-kompatibilitást érintő release előtt legyen ismert recovery point és friss restore-evidence. A backup tartalma, a DB/AppFs konzisztencia, az RPO/RTO, valamint a rollback és restore döntési fa a következő fejezetben szerepel. A deployment felelőssége itt az, hogy a release-váltás előtt ez a gate teljesüljön.

18.7. Deploy utáni minimum ellenőrzés

1. az új process fut és ready;
2. a publikus főoldal vagy smoke route válaszol;
3. nincs új error spike a server logban;
4. access logban a státuszkód-eloszlás normális;
5. audit log írható;
6. DB/Redis dependency metrikák nem romlottak;
7. egy rollback döntéshez ismert a release hash és az adatbázis migration állapota.

18.8. Release-invariantok

Production release-nél a deployment tooling részleteitől függetlenül maradjon igaz néhány alapinvariant:

- a build és verification a locked dependency graphot használja;
- application source, migrationök, server/CLI binárisok és a jóváhagyott config verzió egy release recordhoz köthető;
- production secret nem kerül az immutable release fába;
- candidate alkalmazás csak teljes fordítás és validáció után válhat aktívvá;
- az aktiválás atomikus: request vagy a régi, vagy az új generációt látja, félbemaradt keveréket nem;
- hibás candidate esetén a korábbi known-good generáció szolgál tovább;

- a váltás előtt ismert a DB migration state és az application rollback kompatibilitási határa.

Operátori szabály: a candidate vagy átmegy a teljes gate-en és atomikusan aktiválódik, vagy nem aktiválódik. A natív fordítás hibája deployment hiba, soha nem fallback-jelzés.

18.9. Mi reloadolható és mi igényel restartot?

A source reload szándékosan szűkebb, mint a process reconfiguration. A kettőt külön operátori eseményként kezeld.

Változás	Normál mechanizmus	Indok
Velran application source/modulgráf	candidate compile, validation, atomic generation swap	application-local code generation
Immutable release symlink célja	a figyelt source path candidate generationt indít	megőrzi az atomikus release layoutot
Logrotation támogatott topológiában	SIGHUP/log reopen	nincs application-policy változás
Listener/TLS/process topology	config check + controlled restart	process-level authority
DB/Redis/auth connection config	config check + controlled restart	connection/service lifecycle
Cgroup/resource authority	controlled restart	process-level isolation boundary
Production security requirement	operator review + restart	source reload nem gyengítheti csendben a policyt

Ez a különbség megakadályozza, hogy a kényelmes source reload implicit process-config csatornává váljon.

18.10. Candidate aktiválás: az operátori szabály

Kezeld az aktiválást kétlépcsős határként: *előbb compile/validate, utána activate*. Hiba esetén a known-good generáció szolgál tovább; az aktív generált artifactot ne módosítsd helyben. Alkalmazásfejlesztőként ennél a contractnál megállhatsz.

A motorháztető alatt: hogyan őrzi meg a natív aktiválás a határt

Alkalmazásfejlesztőként első olvasáskor ezt a részt átugorhatod. A source-reload supervisor a logikai entypointot és a teljes tranzitív modulgráfot figyeli. Miután a fájlkészlet a debounce időn át stabil, a fordítás a request handlingtől elkülönítve történik. A candidate ugyanazonon a route-, resource-, cache-, authentication- és hosting ellenőrzéseken megy át, mint startupkor.

Belsőleg az elfogadott forrás verified executable IR-ré, generált safe Rusttá, majd `rustc` által fordított immutable `cdylib`-bé és új runtime-generációvá válik. Csak ezek sikere után történik az adott domain runtime atomikus cseréje. A régi runtime-ot már birtokló requestek azon a generáción fejeződnek be; a későbbiek az újat használják. Sikeres kódaktiváláskor a public-cache generationök is ugyanennél a határnál lépnek előre.

Native compile vagy activation hiba esetén nincs VM/interpreter fallback. Ez implementációs

részlet, de fontos operátori következménnyel: a hiba látható marad, és a known-good generáció marad az authority.

18.11. Pre-deploy evidence

Forgalom- vagy release-váltás előtt legyen elegendő evidence ahhoz, hogy később négy kérdésre válaszolni lehessen:

1. **Mi épült?** release identifier, source revision/artifact identity és hashek.
2. **Milyen dependency és capability lett jóváhagyva?** locked Cargo graph, valamint supply-chain capability/provenance verification.
3. **Milyen sémát várunk?** migration set és sikeres migration verification.
4. **Hogyan állunk helyre?** ismert backup/restore readiness és az application rollback kompatibilitási határa.

Ez az evidence szándékosan secretmentes. Provenance-ot és operációs állapotot bizonyít anélkül, hogy credentialt másolna a release recordba.

18.12. Deployment anti-patternök

Kerüld ezeket a shortcutokat:

- unlocked release build, amely csendben dependency resolutiont módosíthat;
- fájlok közvetlen szerkesztése az aktív release könyvtárban;
- application rollback automatikus adatbázis-rollbackként kezelése;
- migration implicit futtatása server startup közben;
- secret CLI argumentba, release manifestbe vagy source-ba írása;
- forgalom küldése readiness siker előtt;
- hibás candidate elfogadása egy másik execution engine-re való fallbackkel.

18.13. Gyakorlat: próbálj el egy sikertelen aktiválást

Gyakorlási mód: üzemeltetési szcenárió. Használd az előszóban meghatározott üzemeltetési módszert.

Készíts olyan candidate release-t, amely szándékosan elbukik egy pre-activation ellenőrzésen. Bizonyítsd, hogy az aktív generáció változatlan marad, a hiba pedig látható az operátornak. Ezután próbálj el egy sikeres aktiválást és rollbacket is.

18.14. Tipikus hiba: a build sikerét deployabilitynek tekintjük

A lefordított artefakt csak egy része a release evidence-nek. Konfiguráció, migration, secret, TLS anyag, resource policy, health check és activation semantics mind legyen érvényes, mielőtt traffic függ az új generációtól.

18.15. Folyamatos mintaprojekt: Secure Notes

Fagyaszd be az `examples/secure-notes/checkpoints/18-production/main.vrn` snapshotot production candidate-ként. Csomagold a Secure Notes-ot production candidate-ként. Rögzítsd a pontos verification parancsokat, migration állapotot, config checket, activation evidence-et és rollback pontot. A cél ismételhető release-eljárás, nem egyszeri sikeres telepítés.

19. fejezet

Backup, restore és disaster recovery

Tanulási út: Gyors út: production alap. Definiáld a helyreállíthatóságot, mielőtt a production adat pótolhatatlanná válik.

A backup akkor értékes, ha az alkalmazás **bizonyíthatóan újraindítható egy ismert recovery pontból**. Az üzleti állapot application DB-n, local-auth identity store-on és AppFs-en is átnyúlhat, ezért előre rögzítsd a teljes recovery state-et, a konzisztencia-határt és a restore bizonyításának módját.

19.1. Először készíts state inventoryt

Production előtt írd össze, mi tartozik az alkalmazás állapothoz. Tipikusan:

- application database;
- local-auth SQLite DB, ha local auth használatban van;
- AppFs `data_root`, például feltöltött média és más runtime adat;
- az immutable release azonosítója és artifact hash-e;
- a migration set és checksumok;
- a trusted config/policy fájlok verziója vagy hash-e;
- a backup időpontja, az alkalmazásverzió és a DB backend.

A secret fájlok külön secret-management boundaryhoz tartoznak. A recovery runbooknak tudnia kell, honnan állíthatók helyre a szükséges DB-, Redis-, TLS- vagy LDAP-credentialök, de **secret érték ne kerüljön plaintext release- vagy backup-manifestbe**.

19.2. Mi nem üzleti source of truth?

A V1 Redis-használat session-, rate-limit- és public-cache állapot. Ezeket ne kezeld üzleti backup részeként.

Disaster recovery után biztonságos és elfogadható baseline az üres Redis:

- a sessionök megszűnnek, ezért a felhasználók újra belépnek;
- a public cache újraépül;
- a rate-limit ablakok újraindulnak.

Ha ugyanaz a Redis cluster más alkalmazás üzleti adatait is tárolja, annak mentése már annak a rendszernek a saját backup-contractja.

19.3. RPO és RTO: a technikai mentés előtt üzleti döntés

Két számot még a backup implementálása előtt rögzíteni kell:

RPO – Recovery Point Objective legfeljebb mennyi adatvesztés fogadható el;

RTO – Recovery Time Objective mennyi idő alatt kell helyreállnia a szolgáltatásnak.

Ha például az RPO 24 óra, egy napi backup elvileg elegendő lehet. Ha az RPO néhány perc, már DB-szintű WAL/binlog/PITR vagy más infrastruktúra kellhet. A Velran runtime nem tud szervezeti RPO/RTO-t kitalálni; ezt az operatornak kell a backup technológiára lefordítania.

19.4. A V1 biztonságos baseline: kontrollált maintenance backup

A DB és az AppFs együtt alkothat üzleti állapotot. Egy DB sor például hivatkozhat egy feltöltött képre. Külön időpontban készült DB dump és AppFs tar ezért nem automatikusan konzisztens recovery point.

A legegyszerűbb auditálható V1 baseline:

```
traffic drain
-> SIGTERM / graceful shutdown
-> stopped state verified
-> application DB backup
-> local-auth DB backup, if used
-> AppFs data_root snapshot
-> recovery manifest + hashes
-> service start
-> readiness
```

Ez rövid írási szünettel jár, viszont világos konzisztenciahatárt ad. Online backup csak akkor nevezhető konzisztensnek, ha a DB/storage infrastruktúra ugyanahhoz a recovery ponthoz kötött snapshotot biztosít, vagy az alkalmazás adatmodellje bizonyíthatóan tolerálja az eltérő snapshot-időpontokat. A runtime önmagában nem ígér distributed snapshotot.

19.5. Application DB: a backend saját backup eszközét használd

A Velran nem próbálja elrejtetni a natív adatbázis-backup mechanizmusokat. Az operator a választott backend támogatott eszközével dolgozzon, lehetőleg külön backup credentialdel.

PostgreSQL

Egy tipikus logikai backup:

```
pg_dump --format=custom --no-owner --no-acl \  
--file app.dump "$DATABASE_URL"
```

Izolált restore-próbához:

```
createdb velran_restore_test  
pg_restore --no-owner --no-acl \  
--dbname velran_restore_test app.dump
```

A production secretet ne tedd shell historyba. Nagy adatbázisnál fizikai backup és PITR lehet jobb választás; annak runbookja már a PostgreSQL platform része.

MariaDB

Tranzakcionális InnoDB schema esetén tipikus logikai mentés:

```
mariadb-dump --single-transaction \  
--routines --triggers app > app.sql
```

Restore izolált adatbázisba:

```
mariadb velran_restore_test < app.sql
```

A `-single-transaction` nem tesz nem tranzakcionális táblákat varázsütésre konzisztenssé. Vegyes schema esetén maradjon a kontrollált maintenance baseline.

SQLite

Élő SQLite adatbázist ne nevezz backupnak csak azért, mert `cp`-vel lemásoltad. Használd a SQLite backup mechanizmusát vagy stopped állapotban fájlrendszer-snapshotot.

```
sqlite3 /srv/velran/data/app.db \  
".backup '/backup/app.db'"
```

A restore-próba mindig külön test pathra történjen; ne production DB-n próbáld ki, hogy működik-e a mentés.

19.6. A local-auth DB külön security backup

Local auth használatakor az identity store külön SQLite boundary. Password hash, TOTP secret és recovery hash is lehet benne, ezért backupját **secretként** kezeld.

```
sqlite3 /srv/velran/auth/users.db \  
".backup '/backup/local-auth.db'"
```

A backup legyen titkosított storage-on, szűk hozzáféréssel és dokumentált retentionnel. Restore után ellenőrizd a fájl és a parent directory ownership/permission értékeit is. Az identity store elvesztése nem rekonstruálható az application DB-ből.

19.7. AppFs: a teljes data root tartós állapot lehet

Ne csak a HTTP-n publikált media alkönyvtárat mentsd. A deklarált AppFs `data_root` teljes tartalma lehet recovery state.

Stopped baseline mellett például:

```
tar -C /srv/velran -cpf data-root.tar data
```

Restore után őrizd meg vagy állítsd vissza a szükséges ownership/permission értékeket. A service user csak a konfigurációban deklarált writable rootokat írhasa; symlinkkel ne lazítsd az AppFs confinementet.

19.8. Minden recovery ponthoz tartozzon manifest

A backup objektumok önmagukban nem mondják meg, melyik release-hez és migration állapothoz tartoznak. Készíts secretmentes manifestet:

```
backup_id=2026-09-04T180000Z
release_id=2026-09-04.1
server_sha256=...
app_source_sha256=...
migrations_sha256=...
config_sha256=...
db_backend=postgresql
app_db_backup_sha256=...
local_auth_backup_sha256=...
data_root_backup_sha256=...
```

A manifest hashokat és azonosítókat tartalmazzon, ne credential-értékeket. A mellékelt `deploy/release-record.sh` read-only módon segít a release/source/migration/config állapot rögzítésében.

19.9. Restore drill: a backup bizonyítása

A sikeres backup job még nem bizonyít recovery képességet. Rendszeresen, izolált környezetben restore-próbát kell futtatni.

```
new empty restore DB
-> application DB restore
-> local-auth restore, if used
-> AppFs restore into separate root
-> select exact immutable release
-> restore-test config on isolated host/port
-> velran-server --check-config
-> velran-cli check main.vrn
-> velran-cli migrate verify
-> start velran-server on isolated listener
-> /health/live + /health/ready
-> application-specific smoke test
-> log and audit verification
```

A `deploy/restore-verify.sh` a nem mutáló config/source/migration ellenőrzések egy részét automatizálja. Nem indít production listenert és nem futtat `migrate apply`-t.

A drill eredményét dátummal, backup ID-val és release ID-val dokumentáld. A „backup elkészült” nem release gate; a szervezet RPO/RTO elvárásához illő, friss restore-evidence kell.

19.10. Upgrade előtt legyen recovery döntés

Egy kontrollált production upgrade ajánlott sorrendje:

```
immutable release artifact + hashes
-> migration verify
-> compatibility decision
-> consistent backup + recent restore evidence
-> migration apply
```

```
-> migration verify
-> atomic release switch
-> controlled restart
-> live + ready + smoke
-> logs + metrics + audit review
```

A migration kompatibilitását release-szinten rögzíteni kell. A kulcskérdés: a régi alkalmazás fut-e az új schema mellett?

19.11. Expand/contract tartja nyitva a gyors rollback ablakot

Ha az új schema backward-compatible, egy hibás application release visszakapcsolható a korábbi immutable artifactra DB restore nélkül.

Ajánlott minta:

1. **expand**: új nullable/defaultolt oszlop, tábla vagy index úgy, hogy a régi alkalmazás tovább fusson;
2. deployold az új alkalmazást;
3. ha kell, végezz kontrollált adat-backfillt;
4. csak későbbi release-ben **contract**: távolítsd el a régi oszlopot/constraintet, amikor a rollback window már lezárult.

Destruktív rename/drop vagy azonnali inkompatibilis shape-váltás bezárhatja az app-only rollback lehetőségét. Ezt release előtt explicit jelöld.

19.12. Rollback és restore két külön művelet

Hibás alkalmazás, backward-compatible schema

```
traffic drain
-> previous immutable release
-> controlled restart
-> live + ready + smoke
```

Ilyenkor nincs DB restore, ezért a hiba után létrejött üzleti adat megmarad.

Hibás alkalmazás, de forward fix lehetséges

Preferáld az új immutable release-t vagy forward migrationt. Ne köss automatikus reverse SQL-t a release symlink visszaváltásához.

Schema vagy adat sérült

Ez már disaster recovery:

```
stop writes
-> choose and record recovery point
-> restore DB + auth DB + AppFs consistently
-> select matching release and config
-> verify + smoke
-> controlled return to traffic
```

A recovery point után keletkezett írások elveszhetnek. Ez üzleti incidensdöntés, nem technikai részlet: az adatvesztési ablakot explicit vállalni és dokumentálni kell.

19.13. Tiltott rövidítések

- élő SQLite DB egyszerű cp másolatát backupnak nevezni;
- csak az application DB-t menteni, ha AppFs is üzleti állapot;
- restore-próba nélkül sikeresnek tekinteni a backupot;
- automatikus down migrationt kötni application rollbackhez;
- production DB-n restore-próbát végezni;
- migration credentialt átadni a normál application service-nek;
- secret értéket release- vagy backup-manifestbe írni;
- Redis session/cache restore-t a disaster recovery feltételévé tenni.

19.14. Production recovery checklist

Egy rendszer akkor tekinthető recovery-readynek, ha legalább az alábbiak igazak:

1. a state inventory dokumentált;
2. az RPO és RTO rögzített;
3. a backup titkosítási, retention- és hozzáférési policy ismert;
4. van azonosítható, friss sikeres restore drill;
5. a release migration kompatibilitása dokumentált;
6. az app-only rollback eljárás próbált;
7. a destruktív restore tulajdonosa és jóváhagyási útja ismert;
8. Redis elvesztésének hatása elfogadott;
9. a recovery runbookból nem hiányzik a release/config/secret visszaállítási forrás;
10. incidens után live, ready, smoke, log és audit ellenőrzés is történik.

A recovery legfontosabb elve: **a mentés nem fájl, hanem bizonyított visszaállítási képesség**. Ha a restore drill nem fut, az RPO/RTO csak feltételezés.

19.15. Gyakorlat: restore-t bizonyíts, ne backupot

Gyakorlási mód: üzemeltetési szcenárió. Használd az előszóban meghatározott üzemeltetési módszert.

Állíts vissza egy alkalmazásállapot-másolatot izolált környezetbe, majd ellenőrizd a rekordszámokat, kritikus üzleti invariánsokat, media referenciákat és az alkalmazás indulását. Mérd a restore időt, és jegyezd fel az incidenskezelést blokkoló manuális lépéseket.

19.16. Tipikus hiba: komponenseket mentünk közös recovery modell nélkül

Adatbázissorok, feltöltött fájlok, konfiguráció és release identity együtt alkothatnak egy helyreállítható állapotot akkor is, ha külön tárolódnak. A backup terv mondja meg, hogyan illeszkednek össze restore idején.

19.17. Folyamatos mintaprojekt: Secure Notes

Végezz teljes Secure Notes recovery próbát a tartós adattal és az esetleges media attachmentekkel együtt. Ellenőrizd, hogy a helyreállított generáció ugyanazokat az ownership és publication szabályokat szolgálja ki, mint a szimulált hiba előtt.

V. rész

Referencia és operátori gyorssegédlet

20. fejezet

A Velran CLI és a napi fejlesztői/üzemeltetői workflow

Tanulási út: Gyors út: operátori referencia. Használd következetesen a CLI-t check, build, verification és operátori workflow-khoz.

A Velran két telepített programot ad: a hosszú életű **velran-server** processzt és a rövid életű **velran-cli** operátori eszközt. A kettőt érdemes következetesen külön kezelni. A szerver HTTP-t szolgál ki és runtime policyt érvényesít; a CLI forrást ellenőriz, migrációt kezel és local-auth adminisztrációt végez.

Ez a fejezet nem új nyelvi feature-öket vezet be. Azt mutatja meg, hogyan áll össze a napi munka egy ismétlődő parancsfolyamattá a szerkesztéstől a production ellenőrzésig.

20.1. Melyik feladathoz melyik program?

Feladat	Eszköz	Megjegyzés
Velran forrás ellenőrzése	velran-cli <code>check</code>	Compiler-szintű, nem indít HTTP szerveret.
Migration status/verify/apply	velran-cli <code>migrate</code>	DB URL secret fájlból.
Local-auth store admin	velran-cli <code>auth</code>	SQLite identity store, operátori művelet.
Server config preflight	velran-server	<code>-check-config</code> : config + app + lokális policy/resource fájlok ellenőrzése.
Effective config megtekintése	velran-server	<code>-print-effective-config</code> ; a secret értékek redaktálva maradnak.
Webalkalmazás futtatása	velran-server	Hosszú életű processz, proxy vagy közvetlen HTTPS mögött.
Workspace release build	cargo <code>build</code>	<code>-locked -release</code> módban készül a két bináris.
Platform teljes verifikációja	<code>./verify.sh</code>	A Velran repository release-evidence része, nem minden alkalmazás saját parancsa.

A production példákban a programok helye:

```
/usr/local/bin/velran-cli  
/usr/local/bin/velran-server
```

A shell normál `PATH` beállítása mellett ezért a további példákban elég a rövid programnevet használni.

20.2. A CLI nem a szerver alternatívája

A `velran-cli` nem indít listenert, nem szolgál ki route-ot és nem tart fenn session state-et. Egy parancsot végrehajt, majd kilép. Ez biztonsági szempontból is hasznos: migration vagy user-admin művelethez nem kell külön admin HTTP endpointot nyitni.

A másik irányban a `velran-server` nem futtat automatikus migrationt startupkor. A schema változás explicit operátori döntés marad.

20.3. Az első parancs: `velran-cli check`

Napi fejlesztésben minden érdemi forrásmódosítás után ez legyen az első kapu:

```
velran-cli check main.vrn
```

Siker esetén a CLI rövid összegzést ír a fordító által látott model-, query-, page-, action- és route-számról. Hiba esetén nem érdemes szerveret indítani: előbb a compile-time contractot kell kijavítani.

A `check` többek között olyan hibákat fog meg, mint:

- ismeretlen típus vagy hibás nyelvi szintaxis;
- query projection és model shape eltérése;
- hibás typed bind/decode contract;
- route/policy vagy resource profile hivatkozási hiba, amely compiler-szinten bizonyítható;
- tiltott raw HTML/SQL jellegű konstrukció.

A `check` viszont nem bizonyítja, hogy a DB vagy Redis hálózaton elérhető, és nem helyettesíti a HTTP integration tesztet.

20.4. Forrásfából futtatás és telepített CLI

A Velran repository fejlesztésekor ugyanaz a program Cargo-n keresztül is futtatható:

```
cargo run --locked -p velran-cli -- check main.vrn
```

Ez fejlesztői kényelmi forma. Production runbookban az immutable, telepített release-binárist használnd:

```
/usr/local/bin/velran-cli check /srv/velran/app/main.vrn
```

Ne keverd a kettőt egy deploymentben. A production művelet legyen egyértelműen egy release artifacthoz köthető.

20.5. Migration workflow

A migration parancs három műveletet ad:

status megmutatja az applied és pending migrationöket;
verify ellenőrzi a migration historyt és checksum-contractot;
apply lock mellett alkalmazza a pending migrationöket.

A DB URL nem parancssori plaintext secret, hanem egy egysoros secret fájl:

```
/run/secrets/velran/db-url
```

A tipikus ellenőrzési sorrend:

```
velran-cli migrate status \  
  --dir migrations \  
  --db-url-file /run/secrets/velran/db-url  
  
velran-cli migrate verify \  
  --dir migrations \  
  --db-url-file /run/secrets/velran/db-url
```

Jóváhagyott schema-változásnál:

```
velran-cli migrate apply \  
  --dir migrations \  
  --db-url-file /run/secrets/velran/db-url \  
  --lock-timeout-secs 30  
  
velran-cli migrate verify \  
  --dir migrations \  
  --db-url-file /run/secrets/velran/db-url
```

A `-lock-timeout-secs` értéke pozitív egész. Ha a migration lock nem szerezhető meg a megadott időn belül, a művelet hibával álljon meg; ne legyen párhuzamos schema-módosítás.

Az `-allow-insecure-db` nem kényelmi kapcsoló productionre

A migration CLI is ugyanazt a secure-by-default szemléletet követi, mint a server. Az `-allow-insecure-db` csak tudatos, kontrollált környezetben használható. Productionben a megfelelő TLS/DB security legyen az alapértelmezett, ne egy állandósult bypass flag.

20.6. Local-auth adminisztráció

Ha az alkalmazás local authot használ, az identity store kezelése szintén a CLI feladata. Először inicializálni kell a store-t:

```
velran-cli auth init \  
  --db-url-file /run/secrets/velran/local-auth-db-url
```

Felhasználó létrehozása:

```
velran-cli auth user-add \  
  --db-url-file /run/secrets/velran/local-auth-db-url \  
  --username <username> \  
  --password <password>
```

```
--username alice \  
--password-file /run/secrets/velran/bootstrap-password \  
--role Editor
```

Ha nincs `-role`, új usernél a CLI a `User` role-t használja. Több role több `-role` kapcsolóval adható meg.

Jelszócsere és account állapot:

```
velran-cli auth password-set \  
  --db-url-file /run/secrets/velran/local-auth-db-url \  
  --username alice \  
  --password-file /run/secrets/velran/new-password  
  
velran-cli auth disable \  
  --db-url-file /run/secrets/velran/local-auth-db-url \  
  --username alice  
  
velran-cli auth enable \  
  --db-url-file /run/secrets/velran/local-auth-db-url \  
  --username alice
```

Role-csere:

```
velran-cli auth roles-set \  
  --db-url-file /run/secrets/velran/local-auth-db-url \  
  --username alice \  
  --role Editor --role Publisher
```

A `roles-set` a megadott role-listát tekinti az új állapotnak; az operátor ne úgy kezelje, mint egyetlen role inkrementális hozzáadását.

20.7. TOTP adminisztráció különösen érzékeny művelet

TOTP enrolment:

```
velran-cli auth totp-enroll \  
  --db-url-file /run/secrets/velran/local-auth-db-url \  
  --username alice \  
  --recovery-count 8
```

A parancs kiírja a TOTP secretet, az `otpauth` URI-t és az egyszer megjelenő recovery code-okat. Emiatt ezt ne futtasd CI logban, shell transcriptben vagy közös terminálon. Az operátori folyamatnak biztosítania kell a secret és recovery code-ok azonnali, biztonságos átadását és tárolását.

TOTP kikapcsolása:

```
velran-cli auth totp-disable \  
  --db-url-file /run/secrets/velran/local-auth-db-url \  
  --username alice
```

20.8. A secret fájl is input contract

A CLI a DB URL és password fájlkat nem korlátlanul olvassa. A jelenlegi contract szerint a secret URL fájl:

- regular file;
- nem symlink;
- legfeljebb 16 KiB;
- pontosan egy nem üres sort tartalmaz.

A password file szintén regular, nem symlink fájl; legfeljebb 4096 byte, egyetlen sor, a jelszó pedig 12–1024 byte közötti.

Ezért ne process substitutionnel, symlinkkel vagy több soros secret template-tel próbáld megkerülni a contractot. A CLI boundary szándékosan szűk.

20.9. Server preflight: a CLI után a szerver saját ellenőrzése jön

A forrás és migration ellenőrzése után a production konfigurációt a szerverrel validáld:

```
velran-server \  
  --config /usr/local/etc/velran/server.toml \  
  --check-config
```

Ez beolvassa a trusted TOML-t, lefordítja az alkalmazást, ellenőrzi a route-rate/resource policy hivatkozásokat, TLS konfigurációt, valamint a deklarált storage/static rootok lokális feltételeit.

Ha a precedence-t vagy az effektív policyt kell auditálni:

```
velran-server \  
  --config /usr/local/etc/velran/server.toml \  
  --print-effective-config
```

A kimenet kifejezetten secret-redacted. Ettől még operátori információ, ezért automation logban csak indokolt retentionnel tárold.

20.10. Napi fejlesztői ciklus

Egy kis vagy közepes alkalmazásnál jó alapfolyamat:

```
edit .vrn source  
-> velran-cli check main.vrn  
-> velran-cli migrate verify  
-> if schema changed: migrate status/apply/verify on dev DB  
-> velran-server --config server.dev.toml --check-config  
-> start velran-server  
-> HTTP/form/JSON smoke tests  
-> automated application tests
```

A sorrend lényege, hogy az olcsóbb és determinisztikusabb hibákat előbb fogjuk meg. Nincs értelme HTTP smoke tesztet futtatni olyan forrásra, amely már compiler-szinten hibás.

20.11. Application release workflow

Egy production release minimális operátori folyamata:

```
immutable release selected
-> velran-cli check deployed main.vrn
-> velran-server --check-config
-> velran-cli migrate status
-> velran-cli migrate verify
-> backup/recovery gate
-> approved velran-cli migrate apply, if pending
-> velran-cli migrate verify
-> controlled service restart/switch
-> /health/live
-> /health/ready
-> application smoke test
-> logs + metrics + audit review
```

A migration **apply** az egyetlen mutáló schema-lépés ebben a folyamatban; külön approval pontként kezeld.

20.12. A Velran platform release workflowja más

Ha magát a Velran repositoryt buildeljük, a Rust workspace release-evidence is része a folyamatnak:

```
./verify.sh
cargo build --locked --release \
  -p velran-server -p velran-cli
```

A kész binárisok:

```
target/release/velran-server
target/release/velran-cli
```

Telepítés:

```
sudo install -m 0755 target/release/velran-server \
  /usr/local/bin/velran-server
sudo install -m 0755 target/release/velran-cli \
  /usr/local/bin/velran-cli
```

A saját Velran webalkalmazásod napi workflowjához nem kell minden alkalommal a nyelv teljes Rust workspace-ét verifikálni. Ott a saját source-, migration-, config- és integration tesztjeid a relevánsak.

20.13. Automatizálás: read-only és mutáló lépések legyenek külön

CI/CD-ben különítsd el a biztonságosan ismételtető ellenőrzéseket a state-et módosító műveletektől.

Read-only/preflight:

```

velran-cli check main.vrn
velran-cli migrate status \
  --dir migrations --db-url-file /run/secrets/velran/db-url
velran-cli migrate verify \
  --dir migrations --db-url-file /run/secrets/velran/db-url
velran-server --config /usr/local/etc/velran/server.toml \
  --check-config
velran-server --config /usr/local/etc/velran/server.toml \
  --print-effective-config
health/readiness GET

```

Mutáló/operator-approved:

```

velran-cli migrate apply \
  --dir migrations --db-url-file /run/secrets/velran/db-url
velran-cli auth init \
  --db-url-file /run/secrets/velran/local-auth-db-url
# A többi auth művelet ugyanilyen explicit --db-url-file
# mellett kapja a saját --username/--password-file/--role opcióit.

```

Az auth-admin parancsokat általában ne tedd automatikus application deployment pipeline-ba. Ezek identity-admin műveletek, külön audit- és hozzáférési boundaryval.

20.14. Gyakori hibák

check sikeres, a server mégsem indul A compile-time contract rendben van, de konfigurációs, TLS-, filesystem- vagy policy-startup hiba van. Futtasd a `-check-config` ellenőrzést és olvasd a server logot.

-check-config sikeres, readiness piros A lokális config helyes, de runtime dependency – például DB vagy Redis – nem érhető el vagy nem egészséges.

migrate verify hibázik Ne futtass reflexből `apply`-t. Előbb tisztázd a history/checksum eltérést.

Auth CLI nem nyitja meg a DB-t Ellenőrizd, hogy local-auth SQLite URL-t adtál-e, és a store inicializálva van-e.

Secret file hibát kap Ne lazítsd a fájlcontractot; regular, nem symlink, egysoros secretet adj.

20.15. Operátori emlékeztető

A konkrét parancsok deploymentenként eltérnek; a sorrend invariáns:

```

source -> schema -> config -> approved mutation
-> service -> health -> evidence

```

A tényleges parancsokat tartsd verziózott runbookban. Flaget, secret-file útvonalat, unit nevet vagy approval pontot ne emlékezetből rekonstruálj.

20.16. Gyakorlat: a biztonságos út legyen a legrövidebb

Gyakorlási mód: üzemeltetési scenárió. Használd az előszóban meghatározott üzemeltetési módszert.

Írd le a fejlesztő vagy operátor check, test, release verification, config validation és startup parancsait. Távolítsd el a dokumentálatlan shortcutokat, és preferáld azokat a parancsokat, amelyek fail-closed módon reagálnak stale lockfile-ra, policy-ra vagy release evidence-re.

20.17. Tipikus hiba: megjegyzett parancssorokra támaszkodunk

Ha egy biztonságos release öt opcionális flag pontos felidézésétől függ, a workflow törékeny. Az invariánsokat őrző parancssorokat tedd verziózott scriptbe, és legyen az output review szempontból érthető.

20.18. Operátori checkpoint: Secure Notes

Készíts egyoldalas command runbookot a Secure Notes check, build, verify és start lépéseire. Egy új operátor tudja követni anélkül, hogy emlékezetből kellene tudnia, melyik lépés security-critical.

21. fejezet

A `server.toml` konfigurációs referencia

Tanulási út: Teljes út / referencia. Server konfigurációkor keresd vissza; lineáris tananyagként nem kötelező.

Első olvasás. A precedence/fail-closed szabályt, a production baseline-t és az egyes beállítások trust domainjét tanuld meg. Az option táblákat deployment referenciaként használd.

A korábbi fejezetek feladatorientáltan használták a szerverkonfigurációt. Ez a fejezet egy helyen adja meg a V1 production konfiguráció teljes, operátori nézetét. A referencia alapja a tényleges parser és a `config/server.toml.sample`; ha a kettő között eltérés lenne, a parser viselkedése az irányadó.

21.1. Betöltés, precedence és fail-closed szabályok

Az ajánlott production indítás:

```
velran-server \  
  --config /usr/local/etc/velran/server.toml
```

A precedence:

```
built-in defaults < server.toml < explicit CLI override
```

A TOML trusted konfiguráció, de a parser akkor is fail-closed:

- ismeretlen section vagy kulcs: startup error;
- duplikált kulcs vagy tábla: TOML parse error;
- a config fájl csak reguláris, nem symlink fájl lehet;
- maximum mérete 1 MiB;
- TOML-ból érkező filesystem path abszolút kell legyen;
- secret nem kerülhet plaintext mezőbe: DB/Redis/LDAP/local-auth credential csak `*_file` hivatkozással adható meg.

Ellenőrzés listener indítása nélkül:

```
velran-server \  
  --config /usr/local/etc/velran/server.toml
```

```
--config /usr/local/etc/velran/server.toml \
--check-config
```

Secretmentes effective nézet:

```
velran-server \
--config /usr/local/etc/velran/server.toml \
--print-effective-config
```

21.2. Hogyan olvasd a táblákat?

A **beépített default** azt jelenti, ami config nélkül érvényes. A **production minta** a projekt `config/server.toml.sample` ajánlott kiinduló értéke. A kettő nem mindig azonos: a sample több helyen tudatosan nagyobb, productionre szánt keretet ad.

21.3. [server] és [tls]

Kulcs	Beépített default	Jelentés / production megjegyzés
<code>server.app</code>	nincs	Az alkalmazás abszolút <code>main.vrn</code> útvonala. Sample: <code>/srv/velran/current/main.vrn</code> .
<code>server.listen</code>	127.0.0.1:8080	Listener. Reverse proxy mögött tipikusan loopback; közvetlen TLS esetén publikus cím is lehet.
<code>server.insecure_dev_cookies</code>	false	Csak fejlesztési HTTP-hoz. Productionben maradjon false .
<code>server.unix_socket</code>	nincs	Opcionális abszolút Unix-domain socket útvonal; csak Unix alatt és server.behind_proxy=true mellett.
<code>server.behind_proxy</code>	false	Explicit trusted reverse-proxy szemantikát kapcsol be. Backend TLS-sel nem kombinálható.
<code>server.expected_build_id</code>	nincs	Opcionális 64 kisbetűs hex deployment-pin. Eltérő futó build ID esetén a startup fail-closed.
<code>tls.cert_file</code>	nincs	TLS certificate chain abszolút útvonala.
<code>tls.key_file</code>	nincs	TLS private key abszolút útvonala.
<code>tls.handshake_timeout_ms</code>	5000	TLS handshake felső ideje.
<code>tls.http_redirect_listen</code>	nincs	Opcionális HTTP listener HTTPS redirecthez.
<code>tls.public_host</code>	nincs	Canonical/public host a redirect és Host/Origin-validációhoz; trusted HTTPS reverse proxy módban is szükséges.

Reverse proxy mögött

Apache vagy Nginx mögött a Velran tipikusan nem publikus interfészen hallgat:

```
[server]
app = "/srv/velran/current/main.vrn"
listen = "127.0.0.1:8080"
insecure_dev_cookies = false

[tls]
public_host = "www.example.com"
```

A publikus TLS-t ilyenkor a proxy terminálja. A Velran oldali certificate/key elmarad, de a `tls.public_host` továbbra is kötelező trusted Host/Origin pinninghez. A `web.trusted_proxy_cidrs` határozza meg, mely upstreamtől fogadunk el forwarded client/proto metadata-t. TLS nélküli production listener ebben a módban csak loopback címen engedett.

21.4. [database], [redis] és [auth]

Kulcs	Default	Jelentés
<code>database.url_file</code>	nincs	DB URL-t tartalmazó secret fájl. Plaintext <code>database.url</code> nincs.
<code>database.allow_insecure</code>	false	Titkosítatlan remote DB kapcsolat explicit engedélyezése; productionben általában ne használd.
<code>redis.url_file</code>	nincs	Redis URL secret fájlból.
<code>redis.allow_insecure</code>	false	Insecure Redis explicit engedélyezése.
<code>auth.ldap_url</code>	nincs	LDAP/LDAPS endpoint.
<code>auth.ldap_search_base</code>	nincs	LDAP search base.
<code>auth.ldap_username_attribute</code>	uid	Login név LDAP attribútuma.
<code>auth.ldap_service_bind_dn_file</code>	nincs	Service bind DN secret fájlból.
<code>auth.ldap_service_bind_password_file</code>	nincs	Bind jelszó secret fájlból.
<code>auth.totp_secrets_file</code>	nincs	TOTP secret store fájl.
<code>auth.roles_file</code>	nincs	Külső role mapping fájl.
<code>auth.memberships_file</code>	nincs	Opcionális tenant/user membership authority fájl; abszolút útvonalú trusted local policy input.
<code>auth.local_auth_db_url_file</code>	nincs	Local-auth identity store URL secret fájlból.
<code>auth.require_totp</code>	false	Sikeres első faktor után TOTP kötelező-e.
<code>auth.login_max_attempts</code>	8	Maximum jelszóhiba egy source+principal párra ablakonként.
<code>auth.login_principal_max_attempts</code>	16	Maximum jelszóhiba egy principalra több source-ból.
<code>auth.login_source_max_attempts</code>	64	Maximum authentication hiba egy source-ból több principalra.

<code>auth.mfa_max_attempts</code>	5	Maximum második faktor hiba source+principal/principal ablakonként.
<code>auth.login_window_secs</code>	300	Authentication abuse-control ablak másodpercben.

Az LDAP és a local auth ne legyen két párhuzamos elsődleges azonosítási forrás ugyanabban a deploymentben. Az identity authority legyen egyértelmű; a Redis session-, cache- és rate-limit runtime state, nem felhasználói source of truth.

21.5. [web]

Kulcs	Default	Jelentés
<code>web.trusted_proxy_cidrs</code>	üres lista	Csak ezekről a címekről hiteles a forwarded client/proto metadata.
<code>web.allow_missing_origin</code>	false	Origin nélküli unsafe requestek policyja. Productionben fail-closed alapérték.
<code>web.cors_origins</code>	üres lista	Exact-origin allowlist. Wildcard credentials CORS-hoz nem használható.
<code>web.cors_allow_credentials</code>	false	Session cookie/credential cross-origin engedélyezése; CSRF policyval együtt értelmezendő.
<code>web.webhook_secrets_dir</code>	nincs	Verified webhook secreteket tartalmazó abszolút, nem symlink könyvtár. Verified webhook route esetén kötelező.
<code>web.egress_policy_file</code>	nincs	Abszolút outbound integration/egress policy fájl. Outbound integration effectet deklaráló alkalmazásnál kötelező.

21.6. [storage] és [static_assets]

Kulcs	Beépített default	Jelentés
<code>storage.data_root</code>	nincs	AppFs runtime data gyökér. Sample: <code>/srv/velran/data</code> .
<code>storage.fs_mode</code>	rwc	AppFs capability mód. Csak a szükséges jog legyen engedélyezve.
<code>storage.max_upload_bytes</code>	16 MiB	Egy upload felső byte-limitje.
<code>storage.max_image_pixels</code>	40 000 000	Dekódolt kép pixel ceilingje.
<code>static_assets.root</code>	nincs	Read-only static root. Sample: <code>/srv/velran/current/public</code> .
<code>static_assets.url_prefix</code>	/assets/	A static namespace URL prefixe.
<code>static_assets.max_asset_bytes</code>	8 MiB	Egy static asset felső mérete.
<code>static_assets.max_age_secs</code>	300	Nem fingerprintelt asset cache ideje.

<code>static_assets.immutable_</code>	31536000	Fingerprintelt immutable asset
<code>max_age_secs</code>		cache ideje.
<code>static_assets.precompressed</code>	<code>true</code>	Előre generált Brotli/gzip változatok használata.

A `data_root` és a `static root` ne legyen ugyanaz a könyvtár: az egyik mutable alkalmazásállapot, a másik release-owned read-only artifact.

21.7. [lifecycle], [observability] és [logging]

Kulcs	Default	Jelentés
<code>lifecycle.health_live_path</code>	<code>/health/live</code>	Process liveness endpoint.
<code>lifecycle.health_ready_path</code>	<code>/health/ready</code>	Dependency-aware readiness endpoint.
<code>lifecycle.health_dependency_timeout_ms</code>	1000	Readiness dependency probe timeout.
<code>lifecycle.shutdown_grace_ms</code>	30000	Graceful shutdown drain idő.
<code>observability.metrics_listen</code>	nincs	Prometheus listener; sample: <code>127.0.0.1:9090</code> .
<code>observability.allow_public_metrics</code>	<code>false</code>	Publikus metrics listener explicit engedélye.
<code>observability.access_log</code>	<code>true</code>	HTTP access események logolása.
<code>logging.server_file</code>	nincs	Error/system log fájl.
<code>logging.access_file</code>	nincs	Access log fájl.
<code>logging.audit_file</code>	nincs	Security/user-activity audit log.
<code>logging.stderr</code>	<code>true</code>	Stderr log output.

A log parent directoryt az operátor hozza létre. Létező log target nem lehet symlink vagy nem-reguláris fájl. Behind-proxy módban a SIGHUP újranítja a logokat és tranzakciósan újraépíti a domain/application hosting állapotot; a process-szintű beállításokhoz továbbra is restart kell.

21.8. [rate_limit] és [cache]

Kulcs	Default	Jelentés
<code>rate_limit.policies_file</code>	nincs	Route rate-limit policy fájl. Sample: <code>/usr/local/etc/velran/rate-limits.toml</code> .
<code>rate_limit.allow_memory</code>	<code>false</code>	In-memory fallback explicit engedélyezése. Több instance productionnél maradjon <code>false</code> .
<code>cache.max_ttl_secs</code>	3600	Public cache maximális route TTL-je.
<code>cache.max_entries</code>	10000	In-memory cache entry ceiling.
<code>cache.max_bytes</code>	64 MiB	In-memory cache teljes byte ceiling.
<code>cache.allow_memory</code>	<code>false</code>	Redis nélküli memory cache explicit engedélyezése.

<code>cache.singleflight_wait_timeout_ms</code>	5000	Coalesced public-cache fill várakozóinak maximális ideje; nulla érték tiltott.
---	------	--

Ha több `velran-server` példány fut, a `session/cache/rate-limit` közös állapotát Redisnek kell hordoznia; a `memory fallback` csak tudatos, tipikusan egy-instance környezetben elfogadható.

21.9. [native]

A natív végrehajtás a `production execution path`; a konfiguráció a `trusted compiler-invokációt` és `cache-t` szabályozza, nem ad tetszőleges `rustc` flaget az alkalmazáskód kezébe.

Kulcs	Default	Jelentés
<code>native.enabled</code>	<code>true</code>	Natív végrehajtás engedélyezése. A jelenlegi <code>application activation</code> a natív utat igényli.
<code>native.required</code>	<code>false</code>	Startup/activation hiba, ha egyetlen natív shard sem fordítható és tölthető be. A <code>production sample</code> <code>true</code> .
<code>native.rustc</code>	<code>discovery</code>	Opcionális abszolút <code>trusted rustc</code> útvonal; egyébként a konfigurált <code>environment/PATH</code> <code>discovery</code> érvényes.
<code>native.cache_dir</code>	<code>app-local</code>	Opcionális abszolút natív artifact cache könyvtár. Cache hitnél a <code>hash/integrity</code> ellenőrzés megmarad.
<code>native.optimization</code>	<code>release</code>	<code>interactive</code> vagy <code>release</code> ; <code>productionben</code> <code>release</code> .
<code>native.target_cpu</code>	nincs	Opcionális validált CPU baseline, például <code>host-local</code> cache esetén <code>native</code> .
<code>native.cpu_features</code>	nincs	Opcionális validált feature-lista, például <code>+sse2,+avx2</code> ; csak ismert <code>deployment-kompatibilitás</code> mellett pineld.
<code>native.debug_rustc_repro</code>	<code>false</code>	Diagnosztikai reprodukciós segéd; normál <code>production</code> beállításként ne engedélyezd.

Security szabály: a `performance-beállítás` módosíthatja a kódgenerálást, de nem kerülheti meg a `verified loweringot`, `package/capability` ellenőrzést vagy a `native-cache` `integrity` verifikációját.

21.10. [reload]

Kulcs	Default	Jelentés
-------	---------	----------

<code>reload.enabled</code>	<code>true</code>	A shared source-graph watcher engedélyezése.
<code>reload.poll_interval_ms</code>	1000	Metadata polling intervallum.
<code>reload.debounce_ms</code>	250	Candidate generáció fordítása előtti debounce.
<code>reload.debug_compile_errors</code>	<code>false</code>	Fejlesztői compiler-diagnosztika, miközben az utolsó valid generáció tovább szolgál. Production security policy ezt a módot elutasítja.

21.11. [limits]

A request- és runtime-limitek defense-in-depth szerepet töltenek be. A sample több értéket explicit megad, hogy az operator auditálható módon vállalja a production budgetet.

Kulcs	Beépített default	Sample / jelentés
<code>limits.max_header_bytes</code>	16 KiB	Sample ugyanennyi.
<code>limits.max_body_bytes</code>	256 KiB	Sample ugyanennyi; upload külön storage limitet is kap.
<code>limits.max_connections</code>	4096	Egyidejű kapcsolatok ceilingje.
<code>limits.max_requests_per_connection</code>	100	Keep-alive request ceiling.
<code>limits.read_timeout_ms</code>	5000	Request read timeout.
<code>limits.request_timeout_ms</code>	15000	Teljes request budget.
<code>limits.write_timeout_ms</code>	5000	Response write timeout.
<code>limits.max_header_count</code>	64	Header count ceiling.
<code>limits.max_form_fields</code>	64	Form mezők maximuma.
<code>limits.max_form_field_bytes</code>	8 KiB	Egy form mező byte ceiling.
<code>limits.max_json_depth</code>	32	A bounded JSON boundary maximális nesting mélysége.
<code>limits.max_json_string_bytes</code>	64 KiB	Egy dekódolt JSON string maximális byte-mérete.
<code>limits.max_json_array_items</code>	4096	Egy JSON array maximális elemszáma.
<code>limits.max_json_object_fields</code>	1024	Egy JSON object maximális mezőszáma.
<code>limits.max_instructions</code>	100000	Sample: 5000000. Verified execution/instruction budget; emelés auditálandó.
<code>limits.max_runtime_alloc_bytes</code>	32 MiB	Sample: 256 MiB. Runtime allocation ceiling.
<code>limits.session_ttl_secs</code>	28800	8 óra.
<code>limits.max_sessions</code>	100000	Session ceiling.
<code>limits.max_process_memory_bytes</code>	nincs	Sample: 1 GiB; process address-space limit.
<code>limits.resource_profiles_file</code>	nincs	Sample: <code>/usr/local/etc/velran/resource-profiles.toml</code> .

Miért külön a request, runtime és process limit?

A `max_body_bytes` a hálózati inputot korlátozza, a `max_runtime_alloc_bytes` egy request runtime által elszámolt allokációját, a `max_process_memory_bytes` pedig a teljes process address space-t. Egyik nem helyettesíti a másikat.

21.12. [cgroup]

A cgroup korlátok Linux production defense-in-depth réteget adnak a process-szintű limitek fölé.

Kulcs	Default	Jelentés
<code>cgroup.dir</code>	nincs	A már előkészített cgroup könyvtár abszolút útvonala.
<code>cgroup.memory_max_bytes</code>	nincs	Cgroup memória ceiling; a systemd sample ezt nem állítja.
<code>cgroup.memory_swap_max_bytes</code>	nincs	Swap ceiling; delegated cgroup deploymentnél opcionális.
<code>cgroup.cpu_percent</code>	nincs	CPU budget százalékban; delegated cgroup deploymentnél opcionális.
<code>cgroup.pids_max</code>	nincs	PID/thread ceiling; delegated cgroup deploymentnél opcionális.

A cgroup könyvtár és delegáció operátori feladat; a szerver nem rendszeradminisztrációs eszköz. A repository systemd baseline-jában a service manager a cgroup authority (`MemoryMax`, `CPUQuota`, `TasksMax`), ezért ott a `[cgroup]` section szándékosan nincs bekapcsolva. Velran-oldali cgroup írást csak külön delegált, írható cgroup v2 könyvtárnál használj; a két authorityt ne konfiguráld egyszerre.

21.13. Production baseline

Egy tipikus reverse-proxy mögötti minimum konfiguráció:

```
[server]
app = "/srv/velran/current/main.vrn"
listen = "127.0.0.1:8080"
insecure_dev_cookies = false

[tls]
public_host = "www.example.com"

[database]
url_file = "/run/secrets/velran/database-url"

[redis]
url_file = "/run/secrets/velran/redis-url"

[web]
trusted_proxy_cidrs = ["127.0.0.1/32", "::1/128"]
allow_missing_origin = false

[storage]
data_root = "/srv/velran/data"
```

```
[logging]
server_file = "/var/log/velran/server.log"
access_file = "/var/log/velran/access.log"
audit_file = "/var/log/velran/audit.log"
stderr = true

[limits]
resource_profiles_file = "/usr/local/etc/velran/resource-profiles.toml"
```

Ez csak baseline: auth, static, metrics, rate-limit, TLS és cgroup a deployment topológiájához igazítandó.

21.14. Automatikus source reload

A globális [reload] section az `enabled`, `poll_interval_ms`, `debounce_ms` és a fejlesztői `debug_compile_errors` alapértékeket adja. Multi-domain módban ezek [domains.reload] alatt domainenként felülírhatók. Egyetlen közös supervisor figyeli a compiler által visszaadott source graphot `mtime + size` alapján, beleértve az atomikus symlink deploymenthez szükséges logikai entrypoint útvonalat is. Sikertelen candidate esetén a régi generáció marad aktív; sikeres commit csak az érintett domaint cseréli, és invalidálja annak public-cache generationjeit. Karbantartáskor a `-no-source-reload` kapcsoló processz-szinten tiltja a figyelést.

21.15. Változtatási workflow

Production config módosításnál a könyv egyetlen ajánlott sorrendet használ:

1. módosítsd a version-controlled config template-et;
2. secretet külön file-ban módosíts, ne TOML-ban;
3. futtasd a `velran-server -check-config` ellenőrzést;
4. nézd meg a `-print-effective-config` secretmentes összefoglalót;
5. controlled restart;
6. ellenőrizd a live/ready endpointokat, logot, metricet és auditot.

Behind-proxy módban a SIGHUP csak korlátozott hosting reloadot végez. Nem általános folyamat-újrakonfigurálás. Az alkalmazásforrás változásához normál esetben sem SIGHUP, sem restart nem kell. A reload supervisor az entrypointot és a tranzitív modulgráfot figyeli. Csak sikeresen forduló domain-generációt cserél be tranzakción.

21.16. Gyakorlat: trust domain szerint review-zd a konfigurációt

Gyakorlási mód: üzemeltetési scenárió. Használd az előszóban meghatározott üzemeltetési módszert.

Csoportosítsd a konfigurációt listener/TLS, storage, authentication/session, resource limit, egress, observability és native build/runtime területre. Minden csoportot azzal review-ztass, aki ismeri a failure mode-ját, ne egyetlen lapos kulcslistaként kezeld a fájlt.

21.17. Tipikus hiba: nagy konfigurációt másolunk és csak a nyilvánvaló értékeket cseréljük

Egy másolt setting proxy-, path-, limit- vagy credential-feltételezést hordozhat, amely nem illik az új környezethez. A dokumentált minimumból indulj, és csak olyan beállítást adj hozzá, amelynek operátori jelentése ismert.

21.18. Operátori checkpoint: Secure Notes

Készítsd el a Secure Notes minimális production konfigurációját, és jelöld, mely érték környezetfüggő. A secretok referenced file-ban maradjanak, a végső konfigurációt pedig deployment előtt ellenőrizd a szerver check módjával.

22. fejezet

Fejlesztői referencia és továbblépés

Tanulási út: Teljes út / referenciatérkép. Ebből keresd meg egy nyelvi, security vagy operátori kérdés authoritative forrását.

22.1. Canonical dokumentációs térkép

Ez a könyv a hosszú formájú fejlesztői útmutató. A jelenlegi high-level security státusz a `SECURITY-STATUS.md`; a részletes canonical témadokumentáció a `docs/` alatt él. Ahol van magyar operator/fejlesztő companion, az a `docs/hu/` alatt található.

Téma	Canonical projekt-dokumentáció
Aktuális security architecture	<code>SECURITY-STATUS.md</code> , <code>docs/99-security-architecture-and-status.md</code>
Secure-by-construction alap	<code>docs/57-secure-by-construction-foundation.md</code>
Domain típusok és bounded collectionök	<code>docs/62-domain-types-and-bounded-collections.md</code>
Nominal type és refinement	<code>docs/63-nominal-domain-types.md</code> , <code>docs/64-domain-value-refinement.md</code>
Public projection és secret flow	<code>docs/65-explicit-public-projections.md</code> , <code>docs/66-secret-consumer-contracts.md</code>
Password, token és session	<code>docs/67-typed-credentials-and-passwords.md</code> - <code>docs/74-platform-owned-browser-sessions.md</code>
Permission, MFA, critical operation	<code>docs/75-function-level-permissions.md</code> - <code>docs/77-critical-operation-contracts.md</code>
Resource profile és bounded request input	<code>docs/78-route-budget-profiles.md</code> , <code>docs/79-bounded-request-collections.md</code>
Public error boundary	<code>docs/80-public-error-boundaries.md</code>
Tenant authority és isolation	<code>docs/81-tenant-membership-authority.md</code> , <code>docs/82-compiler-enforced-tenant-isolation.md</code>
Idempotens critical operation	<code>docs/83-idempotent-critical-operations.md</code>
Verified webhook	<code>docs/81-verified-webhook-integrity.md</code>
Safe upload state machine	<code>docs/84-safe-file-upload-state-machine.md</code>
Effect/capability modell	<code>docs/85-effect-capability-foundation.md</code>
Typed outbound integration	<code>docs/86-typed-outbound-integration-calls.md</code>
Production security policy	<code>docs/87-production-configuration-policy.md</code>
Typed HTTP metadata	<code>docs/88-typed-http-metadata.md</code>

Crypto key purpose/lifecycle	docs/89-crypto-key-purpose-lifecycle.md
Generált CSP/security header	docs/90-generated-security-headers.md
Supply-chain capability/provenance	docs/92-supply-chain-capability-provenance.md
Authentication abuse protection	docs/93-authentication-abuse-protection.md
Zárt sum type/exhaustive match	docs/94-sum-types-and-exhaustive-match.md
Transaction outcome semantics	docs/95-transaction-outcome-semantics.md
Authenticated encryption/key lifecycle	docs/96-authenticated-encryption-key-lifecycle.md
Advanced resource safety	docs/97-advanced-resource-safety.md
Typed security event	docs/79-typed-security-events.md
Bounded JSON boundary	docs/100-bounded-json-boundary.md
Security monitoring/redaction	docs/98-security-monitoring-and-redaction.md
Dependency security	docs/19-dependency-security.md
Modulok/névterek	docs/21-modules-and-namespaces.md
Matematika/idő, string/list/dict/regex	docs/44-math-and-timing.md, docs/46-string-builtins.md - docs/49-regular-expressions.md
Optimistic locking	docs/24-optimistic-locking.md
IPv6-ready egress	docs/32-ipv6-egress.md
Debian package/install	docs/36-debian-package.md
Reverse proxy/multidomain/reload	docs/37-multi-domain-hosting.md - docs/39-automatic-source-reload.md
Maintainability/architecture gate	docs/50-maintainability-and-clean-code.md

22.2. Napi secure fejlesztési checklist

1. Authority legyen explicit: route policy, DB capability, tenant, permission/MFA és named outbound integration.
2. Untrusted inputot nominal/domain típusra refine-olj; validationt ne keverd authorizationnel.
3. Protected write előtt object/mutation authorization proof kell.
4. Permission/MFA/transaction/audit/idempotency igénynél használj critical contractot.
5. Retry-szenzitív transactionnél kezeld exhaustívan a `Committed`, `RolledBack`, `CommitUnknown` állapotot.
6. Ne vezess vissza raw SQL/HTML, arbitrary redirect/header/filename vagy arbitrary outbound URL kerülőutat.
7. Upload maradjon staged/private inspectionig és explicit publishig.
8. Secret maradjon typed secret/credential/key boundary mögött; monitoringhoz `redact(...)`.
9. DB row, collection, file, outbound body és request idő legyen bounded.
10. Platform release előtt locked Cargo metadata/check/test, `./verify.sh`, supply-chain és release-manifest verification.

22.3. Hogyan használd a dokumentációs készletet?

A kérdéshez illő dokumentációs réteget használd:

Dokumentumosztály	Mire jó?	Authority
Ez a könyv	tanulási út és end-to-end fejlesztési workflow	magyarázó, a canonical docshoz igazítva
<code>docs/*.md</code>	feature-contractok, invariantok és fókuszált példák	canonical témadokumentáció
<code>docs/hu/*.md</code>	magyar companion napi fejlesztéshez/üzemeltetéshez	fordított companion, ahol létezik
<code>SECURITY-STATUS.md</code>	aktuális high-level security architecture/status	jelenlegi architecture summary
<code>RUST_FIRST_LANGUAGE_POLICY.md</code>	nyelvi/backend irány és native execution boundary	jelenlegi language policy
<code>history/development-notes/</code>	régi iterációk, kísérletek és leváltott design-context	nem normatív történet

Ha a források eltérnek, az aktuális canonical feature-docot és a verified compiler/runtime viselkedést preferáld, majd frissítsd a stale szöveget. A történeti note design-előzményt magyaráz, nem aktuális szintaxis- vagy deployment utasítás.

22.4. Feladatorientált olvasási útvonalak

Gyakori munkáknál gyorsabb ez a sorrend, mint a repository numerikus végigolvasása:

- **CRUD alkalmazás:** 3–7. fejezet, majd form, optimistic locking és domain-type dokumentumok.
- **JSON/API service:** 9. fejezet, bounded JSON, typed HTTP metadata és public error boundary.
- **Authentication-sensitive mutáció:** 7. fejezet, function permission, MFA elevation, critical-operation contract és idempotency.
- **Fájl/kép fogadása:** 8. fejezet és a safe upload state-machine dokumentum.
- **Külső service hívása:** 13. fejezet, secure routing/egress és typed outbound integration dokumentumok.
- **Production előkészítés:** 15–23. fejezet, production configuration policy, supply-chain provenance, recovery és production checklist.
- **Security invariant vizsgálata:** indulj a `SECURITY-STATUS.md`-ből, majd kövesd az adott invariant nevét viselő fókuszált dokumentumot.

22.5. Futtatható companion példák

A könyv szándékosan tartalmaz teljes programokat és fókuszált kódrészleteket is. Ha egy példát másolni, módosítani vagy regressziós fixture-ként használni szeretnél, indulj olyan repository-példából, amely részt vesz a projekt verificationben:

Feladat	Verified companion	Mit érdemes tanulmányozni?
Kis moduláris alkalmazás	<code>examples/starter-project/</code>	modulok, typed page/action, CSRF és typed route helper
CRUD és typed SQL	<code>examples/crud/app.vrn</code>	<code>Vec<T></code> , <code>Option<T></code> , transaction, validation és object authorization

Authentication policy	<code>examples/auth/app.vrn</code>	authenticated route policy és session-aware alkalmazásfolyam
Fájl és upload	<code>examples/upload/app.vrn</code>	typed upload boundary és biztonságos fájlkezelés
JSON/API	<code>examples/json-api/app.vrn</code>	typed JSON request/response és API route-viselkedés
Component/layout	<code>examples/components/app.vrn</code>	komponensek, layoutok és typed HTML composition
Wiki/CMS	<code>examples/wiki/main.vrn</code>	slugolt tartalom, Markdown renderelés, layout és mutációs flow
SQL-alapú Markdown	<code>examples/markdown-sql-cache/</code>	typed id lookup, biztonságos szerveroldali Markdown renderelés és opcionális public response cache
Optimistic locking	<code>examples/optimistic-locking/main.vrn</code>	verziózott update és conflict handling
Outbound integration	<code>examples/outbound-native/app.vrn</code>	named, typed outbound hívás és egress boundary
Security event	<code>examples/typed-security-events/</code>	typed security-event emission és monitoring boundary

A könyv rövid listingje szándékosan elhagyhat a tárgyalt fogalomhoz nem szükséges deklarációkat. Az ilyen fragment magyarázó példa, nem állítás arról, hogy önmagában teljes program. Copy/paste kiindulópontként a fenti companion fájlokat használd. A csak verifikációra szolgáló rejection esetek a `tests/fixtures/` alatt vannak, nem a tanulói példakatalógusban; a corpus metaadatai a `tests/manifests/` alatt élnek. A `tools/check-book-velran-examples.sh` pedig védi a könyvet a visszacsúszó legacy szintaxistól, a hiányzó route-policytől, a nem létező példaútvonaltól és az angol/magyar Velran-listing driftjétől.

22.6. A példák hitelességének megőrzése

A dokumentációs példák production-source szabályokat kövessenek: aktuális Rust-like szintaxis, explicit route policy, typed/bounded input, compiler-owned SQL, raw HTML/redirect/egress shortcut nélkül és native-only executionnel. Ha van megfelelő verified `examples/` fixture, azt preferáld. A módosítás csak language-corpus review és EN/HU szerkezeti paritás után kész.

22.7. Gyakorlat: válaszd ki a megfelelő authority-t

Gyakorlási mód: üzemeltetési scenárió. Használd az előszóban meghatározott üzemeltetési módszert.

Egy nyelvi kérdésnél dönts el, hogy a válasz a compiler által ellenőrzött nyelvi felülethez, a web framework contracthoz, deployment dokumentációhoz vagy történeti jegyzethez tartozik. Először a leginkább authoritative aktuális forrást használd, a fordított vagy történeti anyagot pedig a megjelölt szerepe szerint kezeld.

22.8. Tipikus hiba: dokumentációs konfliktust találgatással oldunk fel

Ha két dokumentum eltér, ne kombináld őket csendben. Preferáld a canonical aktuális specifikációt vagy verified viselkedést, majd frissítsd a stale dokumentumot, hogy a konfliktus ne maradjon meg a következő olvasónak.

22.9. Olvasói checkpoint: Secure Notes

Válassz egy Secure Notes tervezési döntést, és keresd meg az azt alátámasztó aktuális authoritative dokumentációt. Ez gyakorlati tesztje annak, hogy a dokumentációs térkép használható, nem csak teljes.

A. függelék

Függelékek és production checklist

A.1. Aktuális verifikációs megjegyzés

A jelenlegi Velran forrásfa teljesítette a repository-szintű Ellenőrzött fejlesztési mérföldkövet: a formázás, a teljes workspace tesztkészlet, a `./verify.sh` és a helyi tesztkiszolgálás sikeresen lefutott. Production release előtt az egzakt release-forrásfán ismét futtasd a lock/provenance frissítést és a repository teljes verifikációját, majd gyűjtsd össze a célkörnyezet deployment-, recovery- és operátori bizonyítékait. A repository verifikáció szükséges, de nem helyettesíti a production-környezeti release evidence-et.

Tanulási út: Gyors út: végső release gate. A checklisttel bizonyítsd a release readiness állapotot, ne emlékezetből feltételezd.

Ez a fejezet nem új nyelvi elemeket vezet be. Az előző fejezetekből összeálló, röviden használható üzemeltetési segédlet: tipikus könyvtárstruktúra, reverse proxy minták, systemd unit, logrotate és egy release előtti ellenőrzőlista. A minták kiindulópontok; a konkrét domaineket, tanúsítványokat, jogosultságokat és erőforráslimiteket a saját környezethez kell igazítani.

A.2. Ajánlott production könyvtárstruktúra

A Velran helyben telepített szoftver, ezért a könyv következetesen a `/usr/local` hierarchiát használja a programhoz és a konfigurációhoz. Az alkalmazás állapota és a logok ettől külön élnek.

```
/usr/local/bin/  
  velran-server  
  velran-cli  
  
/usr/local/etc/velran/  
  server.toml  
  rate-limits.toml  
  resource-profiles.toml  
  
/run/secrets/velran/  
  database-url  
  redis-url  
  local-auth-database-url  
  
/srv/velran/
```

```

current/                # current application release
releases/
  2026-09-04.1/
data/                   # AppFs / durable application data

/var/log/velran/
  server.log
  access.log
  audit.log

```

A release tartalom legyen lehetőleg read-only. A szervernek csak azokon a helyeken legyen írási joga, ahol valóban tartós állapotot vagy logot kell írnia. A secret fájl ne legyen az alkalmazás release könyvtárában és ne kerüljön verziókezelésbe.

A.3. Apache reverse proxy teljes minimálminta

A példa egyetlen Apache reverse proxyt feltételez. A publikus TLS az Apache feladata, a Velran pedig csak loopbacken figyel.

Az ehhez tartozó Velran minimum:

```

[server]
listen = "127.0.0.1:8080"
insecure_dev_cookies = false

[tls]
public_host = "www.example.com"

[web]
trusted_proxy_cidrs = ["127.0.0.1/32", "::1/128"]

```

Reverse proxy production módban a certificate/key a proxyn marad, de a trusted public_host kell a Host/Origin pinninghez. A proxy feladata a hiteles X-Forwarded-Proto: https metadata előállítás.

```

<VirtualHost *:80>
  ServerName www.example.com
  Redirect permanent / https://www.example.com/
</VirtualHost>

<VirtualHost *:443>
  ServerName www.example.com

  SSLEngine on
  SSLCertificateFile \
    /etc/letsencrypt/live/www.example.com/fullchain.pem
  SSLCertificateKeyFile \
    /etc/letsencrypt/live/www.example.com/privkey.pem

  ProxyPreserveHost On
  ProxyPass          / http://127.0.0.1:8080/
  ProxyPassReverse   / http://127.0.0.1:8080/

  RequestHeader set X-Forwarded-Proto "https"
</VirtualHost>

```

Tipikus Apache modulok: `proxy`, `proxy_http`, `headers` és `ssl`. A Velran trusted-proxy listája csak a tényleges proxyt foglalja magában. Loopback proxy esetén ez tipikusan `127.0.0.1/32` és szükség esetén `::1/128`. Internetméretű trusted proxy tartomány hibás konfiguráció.

A.4. Nginx reverse proxy teljes minimálminta

```
server {
    listen 80;
    server_name www.example.com;
    return 301 https://$host$request_uri;
}

server {
    listen 443 ssl;
    server_name www.example.com;

    ssl_certificate
        /etc/letsencrypt/live/www.example.com/fullchain.pem;
    ssl_certificate_key
        /etc/letsencrypt/live/www.example.com/privkey.pem;

    location / {
        proxy_pass http://127.0.0.1:8080;
        proxy_http_version 1.1;
        proxy_set_header Host $host;
        proxy_set_header X-Forwarded-Proto https;
        proxy_set_header X-Forwarded-For $remote_addr;
    }
}
```

A példában az Nginx közvetlenül látja a klienst, ezért a forwarding címet maga állítja elő. Ne örökíts tovább internet felől érkező tetszőleges `X-Forwarded-*` értéket authorityként. Több proxy esetén a trust láncot explicit kell megtervezni.

A.5. systemd service minta

A repository teljes mintája: `examples/systemd/velran.service`. A lényegi forma:

```
[Unit]
Description=Velran application
Wants=network-online.target
After=network-online.target

[Service]
Type=simple
User=velran
Group=velran
WorkingDirectory=/srv/velran/current
ExecStartPre=/usr/local/bin/velran-server \
    --config /usr/local/etc/velran/server.toml \
    --check-config
ExecStart=/usr/local/bin/velran-server \
    --config /usr/local/etc/velran/server.toml
ExecReload=/bin/kill -HUP $MAINPID
```

```

Restart=on-failure
RestartSec=2s
TimeoutStopSec=45s

MemoryMax=1G
MemorySwapMax=0
CPUQuota=200%
TasksMax=256
LimitNOFILE=8192
# Direct TLS 80/443 eseten add hozzá:
# AmbientCapabilities=CAP_NET_BIND_SERVICE
# CapabilityBoundingSet=CAP_NET_BIND_SERVICE
NoNewPrivileges=yes
PrivateTmp=yes
PrivateDevices=yes
ProtectSystem=strict
ProtectHome=yes
ProtectKernelTunables=yes
ProtectKernelModules=yes
ProtectControlGroups=yes
RestrictSUIDSGID=yes
LockPersonality=yes
ReadWritePaths=/srv/velran/data /var/log/velran

[Install]
WantedBy=multi-user.target

```

A systemd hard limit és a Velran request/runtime resource policyja egymást kiegészítő védelmi réteg. A systemd baseline maga legyen a process cgroup authority; ilyenkor ne kapcsolod be mellette a Velran [cgroup] író módját. Közvetlen 80/443 bindnál csak a CAP_NET_BIND_SERVICE capabilityt add, reverse proxy mögötti magas portnál ezt is hagyd el. A TimeoutStopSec legyen nagyobb, mint az alkalmazott graceful shutdown időablak. Behind-proxy módban a SIGHUP tranzakciósan újratöltheti a domain/application hosting állapotot, míg process-szintű konfigurációváltozás után továbbra is kontrollált restart kell; pusztán alkalmazásforrás-változást normál esetben az automatikus source-reload supervisor aktivál.

A.6. logrotate minta

```

/var/log/velran/server.log /var/log/velran/access.log {
    daily
    rotate 14
    compress
    delaycompress
    missingok
    notifempty
    create 0640 velran velran
    sharedscripts
    postrotate
        systemctl kill -s HUP --kill-who=main \
            velran.service >/dev/null 2>&1 || true
    endscript
}

/var/log/velran/audit.log {
    daily

```

```

rotate 90
compress
delaycompress
missingok
notifempty
create 0640 velran velran
sharedscripts
postrotate
    systemctl kill -s HUP --kill-who=main \
        velran.service >/dev/null 2>&1 || true
endscript
}

```

A konkrét retention jogi és kockázati döntés. Az auditlog általában hosszabb megőrzést és központi továbbítást érdemel; a minta számai nem univerzális előírások.

A.7. Release előtti production gate

A következő lista szándékosan rövid és végrehajtható. A részletes indoklást az előző fejezetek adják.

1. **Forrás és build:** a release commit azonosított; `Cargo.lock` rögzített; a platform buildje `cargo build -locked -release -p velran-server -p velran-cli`; az alkalmazás forrása `velran-cli check` ellenőrzésen átment.
2. **Migration:** production előtt `migrate status` és `migrate verify`; a jóváhagyott schema változás explicit `migrate apply`; utána újabb `verify`.
3. **Konfiguráció:** a valódi production `server.toml` config-checken átment. Az effective config redaktált kimenete reviewzható.
4. **Secrets:** DB/Redis/auth secret fájlok nem a repositoryban vannak, szűk filesystem jogosultsággal rendelkeznek, és nem kerülnek logba vagy release-recordba.
5. **HTTP edge:** a domain, TLS és Host kezelés helyes. A trusted-proxy lista kizárólag a tényleges proxykat tartalmazza.
6. **Auth:** production cookie policy aktív; local auth/LDAP konfiguráció ellenőrzött; admin/MFA recovery folyamat dokumentált.
7. **CSRF és CORS:** state-changing böngészős kérés CSRF-védett; CORS allowlist explicit; credentialed CORS esetén nincs wildcard origin.
8. **Adatbázis:** az alkalmazás DB user least privilege; pool/budget összhangban van a backend kapacitásával; fontos queryk indexelése ismert.
9. **Storage/upload:** AppFs root confinement és jogosultságok helyesek; upload byte/pixel limitek aktívak; release tartalom nem írható alkalmazásból.
10. **Redis/cache/rate limit:** multi-instance környezetben a közös security/runtime state Redisben van; Redis kiesési viselkedése ismert és tesztelt.
11. **Egress:** csak named outbound target engedélyezett; host/CIDR/port/TLS/timeout/méret policy explicit; secret nem kerül URL-be vagy logba.
12. **Erőforrások:** request/runtime limitek és process hard limitek konzisztensen be vannak állítva; cgroup authorityként systemd vagy külön delegált Velran cgroup legyen, ne mindkettő.
13. **Observability:** server/access/audit log írható; request ID működik; metrics és readiness elérhető a megfelelő belső hálózatról; riasztási minimum definiált.
14. **Backup/recovery:** van friss, ellenőrzött DB + AppFs + identity backup; recovery manifest rendelkezésre áll; restore drill nem csak papíron létezik.
15. **Deploy/rollback:** a release artifact és checksum ismert; a régi release visszaállítható;

schema-változásnál tisztázott, hogy app rollback vagy teljes restore lehetséges.

16. **Indítás utáni evidence:** liveness, readiness és legalább egy üzleti smoke kérés sikeres; logban nincs startup fallback vagy váratlan policy-hiba.

Platform-release security evidence

A Velran platform release csak akkor nevezhető release-ready állapotúnak, ha pontosan az a source tree átmegy ezen:

```
cargo fmt --all -- --check
cargo metadata --locked --format-version 1 >/dev/null
cargo check --workspace --locked
cargo test --workspace --locked
./verify.sh
```

A supply-chain envelope egyezzen a `Cargo.lock` és `SUPPLY-CHAIN-CAPABILITIES.txt` állapottal. Release packaging nem generálhat vagy pecsételhet le Cargo szerint stale lockfile-t. Minden dokumentáció/book változás után a release manifestet is újra kell verifikálni.

A.8. Gyors hibadiagnosztikai térkép

Tünet	Első hely, ahol nézd	Következő kérdés
Nem indul a service	config-check, server log	Config/secret/path/policy vagy dependency readiness?
502/504 a proxyban	proxy log, Velran listener/readiness	Fut-e a processz, loopback port helyes-e, readiness zöld-e?
400/415	access log + route body-contract	Malformed input, rossz schema vagy rossz Content-Type érkezett?
401	auth/session audit esemény	Nincs session, lejárt, invalidálva vagy MFA hiányzik?
403	route/object authorization, CSRF/CORS	Ki az actor, melyik policy tagadta meg?
404/405/406	route map + request method/Accept	Létezik a route, jó methodot és elfogadható reprezentációt kér a kliens?
409	optimistic-lock/business state	Közben megváltozott a record, vagy a Changed query nulla sort módosított?
413/431	request hard limitek	Body/upload vagy header lépte túl a konfigurált limitet?
421/426	Host/proxy/TLS policy	Helyes a public host, trusted proxy és az effektív HTTPS?
422	named form rerender	Melyik mező typed decode/validation hibája áll fenn?
429	rate-limit metrics/audit	Melyik limiter és melyik identity/IP bucket fogyott el?

Tünet	Első hely, ahol nézd	Következő kérdés
500	request ID alapján server log	Valódi belső invariáns/internal hiba történt?
503	readiness + server log	DB/Redis/auth/storage vagy request resource limit tette átmenetileg kiszolgálhatatlanná?
DB hiba	DB readiness + query környezet	Kapcsolat, timeout, schema drift, lock vagy constraint?
Fájl/media hiba	AppFs path/jogosultság/limit	Storage root írható, confinement és byte/pixel limit rendben?
Külső API hiba	egress policy + adapter log	DNS, CIDR, port, TLS, timeout vagy response limit zárt?
Log eltűnik rotation után	log fallback metric, service log	SIGHUP eljutott-e, új fájl jogosultsága megfelelő-e?

A.9. Mit vigyél magaddal productionbe?

Öt szabály foglalja össze az operátori modellt:

- HTTP felület csak typed adat, explicit policy és bounded külső input után legyen;
- a stabil policy trusted konfigurációban, a secret pedig secret fájlban maradjon;
- schema-módosítás, release-aktiválás és rollback legyen explicit operátori döntés;
- proxy, Redis, DB, AppFs és outbound integráció a deklarált authority boundary szerint működjön;
- release előtt verifikálj, utána evidence-dzsel bizonyíts healtht, observabilityt, backupot és restore-t.

A.10. Velran webfejlesztői kompetencia-checklist

Ez a lista más kérdésre válaszol, mint a production gate: *képes vagy-e egy kisebb Velran webalkalmazást önállóan felépíteni és üzemeltetni anélkül, hogy vakon receptet másolnál?* Egy kipipált pont azt jelentse, hogy a feladatot meg tudod oldani, a releváns boundaryt el tudod magyarázni, és a legvalószínűbb hibát diagnosztizálni tudod.

Alkalmazásfejlesztési alapkompenciák

- [] El tudok olvasni egy meglévő Velran projektet, és felismerem a page, action, query, route, modul, migration és konfigurációs boundarykat.
- [] Tudok page vagy action handlert hozzáadni, és explicit access policyval route-hoz kötni.
- [] El tudom dönteni, hogy query, form, JSON vagy multipart input illik a feladathoz, és a külső kontroll alatt álló stringeket és collectionöket bounded formában tartom.
- [] Rust-szerű structtal, enummal, `Option<T>`, `Vec<T>` és `Result<T,E>` típussal tudok adatot modellezni legacy Velran írásmód nélkül.
- [] Tudok adatbázis-migrationt hozzáadni, typed queryt írni, és az adatbázis-constraintet összhangban tartani az alkalmazásszintű validációval.
- [] Tudok create, read, update és delete folyamatot írni, és tranzakciót használok, ha több írás egyetlen logikai műveletet alkot.

- [] Felismerem, mikor kell optimistic locking vagy más concurrency szabály a frissebb állapot csendes felülírása helyett.
- [] Egy növekvő alkalmazást modulokra tudok bontani anélkül, hogy gyengíteném a route-, adat- vagy capability-boundarykat.
- [] Tudok typed JSON endpointot készíteni, és külön kezelem a böngészős form-flowt és az API request/response folyamatot.
- [] Tudok adatbázisból érkező Markdownot szerveroldalon renderelni, a Markdownot source of truthként megtartani, és public response cache-t csak valóban nem személyre szabott, megfelelő freshness contracttal rendelkező route-on használok.
- [] A canonical repository-példákat és a Secure Notes checkpointokat használok ellenőrzésre ahelyett, hogy emlékezetből támogatás nélküli szintaxist találnék ki.

Security boundaryk

- [] Értem, hogy az authentication azt bizonyítja, ki az actor, az authorization pedig azt dönti el, hogy az actor végrehajthatja-e ezt a műveletet ezen az objektumon.
- [] Tudatosan tudok választani a public, authenticated, MFA, role, permission, webhook és critical-operation route policyk között.
- [] Védett objektum olvasása vagy módosítása előtt ownership vagy más object-level authorization szabályt érvényesítek.
- [] Principal/tenant szűrést már az adatbázis-querybe teszek, ha egy collection soha nem tartalmazhat más actorhoz tartozó rekordot.
- [] El tudom magyarázni, miért security boundary a korlátlan külső input, upload/media feldolgozás és outbound hálózat, nem egyszerű kényelmi feature.
- [] A CSRF-, CORS-, trusted-proxy-, Host-, cookie- és credential-kezelést külön fogalomként értem, nem egyetlen általános „web security” kapcsolóként.
- [] A gyakori compiler/security diagnosticokat értelmezni tudom, és a megsértett invariánst javítom a check elnyomása helyett.
- [] Egy változtatás reviewjakor meg tudom mondani, mely capability-, filesystem-, database-, network- vagy identity-boundary bővül.

Build, verification és production

- [] Önállóan végig tudom vinni a normál fejlesztési ciklust: módosítás, **velran-cli check**, célzott tesztek, build és a változáshoz tartozó repository verification gate-ek.
- [] A hibakereséshez szükséges mélységben értem a natív útvonalat a verified source-tól az executable IR-en és generált Ruston át a **rustc**, immutable generation és atomic activation lépésekig.
- [] Meg tudom különböztetni az alkalmazás reloadolható állapotát azoktól a konfigurációs vagy infrastruktúra-változásoktól, amelyek process restartot igényelnek.
- [] Migrationt elő tudok készíteni és ellenőrizni anélkül, hogy a schema-módosítást a szerver indulásához kötném.
- [] Request ID alapján tudok server/access/audit logot olvasni, és readiness/metrics segítségével különválasztom az alkalmazáshibát a dependency-hibától.
- [] Explicit release artifacttal, checksummal, konfigurációs review-val, smoke tesztel és rollback döntéssel tudok deployolni.
- [] El tudom mondani, mi tartozik a backupba, izoláltan vissza tudok állítani konzisztens DB/AppFs/identity állapotot, és a restore drillt evidence-nek tekintem, nem pusztá dokumentációnak.
- [] Le tudom futtatni a **./verify.sh**-t, megértem, melyik gate bukott el, meg tudom különböztetni a tanulói példákat a rejection fixture-öktől/manifesztéktől, és kerülöm az olyan

„javítást”, amely csak gyengíti az ellenőrzést.

Hogyan használd a checklistet?

Nem minden parancs bemagolása a cél, hanem az önálló döntés. Ha egy pont bizonytalan, térj vissza a megfelelő Secure Notes checkpointhoz, változtass meg tudatosan egy dolgot, futtasd a releváns verifiert, és mondd el, miért biztonságos az eredmény. Ha a fenti pontok mindegyikét egy kisebb alkalmazáson bizonyítani tudod, elérted azt a gyakorlati alapszintet, amelyet egy Velran webfejlesztőtől elvárunk; a későbbi referenciaanyag ezután mélyít, nem pedig helyettesíti ezt az alapot.

A.11. Gyakorlat: evidence-gyűjtésként futtasd a checklistet

Gyakorlási mód: üzemeltetési scenárió. Használd az előszóban meghatározott üzemeltetési módszert.

Ne emlékeztetből pipáld ki a pontokat. Minden release-safety szempontból releváns elemhez kapcsolj parancseredményt, config review-t, teszteredményt, restore próbát vagy más konkrét evidence-et.

A.12. Tipikus hiba: a checklistet engineering judgment helyettesítőjeként kezeljük

A checklist ismert omission-osztályokat fog meg; ismeretlen architektúráról nem bizonyítja, hogy biztonságos. Állítsd meg a release-t, ha egy pont kétértelmű, és oldd fel a tervezési kérdést ahelyett, hogy optimistán értelmeznéd a checkboxot.

A.13. Végző Secure Notes checkpoint

E függelékkel végezz release-readiness review-t a végző checkpointon:

```
examples/secure-notes/checkpoints/23-release-readiness/main.vrn
```

Kövesd végig az evidence-láncot a forrástól és lockfile-től a baseline migráción, compiler verificationön, konfiguráción és natív aktiváláson át a backup/restore eljárásig. Minden nem bizonyítható pont legyen a következő engineering feladat a production aktiválás előtt.

Zárszó

A könyv célja, hogy a webfejlesztő a nyelvi elemeket ne elszigetelt szintaxisként, hanem egy működő webalkalmazás részeként tanulja meg. A részletes magyar, feature-szintű referencia a projekt `docs/hu/` könyvtárában található; ez a könyv a napi fejlesztési útvonalat adja.